

SEATICS MAPS API INTEGRATION GUIDE

OVERVIEW

The Seatics Maps API provides an interactive maps solution to clients who need support for multiple inventory sources or non-TicketNetwork ticket data. The Seatics Maps API is also a solution for clients who want maximum control over the design and styling of the user interface. Clients have the option of using the pre-built responsive UI with its numerous configuration settings or building their own custom UI.

This Seatics Maps API Integration Guide (“Guide”) is the property of TicketNetwork, Inc. and it is provided to and solely for use by licensees who have paid the required license fee to use the Seatics Maps API (“Licensees”) for so long as such TicketNetwork grants license a licensee to use the Seatics Maps API (the “API”). No other party has the right to view, use, or access this Guide. Licensee’s license includes the right to use this Guide and to make one (1) physical copy of it solely for its internal business use as further described in the Seatics Software License Agreement (the “Agreement”). Any rights not granted to Licensee in this Guide or in the Agreement, including but not limited to, the right to make more than one (1) copy and the right to distribute the Guide shall remain with TicketNetwork. Failure to comply with this paragraph will constitute a breach of the Agreement and may result in TicketNetwork’s termination of Licensee’s license to use the Seatics Software and the API.

CONTENTS

Overview	1
Before Integrating - Demo Site	6
The API Store	7
Getting Started with API Integration	7
Using the Pre-Built UI	7
Building A Custom UI	9
Example code – Creating the Map Component	11
Notes about Event Mapping	12
Adding Ticket Data	12
Example Code – Adding Ticket Data	13
Example Code – Adding Ticket Data With Disclaimers	15
Special Ticket Group Types	16
Split Rules	16
Splits Bitmap	17
Service Fee Calculation	18
Adding Ticket Data from TicketNetwork Standard Webservices	18
Adding Ticket Data With Disclaimers From ticketnetwork standard webservices	20
Adding Ticket Data from TicketNetwork Mercury Webservices	22
Adding Ticket Data from TicketNetwork Mercury REST API	22
Integrating Maps with Checkout	24
Integrating Maps into an App	25
Additional Setup	25
Configuration Options for the Pre-Built UI	25
Maps Configuration Management Tool	25
Configuration in Code	26
Customizing Styles	32
Configuration Options for the Custom UI	33
Sorting & Filtering in the Custom UI	35
Example Code – Extending Filter Options	36
Expanding the Quantity Filter	36

Example Code – Extending Sorting Options	37
Interactions between the List and the Map in The Custom UI.....	37
Example Code – Highlighting Map Sections	38
Adding View from Section Images to The Custom UI	39
Example Code – Using VFS Images.....	39
Enabling Section Info Pop-ups in the Custom UI.....	39
Example Code – Enabling Section Info Pop-ups	40
Pop-up ViewPort	41
Example Code – Allow Pop-ups to Display Anywhere in Browser Window	41
Example Code – Enabling Section Info Pop-ups with Custom getMapViewport.....	42
Currency Symbol Support	43
Inventory Disclaimers.....	43
Example Code – Adding Disclaimers to Ticket Groups.....	43
Configuring the Map Legend in the Custom UI.....	45
When a Map Is Unavailable	45
TicketNetwork Zone Maps	46
User Event Tracking.....	46
Example Code – User Event Tracking	46
Pre-Filtering Settings	47
Filtering by Ticket Type	48
Filtering by Delivery Type.....	48
Filtering by Quantity	48
Filtering by Price	49
Going Live.....	49
Support	50
Feedback and Feature Requests	50
Appendix.....	51
Pre-Built UI Screenshots.....	51
Smartphone Layout	51
Tablet Layout.....	55
DeskTop Layout	58

Configuration Details.....	60
Currency Symbol Configuration	60
Dark Theme.....	61
H1 Heading Tag	63
Mobile Header Configuration	63
Large Screen Disclosures Customization	64
Mobile Disclosures Customization	65
Face Value Display.....	68
Custom Google Reseller Certification View	71
All-In Pricing / Fee Display.....	74
Quantity Modal	78
Checkout Deep Linking	79
Feedback Feature	88
Map Color Schemes	89
Section Selection Schemes	93
Small Screen Layout Configuration	94
Scroll to Zoom	97
Zoom Controls.....	98
Map Container.....	99
Legend Display.....	100
Alternate Legend Presentation	101
Large Screen Maps Placement.....	103
CTA Behavior on Mobile.....	104
CTA Behavior on Large Screen	105
Forced Quantity Selection	106
CTA in the Ticket Group Row	107
Featured Inventory Display.....	108
Unmapped Inventory Display.....	109
Quantity Display	111
Price Display	111
Enabling My List.....	112

Pre-Checkout Screen Customization	113
Default Sort	114
Saving Filter Settings.....	115
Local Pickup Filter	115
Packages Marketing	116
No Tickets Handler	121
Special Functions	122
Custom Levels.....	123
Customizing Seatics.Text	124
Change Table.....	125

BEFORE INTEGRATING - DEMO SITE

To test out the Seatix Maps API before integrating, you may experiment with our demo pages.



Please note:

- These demos were created solely to allow potential clients to get an idea of what the product can do before making a licensing decision. We do not recommend building your implementation on top of the demo code.
- These demos use production data, not sandbox. Not all production events are available in the sandbox environment and production event IDs do not match sandbox event IDs. Our support team will provide you with a list of events you can use for testing in the sandbox environment.

Pre-Built UI Demo: <http://s3.amazonaws.com/tn-maps-api-demo/DemoPage.html>.

This page provides an example of the minimal code needed to integrate with the API. It uses our default layout and styles. You may edit the HTML to toggle between configuration settings. You can also test loading other events; however, if you use another venue, the inventory might not match unless you replace the TicketData.js file with another dummy ticket data file.

Custom UI Demo: <http://s3.amazonaws.com/tn-maps-api-demo/CustomUI/DemoPage.html>.

This page provides an example of how to get the maps components and use them to form a custom list and header. There are three JavaScript files it uses:

1. <http://s3.amazonaws.com/tn-maps-api-demo/CustomUI/ticketList.js> for the ticket list construction
2. <http://s3.amazonaws.com/tn-maps-api-demo/CustomUI/filterControls.js> for the filter construction
3. <http://s3.amazonaws.com/tn-maps-api-demo/CustomUI/mapsInterfaceCustomUI.js> to tie it all together

Again, you can test loading other events, but if you use another venue, the inventory may not match unless you replace the TicketData.js file with another dummy ticket data file.



IMPORTANT: If you are considering doing a custom UI implementation, please remember that you are responsible for the development of your user interface. Our support staff cannot give you instructions or guidance on designing or building your custom UI.

THE API STORE

After your licensing agreement is finalized, you will be granted access to the API Store. If you do not yet have a broker ID (BID) and website configuration ID (WCID), those will be created for you. Our support team will create your sandbox API Store account and set up your roles and permissions. They will then send you login information and documentation for the API Store. They will also provide you with a list of events that are available to test with in the sandbox environment.

GETTING STARTED WITH API INTEGRATION

First, you will need to decide whether to use our pre-built UI or build a custom UI. The pre-built UI offers a large variety of configuration options, making it customizable and appropriate for the majority of clients. If the pre-built UI doesn't meet your needs, you also have the option of building a custom UI.



IMPORTANT: If you are considering doing a custom UI implementation, please remember that you are responsible for the development of your user interface. Our support staff cannot give you instructions or guidance on designing or building your custom UI.

Please note that web hosting is required to begin your integration. You will not be able to run your implementation locally.

USING THE PRE-BUILT UI

The following libraries must be included on your map page if your site isn't already using them:

- jQuery (earliest supported version: 3.6.0)

To access the **sandbox** version of the responsive maps, use an API call to:

https://sandbox.tn-apis.com/maps/v3/MapAndLayout?websiteConfigId=690&consumerKey=iLKV6Bo_HOHfObC7EurIGZpxdSga&eventName=The+Week+nd&venue=Allstate+Arena&dateTime=201901202000

To access the **production** version of the responsive maps (**which you will be blocked from until our support staff enables your account and generates your production consumer key**), use an API call to:

https://www.tn-apis.com/maps/v3/MapAndLayout?websiteConfigId=690&consumerKey=BViE7U7RBisT4ud5IZjwQYi_RNia&eventName=Hamilton&venue=Forrest+Theatre&dateTime=201911171830



IMPORTANT: Your production consumer key will be different than your sandbox consumer key. Your production consumer key must be generated by our support staff and will not be made available to you until our team has tested your sandbox implementation and approved you to go live.

Replace the highlighted parameters in the above calls with information specific to the site and event (see below table). The result should be placed in the middle of the map page.

Parameter	Required?	Description	Data Type	Notes
websiteConfigId	Required	The website ID of the site	number	If you don't already have a web config ID, we'll set one up for you as part of your implementation
consumerKey	Required	Your unique API key	string	Your sandbox consumer key will be different from your production consumer key. Your production key can only be generated by our support staff.
eventId	Optional, recommended if known	The TicketNetwork event ID, if known	number	
<i>If eventId isn't known, substitute with:</i>				
eventName	Optional, recommended if not using eventId	The name of the event	string	
venue	Optional, recommended if not using eventId	The name of the venue where the event takes place	string	
dateTime	Required if not using eventId	The date and time of the event, formatted yyyyMMddHHmm	number	If the time of the event is yet to be determined, use the format yyyyMMdd

The UI components expect to be placed in a container. You can set the width and the height of this container. The UI components will fill up to 1500px of width. The recommended height of the container is 900px. It can be set smaller or larger than 900px, if desired.

This product is meant to be a single integration solution that adapts to the user's screen size and displays a layout optimized for their device. To use the responsive pre-built UI with a responsive site, you will need to add CSS to hide the full site header and footer when the layout goes from desktop to mobile. The small screen layout will fill the entire browser window. Adding other elements to the mobile page may throw off the layout calculations and cause the UI to render incorrectly.

The desktop layout is served when the browser window is wider than 991 pixels. Below is an example CSS declaration that could be used to trigger the hiding of your site's header and footer when the browser window is narrower than 992 pixels:

```
@media screen and (max-width: 991px) {  
  
    /* Hide your header and footer here */  
}
```



```
}
```

A stacked layout (map on top, ticket list below) displays when the screen is less than 992px wide and the ratio of width to height is 13:9 or below. A side by side 'mobile landscape' layout displays when the screen is less than 992px wide and the ratio of width to height is greater than 13:9. A side by side 'desktop' layout displays when the screen is greater than 992px wide.

The pre-built UI includes the event info bar, the map region, filtering and sorting controls, and the ticket group list. [Pre-Built UI Screenshots](#) are available in the Appendix.

BUILDING A CUSTOM UI



IMPORTANT: If you are considering doing a custom UI implementation, please remember that you are responsible for the development of your user interface. Our support staff cannot give you instructions or guidance on designing or building your custom UI.

The following libraries must be included on your map page if your site isn't already using them:

- jQuery (earliest supported version: 3.6.0)
- The Mapwidget JavaScript framework (can be cached but should be refreshed at least hourly)
 - Sandbox: <https://mapwidget3-sandbox.seatics.com/Api/framework>
 - Production: <https://mapwidget3.seatics.com/Api/framework>

To access the **sandbox** version of the maps, use an API call to:

https://sandbox.tn-apis.com/maps/v3/EventAndVenueInfo?callback=something&websiteConfigId=690&consumerKey=iLKV6Bo_H0Hf0bC7EurIGZpXdsGa&eventName=The+Weeknd&venue=Allstate+Arena&dateTime=201901202000

To access the **production** version of the maps (**which you will be blocked from until our support staff enables your account and generates your production consumer key**), use an API call to:

<https://www.tn-apis.com/maps/v3/EventAndVenueInfo?callback=something&websiteConfigId=690&consumerKey=BViE7U7RBisT4ud5IZjwQY1RNia&eventName=Hamilton&venue=Forrest+Theatre&dateTime=201911171830>



IMPORTANT: Your production consumer key will be different than your sandbox consumer key. Your production consumer key must be generated by our support staff and will not be made available to you until our team has tested your sandbox implementation and approved you to go live.

Replace the highlighted parameters in the above calls with information specific to the site and event (see below table). The result should be placed in the middle of the map page.

Parameter	Required?	Description	Data Type	Notes
callback	Required	The callback function to use	function	
websiteConfigId	Required	The website ID of the site	number	If you don't already have a web config ID, we'll set one up for you as part of your implementation
consumerKey	Required	Your unique API key	string	Your sandbox consumer key will be different from your production consumer key. Your production key can only be generated by our support staff.
eventId	Optional, recommended if known	The TicketNetwork event ID, if known	number	
<i>If eventId isn't known, substitute with:</i>				
eventName	Optional, recommended if not using eventId	The name of the event	string	
venue	Optional, recommended if not using eventId	The name of the venue where the event takes place	string	
dateTime	Required if not using eventId	The date and time of the event, formatted yyyyMMddHHmm	number	If the time of the event is yet to be determined, use the format yyyyMMdd

The endpoint sends an array to the callback function. The first element in the array will be the `eventInfo` object and the second element will be the `mapData` object if there is interactive map data associated with the event.

Once map data is retrieved, the map component can be created in a container specified by the website. Use the function `Seatics.MapComponent.create()`. It takes a single object type config parameter.

EXAMPLE CODE – CREATING THE MAP COMPONENT

```
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
  },
  mapWidth: 525,
  mapHeight: 545,
  mapName: eventData.mapName,
  eventInfo: eventData
});
```

Most of these parameters should be kept as is.

eventData and **mapData** are the objects retrieved by the EventAndVenueInfo call.

Container is the HTML element within which the map will be constructed. This should be a div.

mapWidth and **mapHeight** define the area in which the map is allowed to move around.

Tickets is an array of ticket objects as explained in the [Adding Ticket Data](#) section of this guide. Note that tickets may also be added after the creation using the function `Seatics.MapComponent.addTicketData()`.

presentationInterface is an object that dictates the interaction with your presentation layer and the map component. The one function that is important to implement in this object is `updateTicketsList()`. This gets called by the map component after various interactions (as well as first load) with an array of segments of ticket groups. Each segment is an object that corresponds to a ticket group type (e.g. parking, passes) and has two properties:

Property	Description	Data Type	Notes
title	The title of this segment	string	Empty for standard event ticket type
tickets	An array of ticket group objects grouped in this segment	array of ticketGroup	

NOTES ABOUT EVENT MAPPING

To retrieve the correct seating chart, the API must map client-supplied events to existing events on the TicketNetwork Exchange. If you know the TicketNetwork event ID for an event, passing that will result in an exact match. In most cases, however, the API will use a combination of event name, venue name, and event date/time data to determine the event mappings. The event mapping logic works roughly as follows:

1. First, the API does a search using event name, venue name, and event date/time. Event name and venue name are compared against all existing aliases, so they do not have to be an exact match to TicketNetwork's naming conventions. The most important data point the API looks at is the date/time of the event.
2. If there is no match, the API discards the time and searches the entire date for the event name and venue name.
3. If there are multiple results, the API attempts to select an event with a venue name that is an exact match to the venue name on our Exchange.
4. If the API can't narrow down the results to a single match or can't determine any match, a static "Map Coming Soon" image displays alongside the inventory and the event information you provide. (See [When a Map Is Unavailable](#) for more information about how to override this image.)

If an interactive map is not displaying for the event and you believe one is available, the most likely reason is discrepancies between the data supplied to the API and our Exchange data, such as a combination of different event start time and an event name or venue name that doesn't match any of our aliases. Though we regularly make improvements to our event mapping logic, data quality is essential to the success of finding an accurate match.

ADDING TICKET DATA

If you are using the pre-built UI, to add inventory to the maps, use the function `Seatics.addTicketData()`. In addition to hooking the tickets into the map, this will also build the tickets list, as well as initialize additional pre-built UI components, such as filtering and ticket criteria selection elements.

If you have chosen to create your own custom UI, use the function `Seatics.MapComponent.addTicketData()` to add inventory to the maps.

For both the pre-built UI and custom UI integrations, call `addTicketData()` at any time **below** the Maps script. If you use a callback to get your ticket data, call `addTicketData()` after the data is received. Both overloads of this function take an array of ticket group objects. Each object can have the following properties:

Property	Required?	Description	Data Type	Notes
<code>tgUserSec</code>	Required	The section the tickets are located in	string	
<code>tgUserRow</code>	Required	The row the tickets are located in, if applicable	string	You can pass an empty string, but the property needs to be there.

tgUserSeats	Optional	The seat number range of inventory (e.g. 101-105)	string	Seat numbers are used for section matching for some venues, but do not display
tgQty	Required	The number of tickets available in the ticket group	number	
tgPrice	Required	The price per ticket	number	
tgID	Optional	A unique ID for the ticket group	number	
tgNotes	Optional	Notes to be displayed with the ticket group (e.g. "Obstructed View")	string	
tgMark	Optional	Use 1 to mark tickets as a featured offer	number	
tgType	Optional, but recommended	The ticket group type ID	number	See additional notes about ticket types like parking and packages
tgDeliveryOptions	Optional	Special shipping options notation	string	Use "EM" for e-ticket, "ID" for instant download, "MD" for mobile delivery, and "LP" for local pickup. Values can be combined (e.g. EMID)
tgFaceValue	Optional	The face value of the tickets, if known	number	
tgPriceSlash	Optional	A "slashed" price to show next to the current price	number	
tgSplitRuleId	Optional	Rule id for valid ticket splits	number	See additional notes about ticket splits
tgClientData	Optional	Extra data you'd like passed with the ticket group	any type	
tgCType	Optional	Relevant only if using TicketNetwork ticket data with zones enabled	string	Use "Z" for zone, "C" for category, and "R" for real
tgDisclaimers	Optional	Legal disclosures to show in a pop-up for the ticket group	array of strings	
tgServiceFee	Optional	The flat per-ticket service fee for this ticket group	Number	Use this OR tgServiceFeeCalculation when providing fees
tgServiceFeeCalculation	Optional	A string representation of a mathematical function used to calculate the service fee for the ticket group		See additional notes about service fee calculation

EXAMPLE CODE – ADDING TICKET DATA

The pre-built UI using `Seatics.addTicketData()`.

```
Seatics.addTicketData([
  {tgUserSec:"Section 100", tgUserRow:"C", tgUserSeats:"5-6", tgQty:2,
    tgPrice:55.00, tgID:1, tgNotes:"Comes with a free soda", tgMark:1, tgType:1,
    tgClientData:"SomeTrackingCode", tgCType:"R"},
  {tgUserSec:"Section 122", tgUserRow:"T", tgUserSeats:"7-8", tgQty:2, tgPrice:34.00, tgID:2,
    tgNotes:"Obstructed view", tgMark:0, tgType:1, tgClientData:"SomeTrackingCode2", tgCType:"R"},
  {tgUserSec:"Section 204", tgUserRow:"Z", tgQty:4, tgPrice:22.00, tgID:3, tgMark:0, tgType:1,
    tgClientData:"SomeTrackingCode3", tgCType:"C", tgDisclaimers:["This seller is committing to procure
```

```
these tickets for you upon receipt of your order. If the seller is unable to provide the tickets or
equivalent or better ones, you will receive a full refund."}}],
{tgUserSec:"Section 300", tgUserRow:"", tgQty:2, tgPrice:14.00, tgID:4, tgMark:0, tgType:1,
tgClientData:"SomeTrackingCode4", tgCType:"C"}, tgDisclaimers: ["This seller is committing to
procure these tickets for you upon receipt of your order. If the seller is unable to provide the
tickets or equivalent or better ones, you will receive a full refund."]]});
```

A custom UI using `Seatics.MapComponent.addTicketData()`.

```
Seatics.MapComponent.addTicketData([
{tgUserSec:"Section 100", tgUserRow:"C", tgUserSeats:"5-6",
tgQty:2, tgPrice:55.00, tgID:1, tgNotes:"Comes with a free soda", tgMark:1, tgType:1,
tgClientData:"SomeTrackingCode", tgCType:"R"},
{tgUserSec:"Section 122", tgUserRow:"T", tgUserSeats:"7-8", tgQty:2, tgPrice:34.00, tgID:2,
tgNotes:"Obstructed view", tgMark:0, tgType:1, tgClientData:"SomeTrackingCode2", tgCType:"R"},
{tgUserSec:"Section 204", tgUserRow:"Z", tgQty:4, tgPrice:22.00, tgID:3, tgMark:0, tgType:1,
tgClientData:"SomeTrackingCode3", tgCType:"C", tgDisclaimers:["This seller is committing to procure
these tickets for you upon receipt of your order. If the seller is unable to provide the tickets or
equivalent or better ones, you will receive a full refund."}},
{tgUserSec:"Section 300", tgUserRow:"", tgQty:2, tgPrice:14.00, tgID:4, tgMark:0, tgType:1,
tgClientData:"SomeTrackingCode4", tgCType:"C"}, tgDisclaimers: ["This seller is committing to
procure these tickets for you upon receipt of your order. If the seller is unable to provide the
tickets or equivalent or better ones, you will receive a full refund."]]});
```

You can also supply disclaimers in the `addTicketData()` call by passing them in as the second parameter for `Seatics.addTicketData()`, and as the third parameter for `Seatics.MapComponent.addTicketData()`. Disclaimers should be an object, with the key for each disclaimer being the disclaimer ID. Each individual disclaimer should also be an object, one that includes the disclaimer text and location(s) to which the disclaimer applies.

Property	Required?	Description	Data Type	Notes
<code>text</code>	Required	The full text of the disclaimer	string	You can pass an empty string, but the property needs to be there.
<code>loc</code>	Required	An array of arrays with each inner array indicating a location that the disclaimer applies to. Each location is made of country ID, state/province ID, and city ID, if applicable.	array	Array of arrays requiring at least 1 inner array be there, but that inner array can be an empty array.
<code>loc[0][0]</code>	Optional	String containing the ISO code of the country that the disclaimer applies to.	string	Empty string should be provided when the disclaimer is not for a specific country.
<code>loc[0][1]</code>	Optional	String containing the ISO code of the state or province that the disclaimer applies to.	string	Empty string should be provided when the disclaimer is not for a specific state or country.

				If a value is provided, then a country is also required.
<code>loc[0][2]</code>	Optional	String containing the name of the city that the disclaimer applies to.	string	Empty string should be provided when the disclaimer is not for a specific city. If a value is provided, then a country and state/province is also required.

EXAMPLE CODE – ADDING TICKET DATA WITH DISCLAIMERS

The pre-built UI using `Seatics.addTicketData()`.

```
Seatics.addTicketData([
  {tgUserSec:"Section 100", tgUserRow:"C", tgUserSeats:"5-6", tgQty:2,
    tgPrice:55.00, tgID:1, tgNotes:"Comes with a free soda", tgMark:1, tgType:1,
    tgClientData:"SomeTrackingCode", tgCType:"R"},
  {tgUserSec:"Section 122", tgUserRow:"T", tgUserSeats:"7-8", tgQty:2, tgPrice:34.00, tgID:2,
    tgNotes:"Obstructed view", tgMark:0, tgType:1, tgClientData:"SomeTrackingCode2", tgCType:"R"},
  {tgUserSec:"Section 204", tgUserRow:"Z", tgQty:4, tgPrice:22.00, tgID:3, tgMark:0, tgType:1,
    tgClientData:"SomeTrackingCode3", tgCType:"C", tgDisclaimers:["This seller is committing to procure
    these tickets for you upon receipt of your order. If the seller is unable to provide the tickets or
    equivalent or better ones, you will receive a full refund."]},
  {tgUserSec:"Section 300", tgUserRow:"", tgQty:2, tgPrice:14.00, tgID:4, tgMark:0, tgType:1,
    tgClientData:"SomeTrackingCode4", tgCType:"C"}, tgDisclaimers: ["This seller is committing to
    procure these tickets for you upon receipt of your order. If the seller is unable to provide the
    tickets or equivalent or better ones, you will receive a full refund."]}, { 3: { text: "The
    tickets you receive may be in the specific section listed here or in an equivalent or better
    location.", loc:[["US", "NY", ""]]}}]);
```

A custom UI using `Seatics.MapComponent.addTicketData()`.

```
Seatics.MapComponent.addTicketData([
  {tgUserSec:"Section 100", tgUserRow:"C", tgUserSeats:"5-6",
    tgQty:2, tgPrice:55.00, tgID:1, tgNotes:"Comes with a free soda", tgMark:1, tgType:1,
    tgClientData:"SomeTrackingCode", tgCType:"R"},
  {tgUserSec:"Section 122", tgUserRow:"T", tgUserSeats:"7-8", tgQty:2, tgPrice:34.00, tgID:2,
    tgNotes:"Obstructed view", tgMark:0, tgType:1, tgClientData:"SomeTrackingCode2", tgCType:"R"},
  {tgUserSec:"Section 204", tgUserRow:"Z", tgQty:4, tgPrice:22.00, tgID:3, tgMark:0, tgType:1,
    tgClientData:"SomeTrackingCode3", tgCType:"C", tgDisclaimers:["This seller is committing to procure
    these tickets for you upon receipt of your order. If the seller is unable to provide the tickets or
    equivalent or better ones, you will receive a full refund."]},
  {tgUserSec:"Section 300", tgUserRow:"", tgQty:2, tgPrice:14.00, tgID:4, tgMark:0, tgType:1,
    tgClientData:"SomeTrackingCode4", tgCType:"C"}, tgDisclaimers: ["This seller is committing to
```

```
procure these tickets for you upon receipt of your order. If the seller is unable to provide the
tickets or equivalent or better ones, you will receive a full refund.]]], null, { 3: { text: "The
tickets you receive may be in the specific section listed here or in an equivalent or better
location.", loc:[["US", "NY", ""]]}}});
```

SPECIAL TICKET GROUP TYPES

The maps recognize and handle various ticket group types differently. Identifying your inventory as the right ticket group type helps it to display, sort, and filter correctly. If the `tgType` property is not specified for a ticket group, the API assumes it is an event ticket¹.

tgType	Description	Notes
1	Event Ticket	Standard event entry
2	Suite	Allows event entry
3	Parking Pass	Does not include event entry
4	Hotel Deal	Does not include event entry
5	Ticket Package	Allows event entry
6	Backstage Pass	Does not include event entry
7	Pit Pass	Does not include event entry
8	VIP Pass	Does not include event entry
9	Meet/Greet Pass	Does not include event entry
10	General Pass	Does not include event entry

SPLIT RULES

Our system recognizes 2 ways to define valid splits for tickets. The first way is via an ID that identifies what “rule” is used to generate the splits. Split rules determine what quantities of tickets customers can purchase from a ticket group. If the `tgSplitRuleId` property is not specified for a ticket group and the caller doesn’t use the second way to specify splits (aka by bitmap – covered in the next section), the API defaults it to split rule ID 1.

Split rule ID 1

- No singles – Customers cannot purchase a quantity of tickets that leaves a single ticket in the ticket group (e.g. customer can’t purchase 5 of 6).
- Evens only for 4 packs – Customers can purchase only quantities of 2 or 4 from a ticket group with 4 tickets left.

Split rule ID 2

¹ There is logic in the section matching algorithm that attempts to find and recategorize inventory that should be listed as parking, but it’s best to assume that the API does not know your ticket groups are special inventory types unless you tell it.

- Allow all splits

Split rule ID 3

- No Splits Allowed (ticket group must be sold in its full quantity)

Split rule ID 4

- Split Multiples of 2 (sold quantity must be a multiple of 2 i.e. in an 8-pack valid splits are 2,4,6,8)

Split rule ID 5

- Split Multiples of 3 (sold quantity must be a multiple of 3 i.e. in a 9-pack valid splits are 3,6)

Split rule ID 6

- Split Multiples of 4 (sold quantity must be a multiple of 4 i.e. in an 8-pack valid splits are 4,8)

Split rule ID 7

- Split Multiples of 5 (sold quantity must be a multiple of 5 i.e. in a 10-pack valid splits are 5,10)

SPLITS BITMAP

The other way to specify ticket splits is via the `tgSplitsBitmap` attribute on the ticket group object. This property should be an integer bitmap of the combined valid splits for the ticket group. In order to calculate the bitmap for a set of splits, convert each valid quantity into the bitmap integer for that value, then add the resulting values together.

To calculate the bitmap value for a single valid quantity you can perform a left-shift operation, followed by a right-shift operation on that quantity:

$$1 \ll qty \gg 1$$

OR you can perform a single left-shift operation on the quantity minus 1.

$$1 \ll qty - 1$$

For example, for a quantity split of 2, 3, and 4 the `tgSplitsBitmap` would be 14.

$$1 \ll 2 - 1 = 2$$
$$1 \ll 3 - 1 = 4$$
$$1 \ll 4 - 1 = 8$$
$$2 + 4 + 8 = 14$$

SERVICE FEE CALCULATION

If there is a flat fee per ticket, then supply that as the `tgServiceFee` property on the ticket group. For example, if the service fee for a single ticket is \$5, and the service fee for 2 tickets is \$10, then set `tgServiceFee` to 5.

However, if the fee calculation is more complicated, then provide a mathematical function string defining the calculation logic as the ticket group's `tgServiceFeeCalculation`. This string will be parsed and used to dynamically calculate the service fee as the ticket quantity selection changes.

Example:

```
"{qty} * .35 + 2.5"
```

Supports:

Parameter/Characters	Used For
{qty}	Replaced with the currently selected ticket quantity
*	Multiplication
/	Division
+	Addition
-	Subtraction and negative numbers
()	Order of operations
0 through 9	Numbers
.	Decimal places

The most common configuration option for service fee is to charge a percentage of the ticket price. For a 25% service fee on a \$20 ticket, use the following string for `tgServiceFeeCalculation`: `"20 * {qty} * 0.25"`

In this kind of formula, replace the ticket price portion of the formula with the ticket's `tgPrice`. To find out the number to use for service fee, simply divide your service fee by 100. For example, a 33% service fee would be `tgPrice + " * {qty} * 0.33"`. A 40% service fee would be `tgPrice + " * {qty} * 0.40"`. Etcetera.

ADDING TICKET DATA FROM TICKETNETWORK STANDARD WEBSERVICES

Use the Webservices `GetEventTickets3` call to get TicketNetwork inventory. Below is sample output. Fields that you may need to pass to the Mapwidget API are highlighted.

```
<?xml version="1.0" encoding="UTF-8"?>
<EventTickets3 xmlns="
http://tnwebservices.ticketnetwork.com/tnwebservice/v3.2" xmlns:xsi="
http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="
http://www.w3.org/2001/XMLSchema">

  <Tickets>
    <TicketGroup3>

      <EventID>999</EventID>
```

```

<FacePrice>1.00</FacePrice>
<HighSeat>100</HighSeat>
<ID>9999</ID>
<IsMine>true</IsMine>
<LowSeat>1</LowSeat>
<Marked>false</Marked>
<Notes>Test</Notes>
<ParentCategoryID>0</ParentCategoryID>
<Rating>1</Rating>
<RatingDescription>TEST</RatingDescription>
<RetailPrice>1.00</RetailPrice>
<Row>TEST1</Row>
<TicketQuantity>100</TicketQuantity>
<ValidSplits><int>100</int> <int>98</int></ValidSplits>
<TicketGroupType>TEST</TicketGroupType>
<CurrencyTypeAbbr>USD</CurrencyTypeAbbr>
<ConvertedActualPrice>1.00</ConvertedActualPrice>
<IsMercury>false</IsMercury>
<DeliveryOptions>EM,LP</DeliveryOptions>
<ActualPrice>1</ActualPrice>
<Section>TEST</Section>
<WholesalePrice>1.00</WholesalePrice>
<PreDiscountPrice>2.00</PreDiscountPrice>

</TicketGroup3>
</Tickets>

```

The below table shows how the GetEventTickets3 output matches up with the Seatix Maps API addTicketData() input. Please note that you will have to convert some of the output into a different data type before passing it to the Map API.

GetEventTickets3 Output	addTicketData() Input	Notes
Section	tgUserSec	
Row	tgUserRow	
HighSeat LowSeat	tgUserSeats	HighSeat and LowSeat will be expressed as a range for tgUserSeats (e.g. 101-102)
TicketQuantity	tgQty	
ConvertedActualPrice	tgPrice	
ID	tgID	
Notes	tgNotes	
Marked	tgMark	Convert from a Boolean to a number (0 or 1)
TicketGroupType	tgType	Convert from a string to a number; use above Special Ticket Types table to determine ticket group type ID
deliveryOptions	tgDeliveryOptions	
PreDiscountPrice	tgPriceSlash	
ValidSplits	tgSplitRuleId	Use the array of valid quantities to determine split rule ID or use 1 as a default

 tgClientData

 tgCType

 tgDisclaimers

ADDING TICKET DATA WITH DISCLAIMERS FROM TICKETNETWORK STANDARD WEBSERVICES

Use the Webservices GetEventTickets4 call to get TicketNetwork inventory. Below is sample output. Fields that you may need to pass to the Mapwidget API are highlighted.

```
<?xml version="1.0" encoding="UTF-8"?>
<EventTickets4 xmlns="
http://tnwebservices.ticketnetwork.com/tnwebservice/v4.0 xmlns:xsi="
http://www.w3.org/2001/XMLSchema-instance xmlns:xsd="
http://www.w3.org/2001/XMLSchema">

<Tickets>
<TicketGroup4>

<EventID>999</EventID>
<FacePrice>1.00</FacePrice>
<HighSeat>100</HighSeat>
<ID>9999</ID>
<IsMine>true</IsMine>
<LowSeat>1</LowSeat>
<Marked>false</Marked>
<Notes>Test</Notes>
<ParentCategoryID>0</ParentCategoryID>
<Rating>1</Rating>
<RatingDescription>TEST</RatingDescription>
<RetailPrice>1.00</RetailPrice>
<Row>TEST1</Row>
<TicketQuantity>100</TicketQuantity>
<ValidSplits><int>100</int> <int>98</int></ValidSplits>
<TicketGroupType>TEST</TicketGroupType>
<CurrencyTypeAbbr>USD</CurrencyTypeAbbr>
<ConvertedActualPrice>1.00</ConvertedActualPrice>
<IsMercury>false</IsMercury>
<DeliveryOptions>EM,LP</DeliveryOptions>
<ActualPrice>1</ActualPrice>
<Section>TEST</Section>
<WholesalePrice>1.00</WholesalePrice>
<PreDiscountPrice>2.00</PreDiscountPrice>
<JurisdictionDisclaimerIDs><int>3</int><int>16</int></JurisdictionDisclaimerIDs>
```

```

</TicketGroup4>
</Tickets>

<PossibleJurisdictionDisclaimers>
<JurisdictionDisclaimerInfo>
<JurisdictionDisclaimerId>3</JurisdictionDisclaimerId>
<DisclaimerText>The tickets you receive may be in the specific section listed here or in an equivalent
or better location. This listing describes tickets that the seller does not own or may not know the exact
location of the seats, but is offering to obtain for you. Once your order is confirmed, we guarantee that
you will receive tickets prior to the start of the event for seats that are in an equivalent or better
location or you will receive a full refund.</DisclaimerText>
<EnforcementLocations>
<JurisdictionEnforcementLocation>
<CountryCode>US</CountryCode>
<StateIsoCode>NY</StateIsoCode>
<City>Buffalo</City>
<CountryId>217</CountryId>
<StateId>35</StateId>
<CityId>645</CityId>
</JurisdictionEnforcementLocation>
</EnforcementLocations>

</JurisdictionDisclaimerInfo>
</PossibleJurisdictionDisclaimers>

```

The below table shows how the GetEventTickets4 output matches up with the Seatix Maps API addTicketData() input. **Please note that you will have to convert some of the output into a different data type before passing it to the Map API.**

GetEventTickets4 Output	addTicketData() Input	Notes
Section	tgUserSec	
Row	tgUserRow	
HighSeat LowSeat	tgUserSeats	HighSeat and LowSeat will be expressed as a range for tgUserSeats (e.g. 101-102)
TicketQuantity	tgQty	
ConvertedActualPrice	tgPrice	
ID	tgID	
Notes	tgNotes	
Marked	tgMark	Convert from a Boolean to a number (0 or 1)
TicketGroupType	tgType	Convert from a string to a number; use above Special Ticket Types table to determine ticket group type ID
deliveryOptions	tgDeliveryOptions	
PreDiscountPrice	tgPriceSlash	

ValidSplits	tgSplitRuleId	Use the array of valid quantities to determine split rule ID or use 1 as a default
	tgClientData	
	tgCType	
	tgDisclaimers	
JurisdictionDisclaimerId		Used as the key for the disclaimer object
DisclaimerText	text	
JurisdictionEnforcementLocation.CountryCode	loc[i][0]	
JurisdictionEnforcementLocation.StateIsoCode	loc[i][1]	
JurisdictionEnforcementLocation.City	loc[i][2]	

ADDING TICKET DATA FROM TICKETNETWORK MERCURY WEBSERVICES

Use the Mercury Webservices GetTickets2 method to get TicketNetwork inventory. The below table shows how the GetTickets2 output matches up with the Mapwidget API addTicketData() input. **Please note that you will have to convert some of the output into a different data type before passing it to the Mapwidget API.**

GetTickets2 Output	addTicketData() Input	Notes
Section	tgUserSec	
Row	tgUserRow	
HighSeat LowSeat	tgUserSeats	HighSeat and LowSeat will be expressed as a range for tgUserSeats (e.g. 101-102)
TicketQuantity	tgQty	
WholesalePrice	tgPrice	You will need to build in your own markups on wholesale price and then use the new price value for the tgPrice input
ID	tgID	
Notes	tgNotes	
	tgMark	
TicketGroupType	tgType	Convert from a string to a number; use above Special Ticket Types table to determine ticket group type ID
ShippingMethodsAvailable	tgDeliveryOptions	
	tgPriceSlash	
ValidSplits	tgSplitRuleId	Use the array of valid quantities to determine split rule ID or use 1 as a default
	tgClientData	
	tgCType	
	tgDisclaimers	

ADDING TICKET DATA FROM TICKETNETWORK MERCURY REST API

Below is sample output. Fields that you need to pass to the Mapwidget API are highlighted.

```
{
  "ticketGroups": [
    {
```

```

    "exchangeTicketGroupId": 9999,
    "eventId": 999,
    "mine": false,
    "availableQuantity": 6,
    "seats": {
      "section": "TEST",
      "row": "TEST1",
      "lowSeat": "1",
      "highSeat": "100"
    },
    "unitPrice": {
      "wholesalePrice": {
        "value": 1,
        "currencyCode": "USD"
      },
      "retailPrice": {
        "value": 1,
        "currencyCode": "USD"
      },
      "facePrice": {
        "value": 1,
        "currencyCode": "USD"
      }
    },
    "ticketGroupType": {
      "id": 1,
      "description": "Event Ticket"
    },
    "deliveryMethods": ["Upload QR Screenshot"],
    "nearTerm": {
      "nearTermDeliveryMethod": {
        "id": 14,
        "description": "Upload QR Screenshot"
      },
      "nearTermDisplay": {
        "id": 2,
        "description": "Only show near-term shipping"
      }
    },
    "notes": "Mobile Entry Tickets. Must have smart device on hand to enter event. Do not print these tickets. Scan your tickets from your mobile phone for this event. Do not print these tickets.",
    "purchasableQuantities": [6, 4, 3, 2, 1],
    "salesTaxApplicable": false,
    "retailDeliveryMethods": ["MD"],
    "isTNPrime": false,
    "highlightId": null,
    "standardSection": null,
    "canonicalSection": null
  },
  "totalCount": 1
}

```

Use the Mercury REST API GET ticketgroups resource to get TicketNetwork inventory, with returnRetailDeliveryMethods=true. The below table shows how the ticketgroups output matches up with the Mapwidget API addTicketData() input. **Please note that you will have to convert some of the output into a different data type before passing it to the Mapwidget API.**

GET ticketgroups Output	addTicketData() Input	Notes
seats.section	tgUserSec	
seats.row	tgUserRow	
seats.highSeat seats.lowSeat	tgUserSeats	highSeat and lowSeat should be expressed as a range for tgUserSeats (e.g. 101-102)
availableQuantity	tgQty	
unitPrice.wholesalePrice.value	tgPrice	You will need to build in your own markups on wholesale price and then use the new price value for the tgPrice input
exchangeTicketGroupId	tgID	
notes	tgNotes	
	tgMark	
ticketGroupType.id	tgType	
retailDeliveryMethods	tgDeliveryOptions	You will need to join this array into a comma-separated list of values. This property will only be populated in the response if you use returnRetailDeliveryMethods=true when calling the resource
	tgPriceSlash	
purchasableQuantities	tgSplitsBitmap	Use the array of quantities to generate a bitmap according to the instructions in the Splits Bitmap section
	tgClientData	
	tgCType	
	tgDisclaimers	

INTEGRATING MAPS WITH CHECKOUT

You will need to overwrite the checkout function to point your customers to your checkout URL. To configure the link to checkout, overwrite:

```
Seatics.Presentation.redirectToCheckout = function (ticketGroup, quantity){}
```

The first parameter will be a ticket group object as defined previously. The second parameter will be the selected quantity. If the user doesn't select a quantity prior to checkout, you can pick a default quantity in the redirectToCheckout() function or pass a null.

If your purchase flow includes a cart, i.e. the user isn't redirected straight to checkout from the map, you will need to use a custom UI implementation of the API instead of the pre-built UI version.

INTEGRATING MAPS INTO AN APP

The Seatics Maps API is purely front end. To integrate it into an app, it must be put inside a webview and hooked into the JavaScript of the webview to interact with the app. Package your ticket data into a JavaScript object to feed to the webview and use the above `Seatics.Presentation.redirectToCheckout()` function to signal to the app to go to your payment system.



IMPORTANT: Please note our support staff are not app developers. Our support staff cannot give you instructions or guidance on designing or building your app experience.

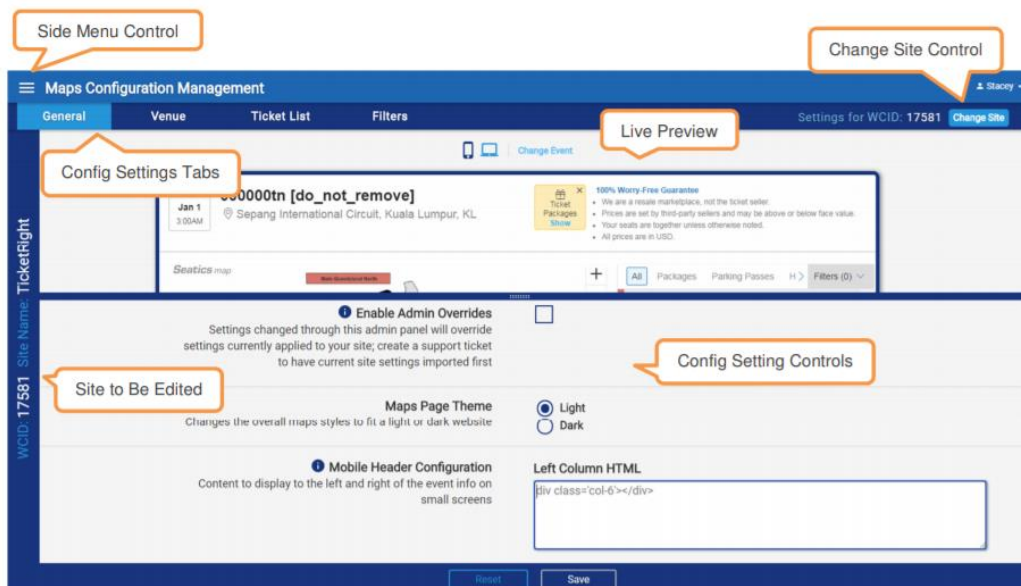
ADDITIONAL SETUP

CONFIGURATION OPTIONS FOR THE PRE-BUILT UI

MAPS CONFIGURATION MANAGEMENT TOOL

The Maps Configuration Management Tool is available through TicketNetwork Portal. Maps Configuration Management is an app that provides site owners with an easy-to-use UI for customizing maps configuration settings. The app allows a site owner to configure most aspects of the pre-built maps UI without the assistance of a developer. More information about the tool and how to get started is available here:

<https://s3.amazonaws.com/tn-release/retail/maps/MapsConfigurationManagement.pdf>.





IMPORTANT: If you'd like to use the Maps Configuration Management tool to handle your maps configuration, please contact our support staff about getting a TN Portal account and permissions to access the tool.

CONFIGURATION IN CODE

Below the inclusion of the Seatrics Maps API call, many properties of the `Seatics.config` object can be overridden to customize the pre-built UI. Configuration can be done in JavaScript, or many settings can be changed through the Maps Configuration Management tool. None of these options is mandatory to set. See [Configuration Details](#) in the Appendix for screenshots of the pre-built UI and more details about each setting.

Property	Definition	Default
<code>Seatics.config.levelColors</code>	An array of colors to use for the map levels on non-zone maps; one property used to set color scheme	<code>["#d86f6f", "#e2a6a6", "#f4ad58", "#ffd797", "#bc91c1", "#e5cae2", "#66b6dd", "#a3ddf8", "#abcb59", "#d2e08b", "#bfa082", "#d9d7ba", "#589caf", "#bcd3e7", "#fff871", "#fff7a9", "#9bc9a2", "#c7ddc8"]</code>
<code>Seatics.config.levelColorsSmallVenue</code>	An array of colors to use for the map levels on non-zone maps when the venue has 3 levels or less; one property used to set color scheme	<code>["#e5cae2", "#a3ddf8", "#d2e08b"]</code>
<code>Seatics.config.noZones</code>	Determines whether to use regular maps or zone maps	<code>true</code>
<code>Seatics.config.zoneColors</code>	An array of colors to use for zones on zone maps; one property used to set color scheme	<code>["#d86f6f", "#e2a6a6", "#f4ad58", "#ffd797", "#bc91c1", "#e5cae2", "#66b6dd", "#a3ddf8", "#abcb59", "#d2e08b", "#bfa082", "#d9d7ba", "#589caf", "#bcd3e7", "#fff871", "#fff7a9", "#9bc9a2", "#c7ddc8"]</code>
<code>Seatics.config.useSuperLevels</code>	Determines whether or not complex maps are simplified; one property used to set color scheme	<code>true</code>
<code>Seatics.config.sectionNoTicketsColor</code>	The color used for sections with no tickets available.	<code>#e2e2e2</code>
<code>Seatics.config.customLevels</code>	Allows the site to define custom venue levels	
<code>Seatics.config.noMapImageUrl</code>	Allows the site to override the placeholder 'Map Coming Soon' image with a custom image	https://d340sbn9oxreq3.cloudfront.net/mapcoming_soon.png
<code>Seatics.config.ticketsState</code>	Specifies which presentation of the legend to use	<code>legendDriven: false</code> <code>showLegendFirstOnMob: false</code>
<code>Seatics.config.noEventHandler</code>	A function that will be called if the event is expired	Error message
<code>Seatics.config.noTicketsHandler</code>	A function that will be called if there are no tickets for the event	Error message

Seatics.config.selectionScheme	Sets how selected sections on the map appear	0
Seatics.config.selectionColor	Sets pulse color for map highlighting and selected section fill color for selection scheme 1	#ffff00
Seatics.config.alwaysShowAvailQty	Determines how quantity of tickets available is expressed in the ticket list	false
Seatics.config.preCheckoutButtonHtml	Sets content for the 'Go to Secure Checkout' button	Go to Secure Checkout
Seatics.config.buyButtonContentHtml	Sets content for the CTA button in the TG row	Price per ticket
Seatics.config.tgGuaranteeNoteHtml	Allows editing of guarantee text in pre-checkout screen	100% Worry-Free Guarantee
Seatics.config.ticketSeparationOptions	Sets how to handle unmapped inventory by ticket group type	packages: Seatics.TicketGroupSeparationOptions.Integrate, parking: Seatics.TicketGroupSeparationOptions.Isolate, passes: Seatics.TicketGroupSeparationOptions.Isolate, hotels: Seatics.TicketGroupSeparationOptions.Isolate, unmappedFlex: Seatics.TicketGroupSeparationOptions.Integrate, unmappedStandard: Seatics.TicketGroupSeparationOptions.Separate
Seatics.config.useH1Tag	Controls whether to use an <h1> heading tag for the event name or a <div> tag	true
Seatics.config.tgMarkHtml	Allows the site to specify what symbol to use to mark featured inventory	★
Seatics.config.tgMarkTooltipText	Allows the site to customize the tooltip text that explains the inventory mark	We recommend this seller's tickets.
Seatics.config.defaultSort	Sets how ticket groups are sorted by default	Seatics.SortOptions.PriceAsc
Seatics.config.SaveFilterOptions	Allows site to disable saving filter settings	true
Seatics.config.showLocalPickupFilter	Allows site to enable Local Pickup filter	false
Seatics.config.onBuyButtonClicked	A function that can be used to track when users click the CTA button sectionNoTicketsColor: "#e2e2e2", levelColors: ["#d86f6f", "#e2a6a6", "#f4ad58", "#ffd797", "#bc91c1", "#e5cae2", "#66b6dd", "#a3ddf8", "#abcb59", "#d2e08b", "#bfa082", "#d9d7ba", "#589caf", "#bcd3e7",	level

```
"#fff871", "#fff7a9",
"#9bc9a2", "#c7ddc8"
],
levelColorsSmallVenue:
[
"#e5cae2", "#a3ddf8",
"#d2e08b"
],
zoneColors: [
"#d86f6f", "#e2a6a6",
"#f4ad58", "#ffd797",
"#bc91c1", "#e5cae2",
"#66b6dd", "#a3ddf8",
"#abcb59",
"#d2e08b",
"#bfa082", "#d9d7ba",
"#589caf", "#bcd3e7",
"#fff871", "#fff7a9",
"#9bc9a2", "#c7ddc8"
],
```

Seatics.config.mapFinishedRenderingHandler	A function that can be used to track interactions with the UI	
Seatics.config.enableMyList	Allows site to enable the My List feature	false
Seatics.config.showCents	Allows site to display prices with decimal values	false
Seatics.config.displayFaceValuePreCheckout	Allows site to enable face value display on pre-checkout screen	false
Seatics.config.displayFaceValueTicketList	Allows site to enable face value display on main list	false
Seatics.config.displayFaceValueForUnknownValues	Additional configuration option for when either displayFaceValueTicketList or displayFaceValuePreCheckout is true. Allows site to configure whether to include face value display artifacts when the face value of a ticket is unknown (0 or not provided). If false, this will exclude any face value labels, values, or help text from the display for tickets with an unknown face value. If true, this will display the text "No Face Price" in place of where the face value is normally shown.	false
Seatics.config.faceValueHelpText	Allows site to customize explanation for face value for real tickets. NOTE: does not apply to speculative inventory.	This is the original ticket price, as set by the primary seller. Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value.
Seatics.config.faceValueHelpAlternateText	Allows site to customize explanation for unknown face value for real tickets. NOTE:	The seller has not provided the original price for these tickets. Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value.

	does not apply to speculative inventory.	
Seatics.config.enableQuantityModal	Allows site to ask user to select quantity when entering the page	false
Seatics.config.enableFeedback	Allows site to disable the embedded feedback feature	true
Seatics.config.disableBuyButtonNoQuantitySelection	Allows site to disable the buy button until user selects ticket quantity	false
Seatics.config.enablePrecheckoutCountdown	Allows site to show a countdown timer in precheckout to create urgency to purchase	true
Seatics.config.precheckoutCountdownHtml	Changes the HTML of the countdown timer	
Seatics.config.precheckoutCountdownTimeMilliseconds	Sets the length of time for the timer	240999 (4 min)
Seatics.config.showAllInPricingToggle	Displays a toggle to allow user to choose between showing prices with or without fees	true
Seatics.config.includeServiceFeesInTicketPrice	Allows site to display ticket prices including the fees. When true, users who proceed to Checkout will immediately see fees regardless of the normal behavior.	false
Seatics.config.showPriceBreakdownPrecheckout	Works in combination with includeServiceFeesInTicketPrice. Allows the site to display an itemized breakdown of the ticket price and the fees within the pre-checkout.	false
Seatics.config.showPriceBreakdownTicketList	Works in combination with includeServiceFeesInTicketPrice. Allows the site to display an itemized breakdown of the ticket price and the fees within the main list.	false
Seatics.config.showFeesIncludedText	When prices are shown with fees and price breakdowns are off, controls whether to show the text "Fees Included" within the Ticket List and the Pre-checkout.	true
Seatics.config.enableCheckoutDeepLinking	Allows site to disable the deep link capability that sends customers straight to checkout	true
Seatics.config.checkoutDeepLinkTgIDKey	Allows site to customize which query string parameter is used to specify the deep linked ticket group's id	tgid
Seatics.config.checkoutDeepLinkTgID	Allows site to provide an alternate value or function to	

	determine the deep linked ticket group's id	
Seatics.config.checkoutDeepLinkQuantityKey	Allows site to customize which query string parameter is used to specify the quantity of tickets the customer wants for the deep linked ticket group	qty
Seatics.config.checkoutDeepLinkQuantity	Allows site to provide an alternate value or function to determine the quantity of tickets the customer wants for the deep linked ticket group	
Seatics.config.checkoutDeepLinkExpectedPriceKey	Allows site to customize which query string parameter is used to specify the price (each) the customer expects to pay for the deep linked tickets	prc
Seatics.config.checkoutDeepLinkExpectedPrice	Allows site to provide an alternate value or function to determine the price (each) the customer expects to pay for the deep linked tickets	
Seatics.config.checkoutDeepLinkCustomerNameKey	Allows site to customize which query string parameter is used to specify the name of the customer that the deep link is for	cstnm
Seatics.config.checkoutDeepLinkCustomerName	Allows site to provide an alternate value or function to determine the name of the customer that the deep link is for	
Small Screen Specific Options		
Seatics.config.smallScreenMapLayout	Specifies how much of the map to show on small screen	Seatics.SmallScreenMapOptions.HiddenWithPreview
Seatics.config.eventInfoLeftCol	HTML for site logo or other content in mobile header	null
Seatics.config.eventInfoRightCol	HTML for phone icon or other content in mobile header	null
Seatics.config.skipPrecheckoutMobile	Determines whether user must view pre-checkout info before proceeding to checkout on mobile	false
Seatics.config.enableLegalDisclosureMobile	Determines whether reseller disclosures display on the small screen layout	true
Seatics.config.listLegalDisclosure	Allows site to edit the HTML of the disclosure banner on mobile	
Seatics.config.enableListMarketingMobile	Determines whether to display packages marketing above the ticket list on mobile	true

Seatics.config.listMarketingHtmlFilter	Allows site to edit the HTML displayed above the ticket list to encourage user to browse packages	<code>'<div class="sea-list-marketing-inner vertical-align-middle">Ticket Packages Available!Show Packages</div>'</code>
Seatics.config.listMarketingHtmlAll	Allows site to edit the HTML displayed above the ticket list when the user filters to packages	<code>'<div class="sea-list-marketing-inner vertical-align-middle">Ticket Packages Available!Hide Packages</div>'</code>
Seatics.config.showZoomControls	Changes default display of zoom controls	display on desktop hidden on mobile
Large Screen Specific Options		
Seatics.config.ticketListOnRight	Sets whether the ticket list is on the left with the map on the right or the map is on the left with the ticket list on the right	true
Seatics.config.mapContained	Sets whether the map is contained or is the background of the page	false
Seatics.config.mouseWheelZoomEnabled	Determines whether scrolling the mouse wheel zooms the map	false
Seatics.config.desktopEventInfoRightCol	Allows the site to override HTML to customize guarantee language and legal disclosures	
Seatics.config.legendExpanded	Sets whether the menu-style legend is expanded or collapsed on page load	false
Seatics.config.skipPrecheckoutDesktop	Determines whether user must view pre-checkout info before proceeding to checkout on desktop	false
Seatics.config.enableMapMarketing	Determines whether to display packages marketing in the map area	false
Seatics.config.mapMarketingHtmlFilter	Allows site to edit the HTML displayed in the map marketing area to encourage user to browse packages	<code>'<div id="sea-map-marketing-close" class="sea-marketing-header-close icon-cancel"></div>Ticket PackagesShow'</code>
Seatics.config.mapMarketingHtmlAll	Allows site to edit the HTML displayed in the map	<code>'<div id="sea-map-marketing-close" class="sea-marketing-header-close icon-cancel"></div><span class="sea-marketing-</code>

	marketing area when the user filters to packages	header-label">Ticket PackagesHide'	
Seatics.config.enableHeaderMarketing	Determines whether to display packages marketing in the event info area		true
Seatics.config.headerMarketingHtmlFilter	Allows site to edit the HTML displayed in the event info marketing area to encourage user to browse packages	'<div id="sea-marketing-header-close" class="sea-marketing-header-close icon-cancel"></div>Ticket PackagesShow'	
Seatics.config.headerMarketingHtmlAll	Allows site to edit the HTML displayed in the event info marketing area when the user filters to packages	'<div id="sea-marketing-header-close" class="sea-marketing-header-close icon-cancel"></div>Ticket PackagesHide'	
Seatics.config.enableListMarketingDesktop	Determines whether to display packages marketing above the ticket list on desktop		false

CUSTOMIZING STYLES

The default styling of the UI is a generic, light colored theme. There is also the option to use a dark colored theme for your base styling. To use the dark theme, you will need to append a parameter to your API call as shown below:

https://www.tn-apis.com/maps/v3/MapAndLayout?websiteConfigId=690&consumerKey=BViE7U7RBisT4ud5IZjwQYi_RNIa&eventName=Luke+Bryan&venue=Ford+Field&dateTime=201610291800&UseDarkTheme=true

Both themes were created with the expectation that sites will customize styles to match brand standards. Customization of styling can best be achieved by adding a new stylesheet. Call a custom stylesheet after inclusion of the API call.



Please note: Editing the UI's default stylesheet directly is not recommended. Any changes to the default stylesheet may be overwritten when subsequent updates to the API are released. **Add a separate, custom stylesheet to apply your styling.**

If the pre-built UI is integrated into a responsive site, CSS will need to be added to hide the full site header and footer when the layout switches from desktop to mobile. The mobile layout expects to use the full screen and may not work correctly if additional page elements are placed above or below it. Below is an example CSS declaration that could be used to trigger the hiding of your site's header and footer:

```
@media screen and (max-width: 991px) {
```



```
/* Hide your header and footer here */
```

```
}
```

CONFIGURATION OPTIONS FOR THE CUSTOM UI

Even with a custom UI implementation, some properties of the `Seatics.config` object can be overridden to customize aspects your integration. Configuration can be done in JavaScript below the API call. None of these options is mandatory to set. See [Configuration Details](#) in the Appendix for more details about each setting.

Property	Definition	Default
<code>Seatics.config.levelColors</code>	An array of colors to use for the map levels on non-zone maps; one property used to set color scheme	<code>["#d86f6f", "#e2a6a6", "#f4ad58", "#ffd797", "#bc91c1", "#e5cae2", "#66b6dd", "#a3ddf8", "#abcb59", "#d2e08b", "#bfa082", "#d9d7ba", "#589caf", "#bcd3e7", "#fff871", "#fff7a9", "#9bc9a2", "#c7ddc8"]</code>
<code>Seatics.config.levelColorsSmallVenue</code>	An array of colors to use for the map levels on non-zone maps when the venue has 3 levels or less; one property used to set color scheme	<code>["#e5cae2", "#a3ddf8", "#d2e08b"]</code>
<code>Seatics.config.noZones</code>	Determines whether to use regular maps or zone maps	<code>true</code>
<code>Seatics.config.zoneColors</code>	An array of colors to use for zones on zone maps; one property used to set color scheme	<code>["#d86f6f", "#e2a6a6", "#f4ad58", "#ffd797", "#bc91c1", "#e5cae2", "#66b6dd", "#a3ddf8", "#abcb59", "#d2e08b", "#bfa082", "#d9d7ba", "#589caf", "#bcd3e7", "#fff871", "#fff7a9", "#9bc9a2", "#c7ddc8"]</code>
<code>Seatics.config.useSuperLevels</code>	Determines whether or not complex maps are simplified; one property used to set color scheme	<code>true</code>
<code>Seatics.config.sectionNoTicketsColor</code>	The color used for sections with no tickets available.	<code>#e2e2e2</code>
<code>Seatics.config.customLevels</code>	Allows the site to define custom venue levels	
<code>Seatics.config.noMapImageUrl</code>	Allows the site to override the placeholder 'Map Coming Soon' image with a custom image	
<code>Seatics.config.selectionScheme</code>	Sets how selected sections on the map appear	<code>0</code>
<code>Seatics.config.selectionColor</code>	Sets pulse color for map highlighting and selected section fill color for selection scheme 1	<code>#ffff00</code>

Seatics.config.ticketSeparationOptions	Sets how to handle unmapped inventory by ticket group type	packages: Seatics.TicketGroupSeparationOptions.Integrate, parking: Seatics.TicketGroupSeparationOptions.Separate, passes: Seatics.TicketGroupSeparationOptions.Separate, hotels: Seatics.TicketGroupSeparationOptions.Separate, unmappedFlex: Seatics.TicketGroupSeparationOptions.Integrate, unmappedStandard: Seatics.TicketGroupSeparationOptions.Separate
Seatics.config.defaultSort	Sets how ticket groups are sorted by default	Seatics.SortOptions.PriceAsc
Seatics.config.showCents	Allows site to display prices with decimal values	false
Seatics.config.mouseWheelZoomEnabled	Determines whether scrolling the mouse wheel zooms the map	false
Seatics.config.enableLegend	Determines whether to display the legend	true
Seatics.config.legendExpanded	Sets whether the menu-style legend is expanded or collapsed on page load	false
Seatics.config.enableCheckoutDeepLinking	Allows site to disable the deep link capability that sends customers straight to checkout	true
Seatics.config.checkoutDeepLinkTgIDKey	Allows site to customize which query string parameter is used to specify the deep linked ticket group's id	tgid
Seatics.config.checkoutDeepLinkTgID	Allows site to provide an alternate value or function to determine the deep linked ticket group's id	
Seatics.config.checkoutDeepLinkQuantityKey	Allows site to customize which query string parameter is used to specify the quantity of tickets the customer wants for the deep linked ticket group	qty
Seatics.config.checkoutDeepLinkQuantity	Allows site to provide an alternate value or function to determine the quantity of tickets the customer wants for the deep linked ticket group	
Seatics.config.checkoutDeepLinkExpectedPriceKey	Allows site to customize which query string parameter is used to specify the price (each) the customer expects to pay for the deep linked tickets	prc

<code>Seatics.config.checkoutDeepLinkExpectedPrice</code>	Allows site to provide an alternate value or function to determine the price (each) the customer expects to pay for the deep linked tickets	
<code>Seatics.config.checkoutDeepLinkCustomerNameKey</code>	Allows site to customize which query string parameter is used to specify the name of the customer that the deep link is for	<code>cstnm</code>
<code>Seatics.config.checkoutDeepLinkCustomerName</code>	Allows site to provide an alternate value or function to determine the name of the customer that the deep link is for	

SORTING & FILTERING IN THE CUSTOM UI

To filter and order ticket groups, a `filterOptions` object can be passed to the map component. A new `filterOptions` object can be created like this:

```
var defaultOptions = new Seatics.FilterOptions();
```

To get the current options object being used by the map component, use the following function:

```
Seatics.MapComponent.getFilterOptions();
```

To set the filter options, use this function:

```
Seatics.MapComponent.setFilterOptions(filterOptions);
```

The `filterOptions` properties are described below:

Property	Description	Options
quantity	Filter out ticket groups that don't have this quantity available (accounts for split rules)	number
minQuantity	Filter out ticket groups with total ticket counts below this level	number
minPrice	Filter out tickets with a price below this value	number
maxPrice	Filter out tickets with a price above this value	number
sortType	How the ticket list should be ordered	<code>Seatics.SortOptions.PriceAsc</code> (price low to high) <code>Seatics.SortOptions.PriceDesc</code> (price high to low) <code>Seatics.SortOptions.SectionAsc</code> (section near to far) <code>Seatics.SortOptionsSectionDesc</code> (section far to near) <code>Seatics.SortOptions.Marked</code> (featured tickets show first, then sorts price ascending)

ticketTypeFilters	Filter by special inventory types – an array of one or more options. Set to null or the empty array to remove filter.	Seatics.TicketType.EventTicket Seatics.TicketType.Parking Seatics.TicketType.Hotels Seatics.TicketType.Packages Seatics.TicketType.FlexTickets Seatics.TicketType.AccessPasses
deliveryTypeFilters	Filter by special delivery types – an array of one or more options. Set to null or the empty array to remove filter.	Null Seatics.DeliveryType.Etickets Seatics.DeliveryType.InstantDownload Seatics.DeliveryType.LocalPickup Seatics.DeliveryType.MobileEntry

The API provides basic sorting and filtering functionality. If you'd like to extend the `filterOptions` functionality, you can define a `Seatics.FilterOptionExtensions` object. This object should contain at least the following two functions:

4. `Seatics.FilterOptionExtensions.filter(filterObj, ticketGrp)`

This should return true if the `ticketGrp` object passes the filters of `filterObj`, and false if it doesn't.

2) `Seatics.FilterOptionExtensions.copy(fromFilterObj, toFilterObj)`

This should copy or create the extra filter object properties you want to append.

EXAMPLE CODE – EXTENDING FILTER OPTIONS

A simple example might be a “minimum wholesale price” filter. Imagine you have `wholesalePrice` defined in `tgClientData` and a `minWholesale` property you're using in the filter object:

```
Seatics.FilterOptionExtensions.copy = function(fromFilterObj, toFilterObj){
    toFilterObj.minWholesale = fromFilterObj.minWholesale || 0;
}
Seatics.FilterOptionExtensions.filter = function(filterObj, ticketGrp){
    return ticketGrp.tgClientData.wholesalePrice >= filterObj.minWholesale;
}
```

EXPANDING THE QUANTITY FILTER

In the pre-built UI, the quantity filter goes only to 4+ because over 85% of customers buy two tickets. However, in the custom UI implementation, it is possible to add other quantity options to the filter, if desired.

A quantity filter is an object with the following properties:

Property	Description	Options
value	Number value of the filter option	number
exact	Exact:false is used for options that are + Boolean	true false
maxQtySelected	Optional property, not important to extending quantity filter options Boolean	true false

To add an exact quantity, such as a quantity filter of 5, the properties would be: {value: 5, exact: true} Filtering by quantity 5 would then return only ticket groups where the user can purchase 5 tickets (accounting for split rules).

To add an 'equal or greater' quantity filter, such as a quantity filter of 6+, the properties would be: {value: 6, exact: false} Filtering by quantity 6+ would then return ticket groups where the user can purchase 6 or more tickets (accounting for split rules).

EXAMPLE CODE – EXTENDING SORTING OPTIONS

To add new sorting types, you can define the following dictionary: `Seatics.FilterOptionExtensions.sortFuncByType`

The methods will be standard sort compare functions as outlined here: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort

```
Seatics.FilterOptionExtensions.sortFuncByType = {"WholesalePriceSort": function (t1, t2) {
    return t2.tgClientData.wholesalePrice - t1.tgClientData.wholesalePrice;
}}
```

And you can set the sort type like this:

```
filterObj.setSortType("WholesalePriceSort");
```

INTERACTIONS BETWEEN THE LIST AND THE MAP IN THE CUSTOM UI

While you may want to customize the look of your ticket list, you will likely want to implement some of the same functionality that is built into the default ticket list UI. For example, when a user hovers a ticket listing, the expectation is that the corresponding section on the map will be highlighted. To achieve this in the custom implementation of the API, use the method `Seatics.MapComponent.highlightTicket`. It takes a single parameter, the ticket group object. The ticket group object passed in must have been processed by the map. Processed ticket groups can be looked up by ID using

Seatics.MapComponent.getTgFromId(tgId), or they can be captured in the presentationInterface.updateTicketsList() function, as shown below.

The key line in the below example is `ticketGroupsById[ticketGrp.tgID] = ticketGrp`; This captures the processed ticket group for use in making `highlightTicket` calls later on. Full implementation of the sample code can be viewed here: <https://s3.amazonaws.com/tn-maps-api-demo/CustomUI/DemoPage.html>.

EXAMPLE CODE – HIGHLIGHTING MAP SECTIONS

```
function createList(ticketDataSegmented) {
    if (!container) {
        return;
    }

    var tableHtml = ['<table id="ticketTable" style="width:100%;"><colgroup><col span="1"
style="width: 5%;"><col span="1" style="width: 70%;"><col span="1" style="width: 25%;"></colgroup>'];
    $.each(ticketDataSegmented, function (id, segment) {
        if (!segment.tickets) {
            return;
        }
        if (segment.title) {
            tableHtml.push("<tr><td colspan='3'>" + segment.title + "</td></tr>");
        }
        $.each(segment.tickets, function (i, ticketGrp) {
            ticketGroupsById[ticketGrp.tgID] = ticketGrp;
            tableHtml.push('<tr tgid=' + ticketGrp.tgID + '><td style="background: ' +
(ticketGrp.tgColor || 'white') + '"></td>');
            tableHtml.push("<td>" + ticketGrp.getSectionDisplayName() + "<br/>Row " +
ticketGrp.getRowDisplayName() + " &bull; " + ticketGrp.tgQty + " Tickets</td>");
            tableHtml.push("<td><input type='button' value='$" +
numeral(ticketGrp.tgPrice).format('0,0') + "' /></td></tr>");

        });

    });
    tableHtml.push("</table>");
    container.html(tableHtml.join(""));
    container.find("tr").hover(function(){
        var tgId = $(this).attr("tgid");
        if(!tgId)
            return;
        highlightTicketById(parseInt(tgId));
    });
}
```

```
}
```

ADDING VIEW FROM SECTION IMAGES TO THE CUSTOM UI

View from section images are available for most major sports venue configurations. With the custom UI, you have the flexibility to implement the VFS image where and how you'd like. You can retrieve the view from section images from the ticket group object. If an image exists, you'll be able to access it using `ticketGroup.section.vfsUrl`.

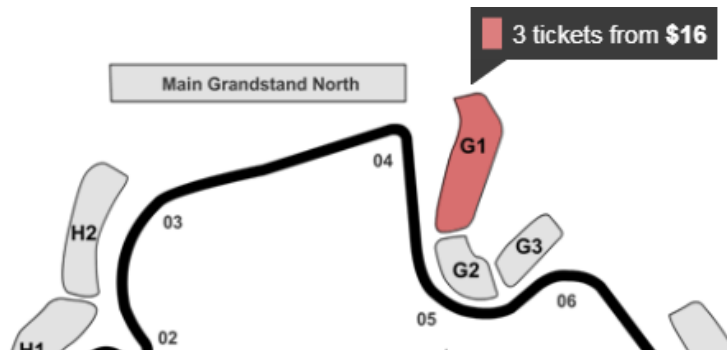
Make sure you're passing the VFS domain to the `MapComponent.create()` method.

EXAMPLE CODE – USING VFS IMAGES

```
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
  },
  mapWidth: 525,
  mapHeight: 545,
  mapName: eventData.mapName
});
```

ENABLING SECTION INFO POP-UPS IN THE CUSTOM UI

One feature of the pre-built UI that can also be utilized in the custom UI is the ability to show additional section information in a pop-up over the map, either when a section is clicked, or on hover over a section.



To enable this feature:

- 1) Set `Seatics.config.largeScreenFormat` to `true`. This controls whether selecting a section by clicking on it will show the section's pop-up.
- 2) Set `Seatics.config.hoverEnabled` to `true`. This controls whether mouse hover over a section will show the section's pop-up.
- 3) In the call to `Seatics.MapComponent.create()`, pass the `enableSectionInfoPopups` parameter and set it to `true`. This will enable the ability to use the pop-ups.

If you only want clicks on a section to show the pop-up (and not mouse hover), only do steps 1 and 3.

EXAMPLE CODE – ENABLING SECTION INFO POP-UPS

```
Seatics.config.largeScreenFormat = true;
Seatics.config.hoverEnabled = true;
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
  },
  mapWidth: 525,
  mapHeight: 545,
  mapName: eventData.mapName,
  enableSectionInfoPopups: true
});
```


By default, the section info pop-up will display the number of tickets available in that section and the starting price. To include view from section images (if available) as well, retrieve the VFS with `vfsUrl`: 'https://vfs.seatics.com' when calling `MapComponent.create()`.

To enable section info pop-ups after the map has already loaded, set `Seatics.config.largeScreenFormat` and `Seatics.config.hoverEnabled` to control pop-up behavior, then call `Seatics.MapComponent.enableSectionInfoPopups()`.

To disable section info pop-ups after they were already enabled (either in the `MapComponent.create()` call or by using `MapComponent.enableSectionInfoPopups()`), call `Seatics.MapComponent.disableSectionInfoPopups()`. This will also reset some of the `Seatics.config` properties that control pop-up behavior.

To manually open the section pop-up for a specific ticket group, first get a reference to the ticket group object by calling `Seatics.MapComponent.getTgFromId(ticketGroupId)`, then call `Seatics.Tooltip.showTooltipForTg(ticketGroup)` to open the pop-up.

To manually hide the currently open pop-up, call `Seatics.Tooltip.hide()`.

POP-UP VIEWPORT

By default, the pop-ups will display within the bounds of the container element passed in the `container` parameter of the `Seatics.MapComponent.create()` call. If you want to specify a different area where the pop-ups will be displayed, you can create a custom implementation of the `Seatics.MapPresentationInterface.getMapViewport()` function.

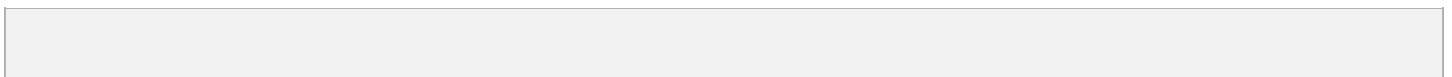
The function should return an object of this form:

```
{
  x1: <Number>,
  x2: <Number>,
  y1: <Number>,
  y2: <Number>
}
```

The points (x1, y1) and (x2, y2) are the top left and bottom right corners of a rectangle in the browser window where the pop-ups will be allowed to display. The `getMapViewport` function is called to grab the current viewport every time the pop-up is shown.

Below is a sample implementation that allows the pop-ups to display anywhere inside the browser window.

EXAMPLE CODE – ALLOW POP-UPS TO DISPLAY ANYWHERE IN BROWSER WINDOW



```
function() {  
  return {  
    x1: 0,  
    x2: $(window).width(),  
    y1: 0,  
    y2: $(window).height()  
  }  
}
```

You should pass your custom `getMapViewport` function as part of the `presentationInterface` object in your `MapComponent.create()` call:

EXAMPLE CODE – ENABLING SECTION INFO POP-UPS WITH CUSTOM GETMAPVIEWPORT

```
Seatics.MapComponent.create({  
  imgSrc: eventData.mapImage,  
  tickets: ticketData,  
  mapData: mapData,  
  vfsUrl: 'https://vfs.seatics.com',  
  container: container,  
  presentationInterface: {  
    updateTicketsList: myCreateListFunction  
    getMapViewport: function() {  
      return {  
        x1: 0,  
        x2: $(window).width(),  
        y1: 0,  
        y2: $(window).height()  
      }  
    }  
  },  
  mapWidth: 525,  
  mapHeight: 545,  
  mapName: eventData.mapName,  
  enableSectionInfoPop-ups: true  
});
```

CURRENCY SYMBOL SUPPORT

By default, all sites are assumed to be listing their prices in USD and Maps will use a dollar sign (\$) as the currency symbol.

2264 tickets from \$5

However, the dollar sign is used for multiple currencies around the world and can be ambiguous to some users. In cases where the currency might be unclear, Maps will format prices using the globally recognized “US\$” symbol instead of just the dollar sign.

2264 tickets from US\$5

This clarification will happen for all events that are outside of the United States, as well as for all users who are not native to the US.

If you would like to adjust your currency, symbol, or currency globalization settings please see the [Currency Symbol Configuration](#) section in the Appendix.

INVENTORY DISCLAIMERS

In some jurisdictions, ticket marketplaces are required to disclose if tickets have not yet been purchased by seller, but the seller guarantees to procure the tickets once requested. Use the `tgDisclaimers` property of the ticket group object to include any disclosures needed for the ticket group. The parameter takes an array of strings to support one or multiple disclosures.

EXAMPLE CODE – ADDING DISCLAIMERS TO TICKET GROUPS

```
Seatics.MapComponent.addTicketData([
  {tgUserSec:"Section 100", tgUserRow:"C", tgUserSeats:"5-6",
    tgQty:2, tgPrice:55.00, tgID:1, tgNotes:"Comes with a free soda", tgMark:1, tgType:1,
    tgClientData:"SomeTrackingCode", tgCType:"R"},
  {tgUserSec:"Section 122", tgUserRow:"T", tgUserSeats:"7-8", tgQty:2, tgPrice:34.00, tgID:2,
    tgNotes:"Obstructed view", tgMark:0, tgType:1, tgClientData:"SomeTrackingCode2", tgCType:"R"},
  {tgUserSec:"Section 204", tgUserRow:"Z", tgQty:4, tgPrice:22.00, tgID:3, tgMark:0, tgType:1,
    tgClientData:"SomeTrackingCode3", tgCType:"C", tgDisclaimers:["This seller is committing to procure
    these tickets for you upon receipt of your order. If the seller is unable to provide the tickets or
    equivalent or better ones, you will receive a full refund."]},
  {tgUserSec:"Section 300", tgUserRow:"", tgQty:2, tgPrice:14.00, tgID:4, tgMark:0, tgType:1,
    tgClientData:"SomeTrackingCode4", tgCType:"C"}, tgDisclaimers: ["This seller is committing to
```

```
procure these tickets for you upon receipt of your order. If the seller is unable to provide the
tickets or equivalent or better ones, you will receive a full refund.”]],]);
```

In the above example, ticket groups that the client determined are ‘speculative’ include disclaimer text to display. If your site uses the pre-built UI implementation, the disclaimer will automatically be inserted into the popup created for widget and plugin users.

SUN
Oct 31, 2021
7:00PM

Wicked

📍 Gershwin Theatre, New York, NY

🔔 Important Event Information

Tickets

Parking Passes

Filters (0) ▾

Middle & Rear Mezzanin...

Row H

1-6 Tickets 📄

Important: Zone Seating ⓘ

\$121/ea

Middle & Rear Mezzanin...

Row H

1-4 or 6 Tickets 📄

Important: Zone Seating ⓘ

\$123/ea

Orchestra Left

Row W

1-6 Tickets 📄

Important: Zone Seating ⓘ

\$131/ea

Tickets with disclaimers are marked with links

A screenshot of a seat selection interface. In the background, there are two seating charts. The left chart is titled 'Seats 2-36 (even)' and shows a green section labeled 'Rear'. The right chart is titled 'Seats 118-101' and also shows a green section labeled 'Rear'. A white pop-up box with a close button (X) in the top right corner is centered on the screen. The pop-up contains the text: 'Please Note: The tickets you receive may be in the specific section listed here or in an equivalent or better location. This listing describes tickets that the seller does not own or may not know the exact location of the seats, but is offering to obtain for you. Once your order is confirmed, we guarantee that you will receive tickets prior to the start of the event for seats that are in an equivalent or better location or you will receive a full refund.'

Clicking the 'Important Message' link opens the disclaimer modal

To use the popup with the custom UI implementation of the Seatics Mapwidget API, add a new `Seatics.TicketDisclaimerModal.setup`(\$("body")); call when making the `MapComponent.create()` call. The input for `TicketDisclaimerModal.setup()` is a jQuery element that the modal HTML is injected into. To show the modal on the site, call `Seatics.TicketDisclaimerModal.show(["TEST"]);` where the input is an array of disclaimer strings.

The `tgDisclaimers` property is useful when clients know what inventory to mark at the event level. To specifically target customers by location on the map page, we recommend using an IP detection service or similar geolocation solution.

CONFIGURING THE MAP LEGEND IN THE CUSTOM UI

The dropdown-style map legend is available in custom UI implementations of the Seatrics Maps API. The map legend HTML is added to the container element parameter passed in the call to `Seatics.MapComponent.create()`. Currently it is not possible to have the legend HTML inserted elsewhere in the page.

There are a few configuration options for the legend:

- Enabled and collapsed on page load (default setting)
- Enabled and expanded on page load
- Disabled

To configure the legend to be fully expanded on page load, set the configuration parameter `Seatics.config.legendExpanded=true`; before calling `Seatics.MapComponent.create()`.

To disable the legend so it does not display, set the configuration parameter `Seatics.config.enableLegend=false`; before calling `Seatics.MapComponent.create()`.

WHEN A MAP IS UNAVAILABLE

We offer a large catalog of interactive and static maps, but there may be the rare case that we don't have a map for an event. By default, we display a "Map Coming Soon" image when there is no map available. You can see an example of this scenario on our demo site here: <http://s3.amazonaws.com/tn-maps-api-demo/DemoPageMissingEvent.html>. The `Seatics.config.noMapImageUrl` configuration setting can be used to pass in the URL of your own map image, replacing our placeholder:

```
Seatics.config.noMapImageUrl = "https://yourOwnImageUrl";
```



IMPORTANT: Please note that if you choose to override the map or image provided by the API, you are responsible for the accuracy of the map you display.

You can also contact your account manager to request the desired map be created by our cartography team.

TICKETNETWORK ZONE MAPS

If you include TicketNetwork amongst your inventory sources, you may be interested in using TicketNetwork's zone maps. By default, zones are disabled in the API due to them being TicketNetwork-specific. However, if you want to enable zone maps, simply add the following to your configuration settings:

```
Seatics.config.noZones = false;
```

Remember to identify zone inventory with the `tgCType` property when adding ticket data.

USER EVENT TRACKING

User event tracking in the maps can be accomplished by creating a "user event listener" and registering it with the map event tracker.

The listener will be a function that takes two parameters: `eventType` and `eventData`. Below is a basic example:

EXAMPLE CODE – USER EVENT TRACKING

```
$(function() {
    var userEventListener = function(eventType, eventData) {
        console.log("Tracking Event " + eventType);
        console.log(eventData);
    };
    Seatics.TrackingEvents.registerEventListener(userEventListener);
});
```

There are four options for `eventType` currently:

- FinishedLoading
- MapInteraction
- FiltersChanged
- BuyButtonClicked

Each `eventType` will have different `eventData` associated with it:

eventType	eventData	Example
FinishedLoading	eventId numTickets numTicketGroups	{eventId:1234567, numTickets:10, numTicketGroups:2}
MapInteraction	pan zoom	{interactionType:"pan"}

	mapReset expandedMap selection	
FiltersChanged	changeType	{changeType:"priceMin", changeToValue:25}
	- sort - quantity - etixOnly - ticketType - priceMin - priceMax - reset	
	changeToValue	
BuyButtonClicked	listPlacement filters	{listPlacement:2, filters:{aBigJsObject}}

FinishedLoading will have the event ID, number of tickets available for the event at the time of visit, and number of ticket groups available for the event at the time of visit.

MapInteraction will have an `interactionType`. Interaction types include pan, zoom, resetting the map, expanding the map (unhiding it on mobile) or section selection.

FiltersChanged will have a `changeType` and a `changeToValue`. The possible change types include sort, quantity, e-tickets only, ticket type, minimum price, maximum price, and resetting the price filter. The `changeToValue` passes the new value set by the user.

BuyButtonClicked will have `listPlacement` (the placement of the ticket group in the list, from the top) and the filter settings at time of button click. For example, if the user clicks to buy the third ticket down on list `{listPlacement:2, filters:{aBigJsObject}}`.²

PRE-FILTERING SETTINGS

There are cases where a site wants to be able to send a user to a maps page that is pre-filtered to show certain inventory. They may want to deep link the user to just parking or packages from another page on the site, for example. Or they may want to direct users to inventory under \$100 for the weekend's Knicks game from an email campaign.

We've created several configuration settings that can be set using query string parameters appended to the URL of the page. The site will be responsible for setting up the query string handling.

Property	Definition	Options

² The first ticket group in the list is at `listPlacement:0`.

Seatics.config.ticketTypeConstraints	Used to filter the list by ticket type	Seatics.TicketType.Parking Seatics.TicketType.Hotels Seatics.TicketType.Packages Seatics.TicketType.AccessPasses Seatics.TicketType.FlexTickets
Seatics.config.deliveryTypeConstraints	Used to filter the list by delivery type	Seatics.DeliveryType.Etickets Seatics.DeliveryType.InstantDownload Seatics.DeliveryType.LocalPickup
Seatics.config.ticketQuantityConstraints	Used to filter the list by quantity	1 2 3 4
Seatics.config.ticketPriceConstraints	Used to filter the list by price	See notes below

FILTERING BY TICKET TYPE

To pre-filter the TG list by ticket type, set the `ticketTypeConstraints` parameter. You can set a **single** ticket type like this:

```
Seatics.config.ticketTypeConstraints = Seatics.TicketType.Packages;
```

Or **multiple** options like this:

```
Seatics.config.ticketTypeConstraints = [Seatics.TicketType.Packages, Seatics.TicketType.Hotels];
```

FILTERING BY DELIVERY TYPE

To pre-filter the TG list by delivery type, set the `deliveryTypeConstraints` parameter. You can set a **single** delivery type like this:

```
Seatics.config.deliveryTypeConstraints = Seatics.DeliveryType.Etickets;
```

Or **multiple** options like this:

```
Seatics.config.deliveryTypeConstraints = [Seatics.DeliveryType.Etickets,  
Seatics.DeliveryType.InstantDownload];
```

FILTERING BY QUANTITY

To pre-filter the TG list by quantity, set the `ticketQuantityConstraints` parameter. Because the default quantity filter goes up to 4+, the largest value you can enter is 4.

Example:


```
Seatics.config.ticketQuantityConstraints=4;
```

FILTERING BY PRICE

To pre-filter the TG list by price, set the `ticketPriceConstraints` parameter. This config parameter takes a two element array.

Example:

```
Seatics.config.ticketPriceConstraints = [5, 500];
```

The first element is the minimum price and the second element is the maximum price.

To set only a **minimum price**, pass in null for the second value like this:

```
Seatics.config.ticketPriceConstraints = [5, null];
```

Alternately, you can use just a single element array:

```
Seatics.config.ticketPriceConstraints = [5];
```

To set only a **maximum price**, pass in null for the first value like this:

```
Seatics.config.ticketPriceConstraints = [null, 500];
```

For any of these settings, if nothing matches the pre-filter criteria, all tickets will display.

GOING LIVE

When you are ready to go live, our support team will test your integration to verify it is set up correctly. They must whitelist the domains, including all subdomains, from which you will be calling the API. They will create a production API account for you and give you your production consumer key, which you will insert into the query string of your API calls as explained earlier in the guide.



IMPORTANT: Your production consumer key will be different than your sandbox consumer key. Your production consumer key must be generated by our support staff and will not be made available to you until our team has tested your sandbox implementation and approved you to go live.

SUPPORT

If you have questions or would like to report bugs in the code or errors in the documentation, please contact us with any example code or screenshots that may help us understand your issue. We'll get support staff in touch with you to help troubleshoot the problem.

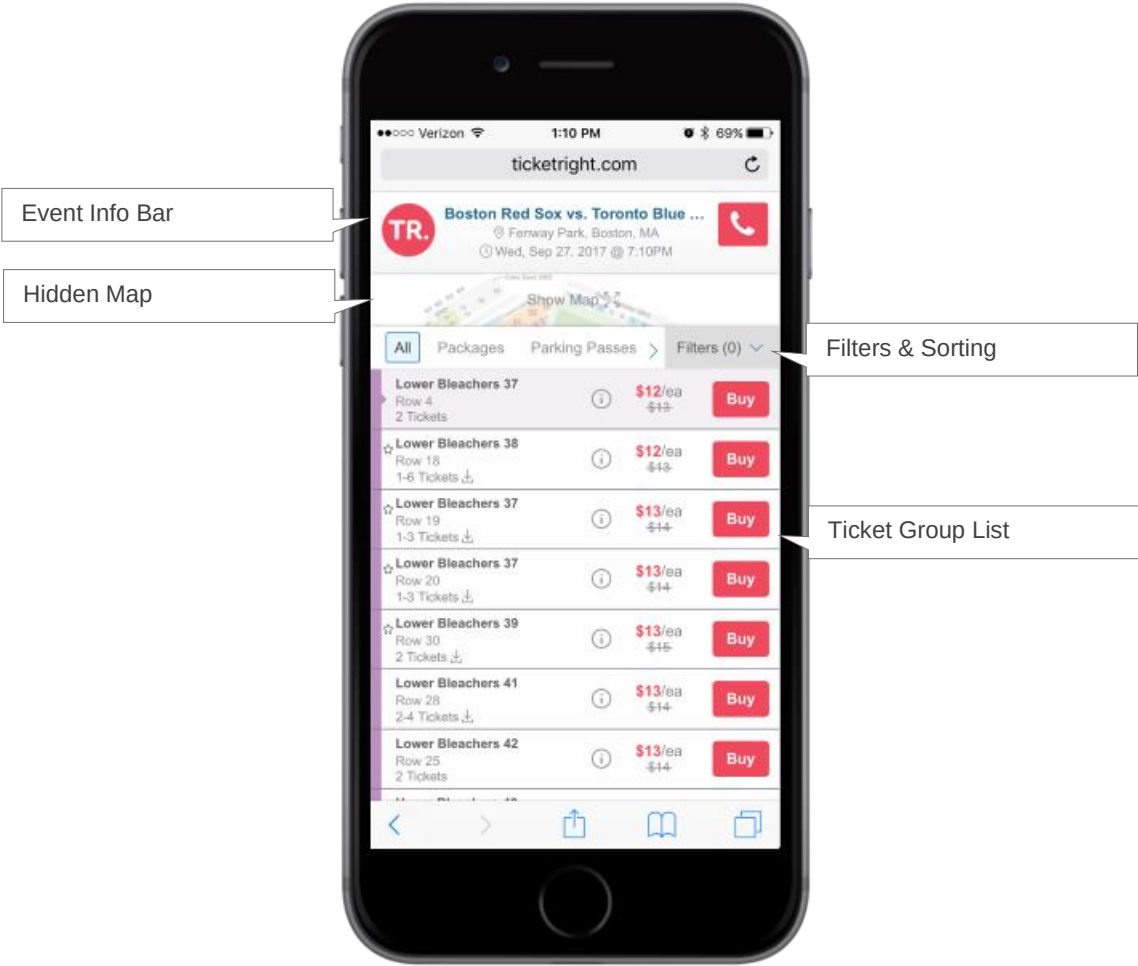
FEEDBACK AND FEATURE REQUESTS

Your feedback is important to us so we can build the best product for your needs. Please submit comments and feature requests to MapsFeedback@ticketnetwork.com. Our team may contact you to follow up on your feedback.

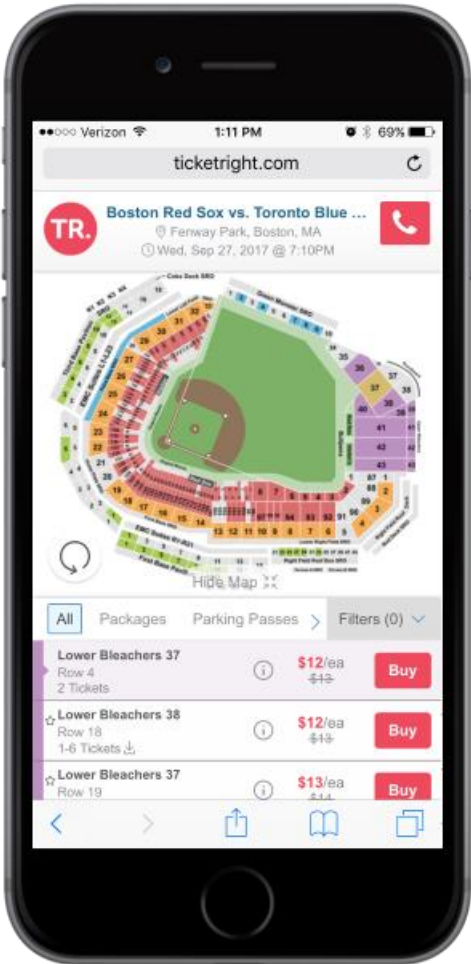
APPENDIX

PRE-BUILT UI SCREENSHOTS

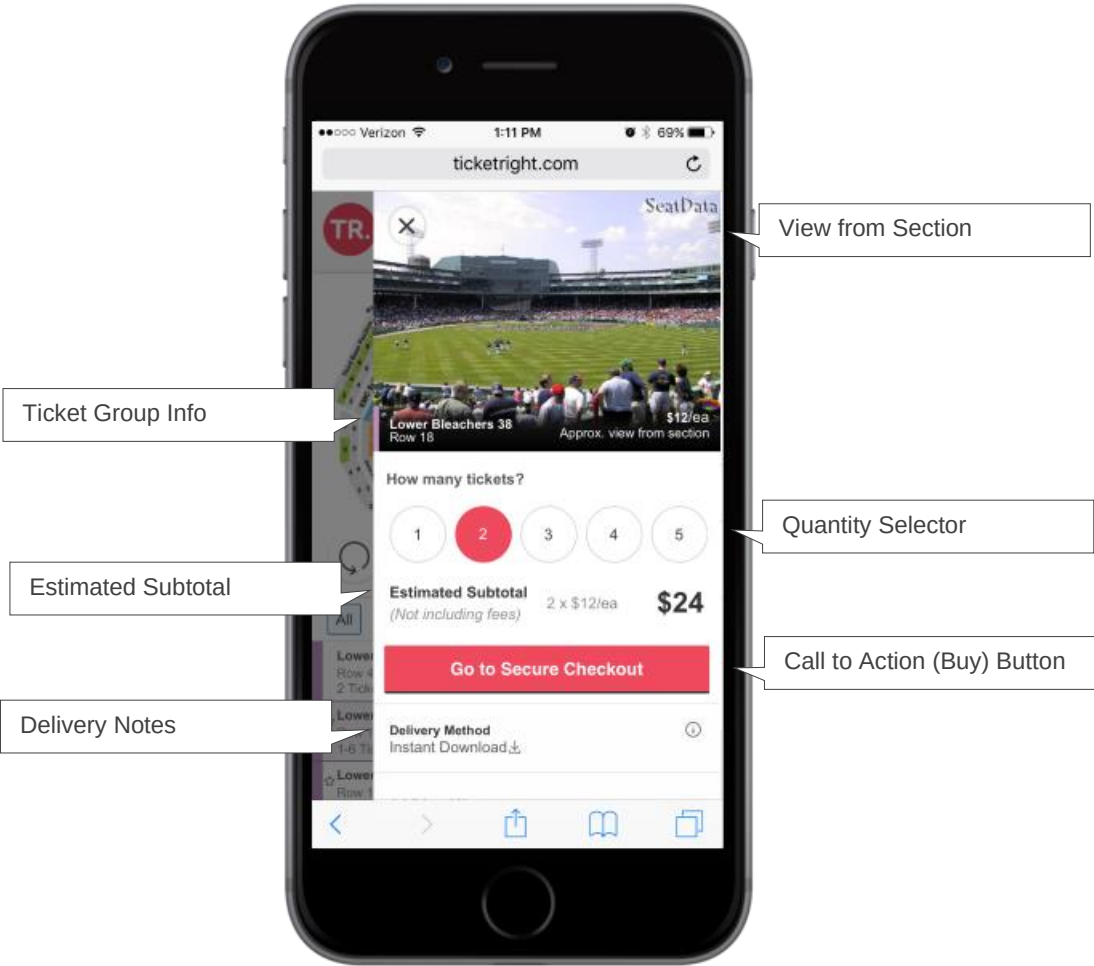
SMARTPHONE LAYOUT



Main Screen



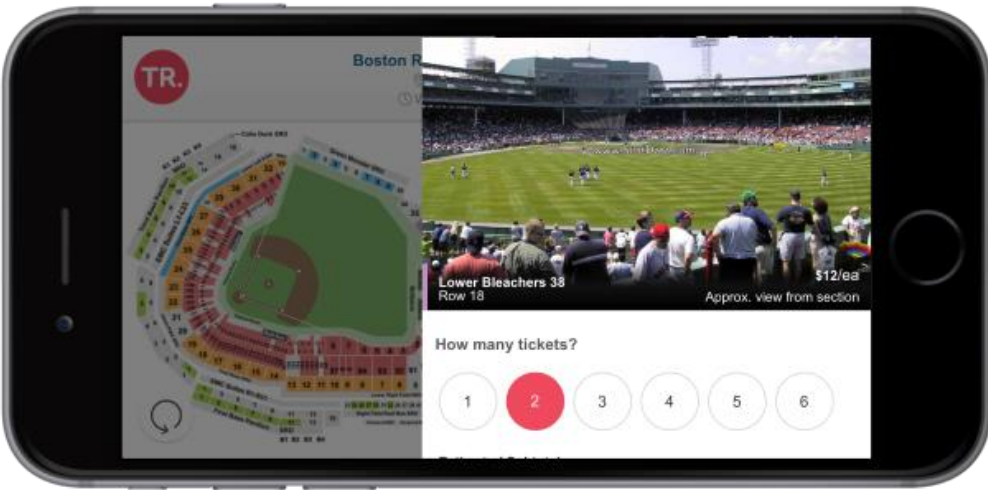
Main Screen with Map Expanded



Ticket Details Slide Out



Main Screen in Landscape Orientation



Ticket Details Slide Out in Landscape Orientation

TABLET LAYOUT

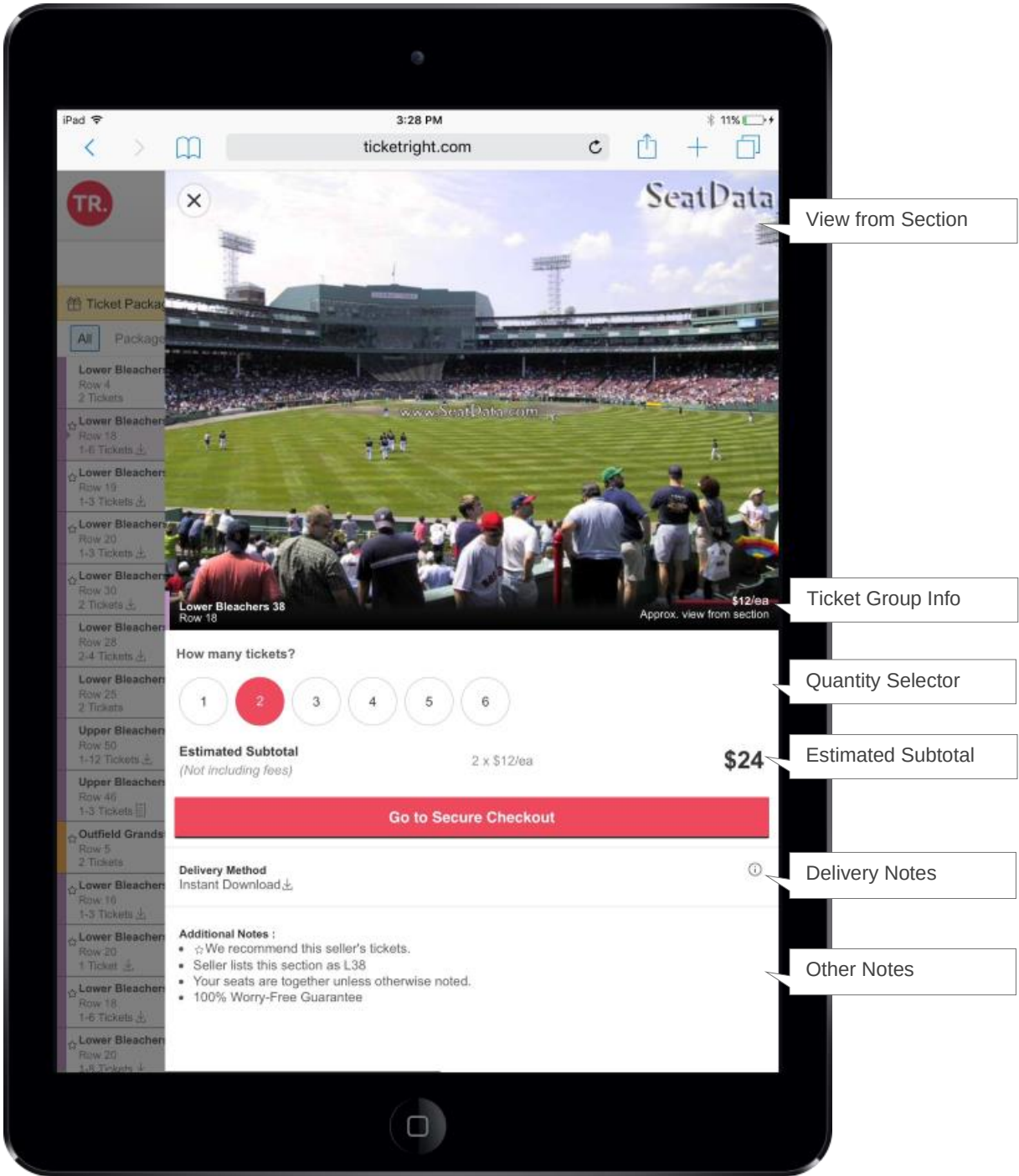
Event Info Bar

Hidden Map

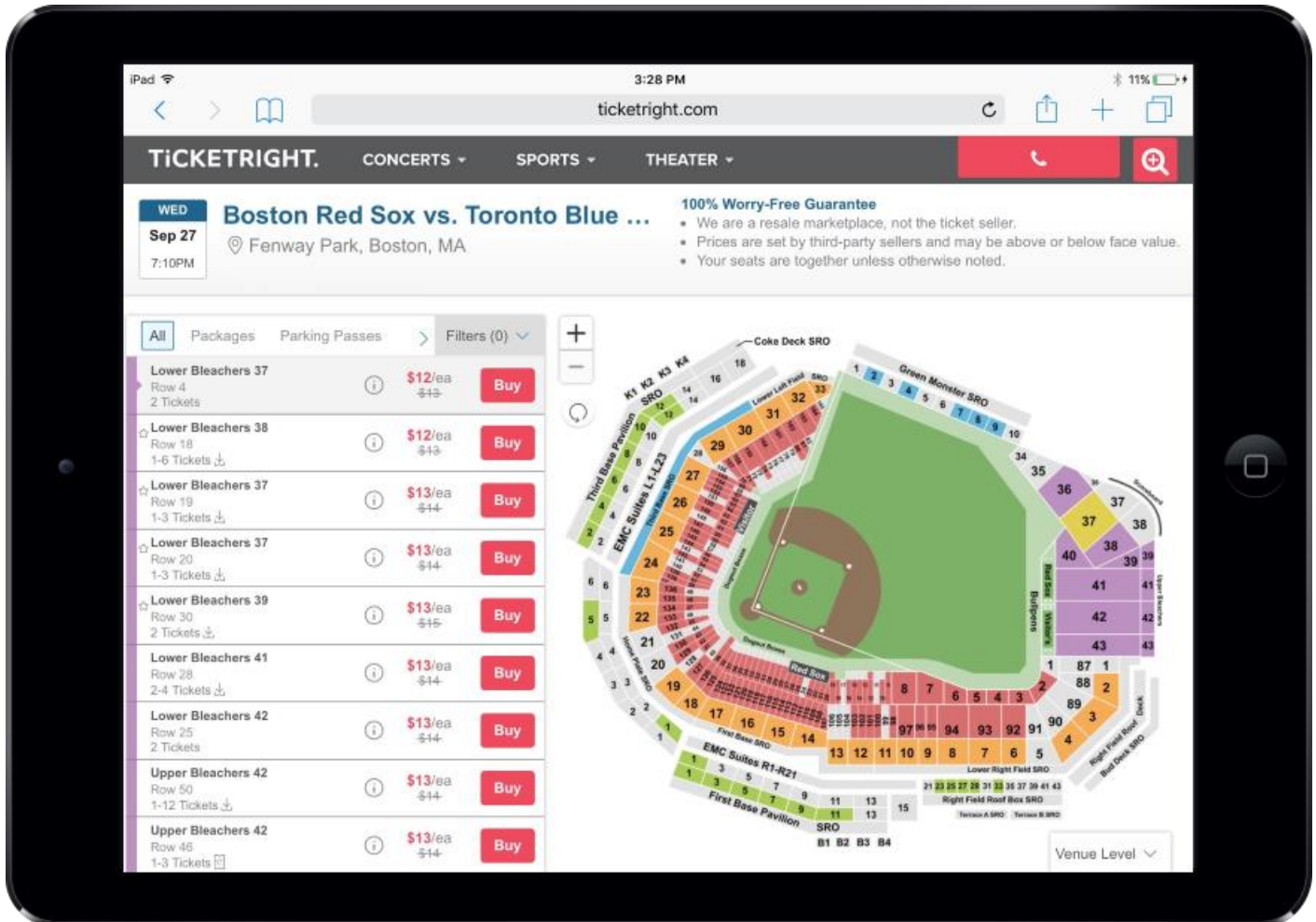
Filters & Sorting

Ticket Group List

Main Screen

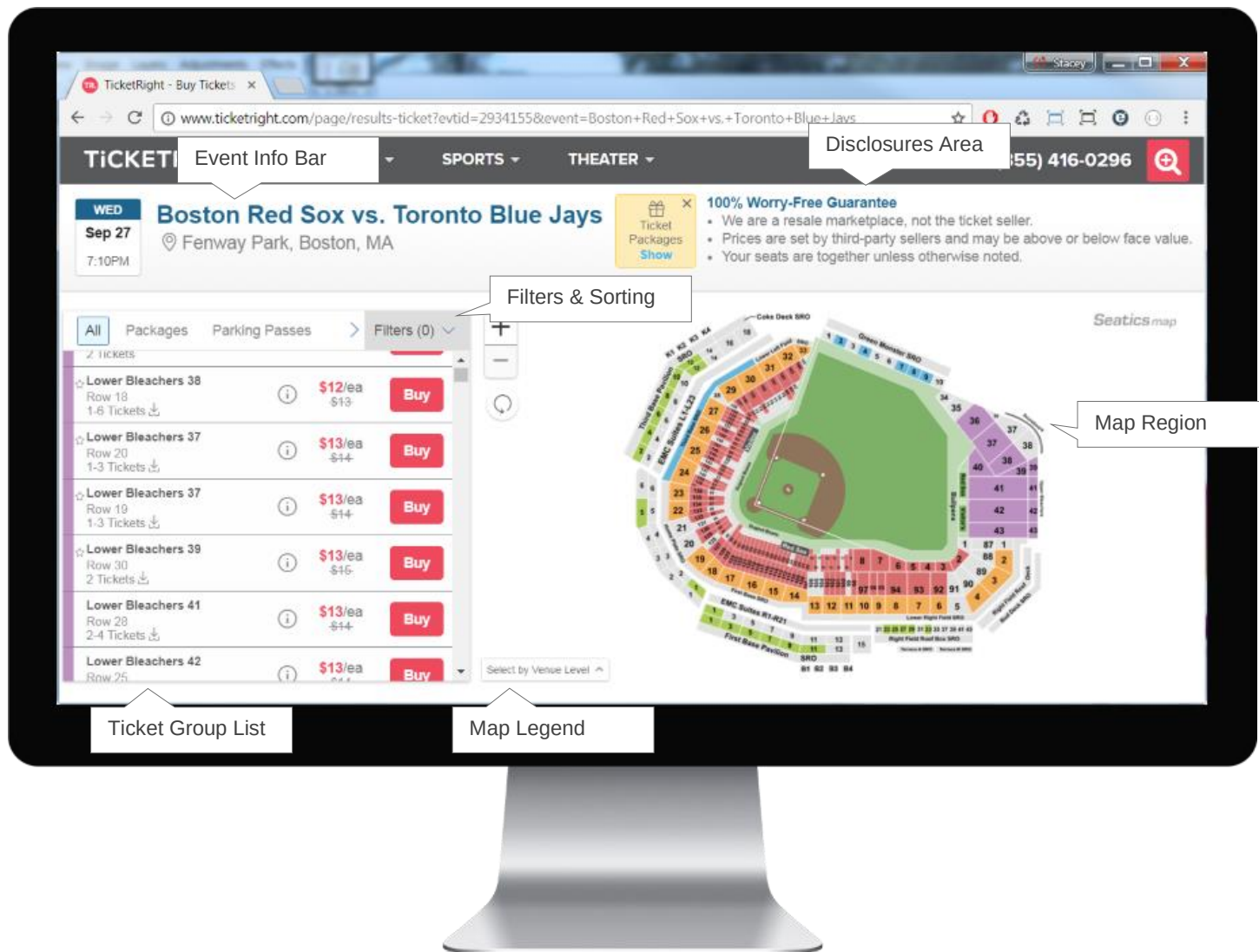


Ticket Details Slide Out

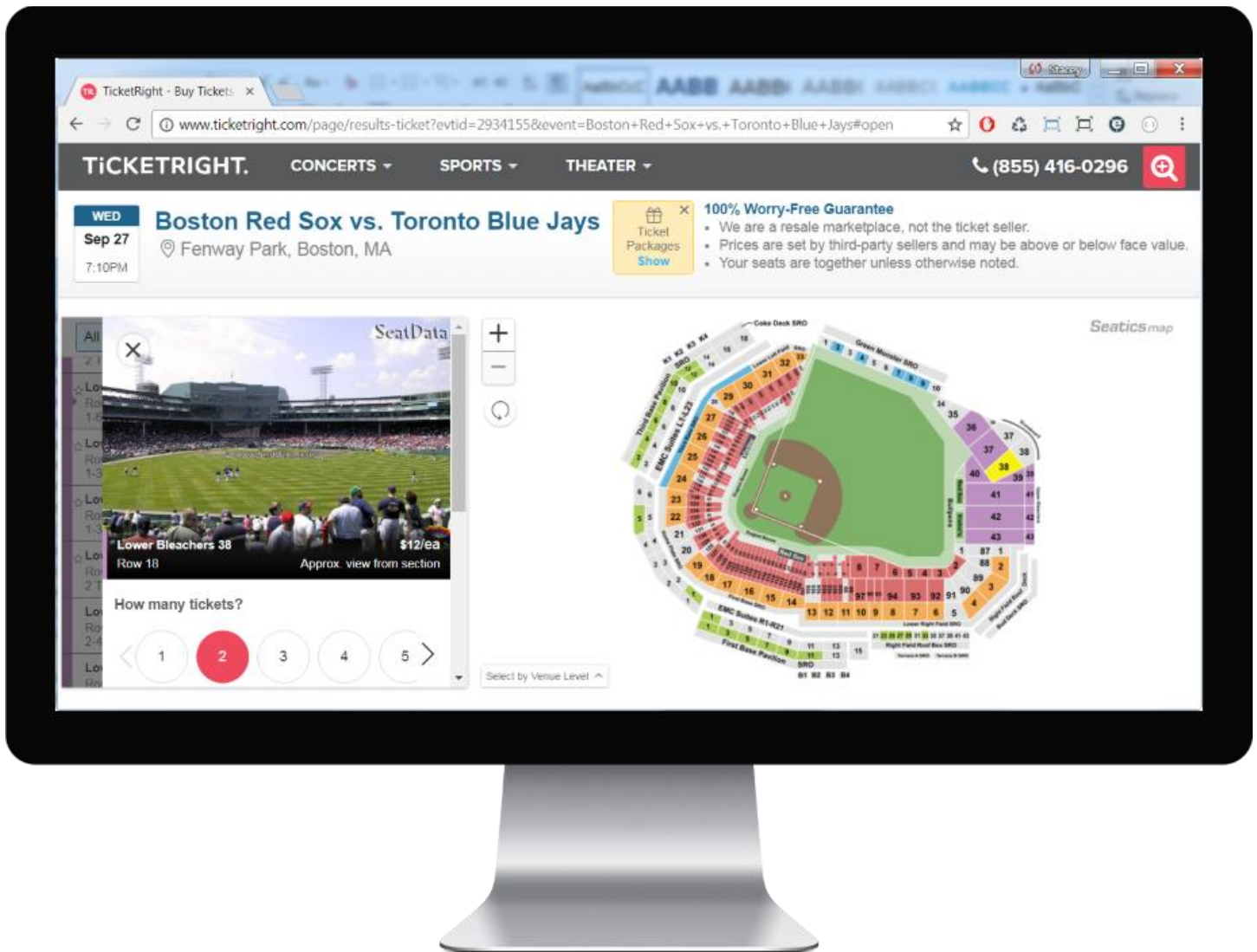


Main Screen in Landscape Orientation

DESKTOP LAYOUT



Main Screen



Ticket Details Slide Out

CONFIGURATION DETAILS

There are many ways to customize the UI without having to build your own. This section provides greater detail about the available configuration settings, including screenshots of different options and sample code to illustrate how to change the settings. Default settings are noted.

CURRENCY SYMBOL CONFIGURATION

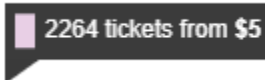
Maps uses an object called `currencyIntl` (currency internationalization) to define what currency symbol to use when formatting prices for various events and users. Maps comes preconfigured with settings for USD and will automatically use the clearer “US\$” symbol when appropriate (based on the user’s language and event location). However, it is possible to adjust these configurations to remove or customize the globalization effect.

DOLLAR SIGN (\$) ONLY, NO USE OF US\$

If you would like to turn off currency globalization altogether so that just the dollar sign (\$) is used, clear out the settings in `currencyIntl`. This will remove the globalization effect for USD, and prices will never be listed with the symbol “US\$”.

code snippet:

```
<script>
  Seatics.config.currencyIntl = {};
</script>
```



NON-USD CURRENCY THAT USES DOLLAR SIGN (\$)

IMPORTANT: The following two sections explain how to change the currency symbol that is displayed in Maps. However, this does not imply support for currency exchange, nor does the TicketNetwork system support charging for tickets in non-USD currencies. Integrators who change the currency symbol are responsible for making sure that the prices shown in Maps are accurate and correct regarding what is charged.

If you are listing your prices in a currency that is not USD, but which still uses a dollar sign (\$) for notation, you must make at least some configuration changes so that Maps does not mistakenly show the “US\$” symbol.

You can follow the directions in the previous section if you’d like to totally remove all globalization effects and just show “\$” in all cases. Or you can set up your own globalization settings for your currency.

code snippet:

```
<script>
```

```
Seatics.config.c3CurrencyCode = "AUD"; // Lets Maps know that you are Australian Dollars instead of US Dollars.
This tells Maps which currencyIntl globalization setting to use.
Seatics.config.currencyIntl = { "AUD": { country: "AU", foreignSymbol: "AU$", nativeSymbol: "$" } }; // Configures
globalization settings for Australian Dollars. Users who are native Australians and are viewing tickets for events in
Australia will see "$". Non-Australian users and users looking at events outside of Australia will see the "AU$" currency
symbol instead.
</script>
```

2264 tickets from \$5

2264 tickets from AU\$5

ALTERNATE CURRENCY SYMBOL

If you would like to completely change the currency symbol used within the display to something other than the dollar sign (\$), just set the currencySymbol configuration property. Changing the currencySymbol will also bypass the logic to show “US\$” because Maps will detect that your symbol does not match the native symbol for USD.

code snippet:

```
<script>
Seatics.config.currencySymbol = "€";</script>
```

2264 tickets from €5

Currency symbol can also be set from the Ticket List tab in Maps Configuration Management:

i Currency Symbol

Replaces the \$ symbol throughout the mapwidget UI

DARK THEME

Append &useDarkTheme=true to the API call to use the dark styling. This theme is available only in Mapwidget3.

This can also be set from the General tab in Maps Configuration Management:

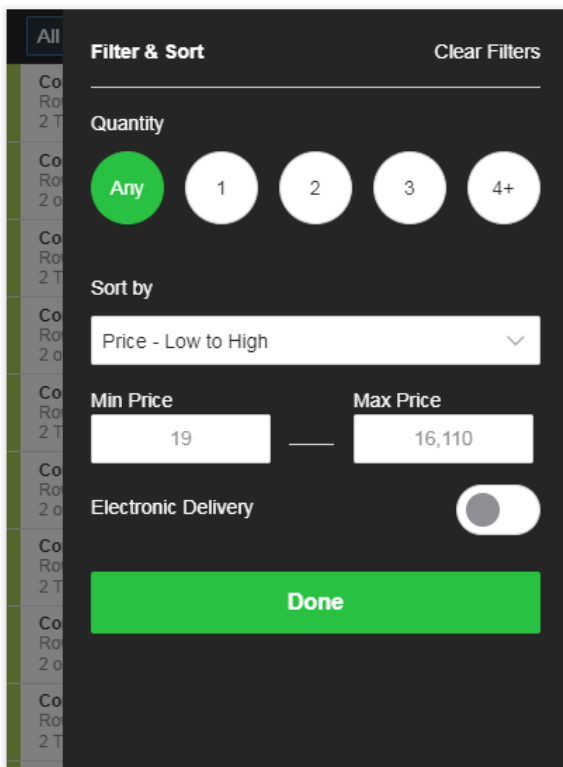
Maps Page Theme

Changes the overall maps styles to fit a light or dark website

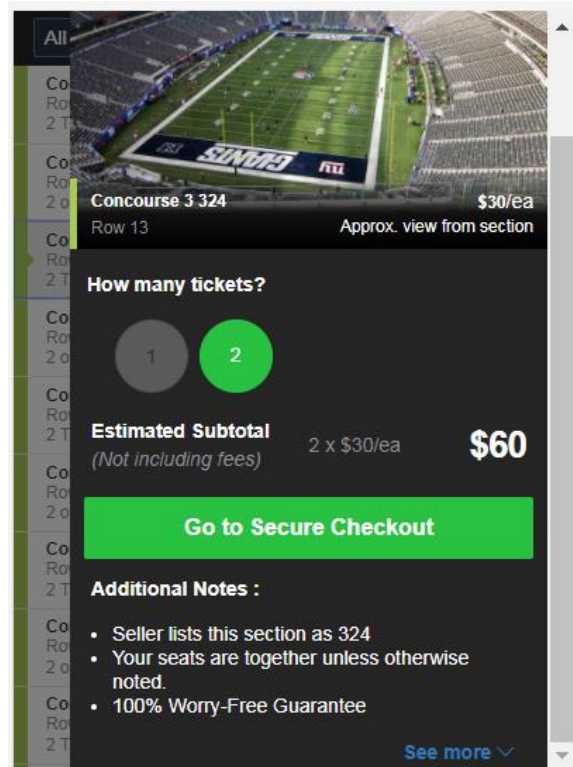
☒ Light ☐ Dark



Dark theme applied to the main maps screen

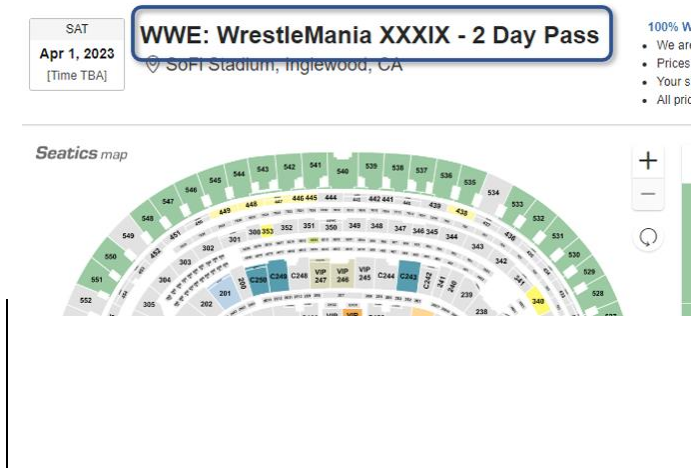


Dark theme applied to the filters menu



Dark theme applied to the ticket details screen

H1 HEADING TAG

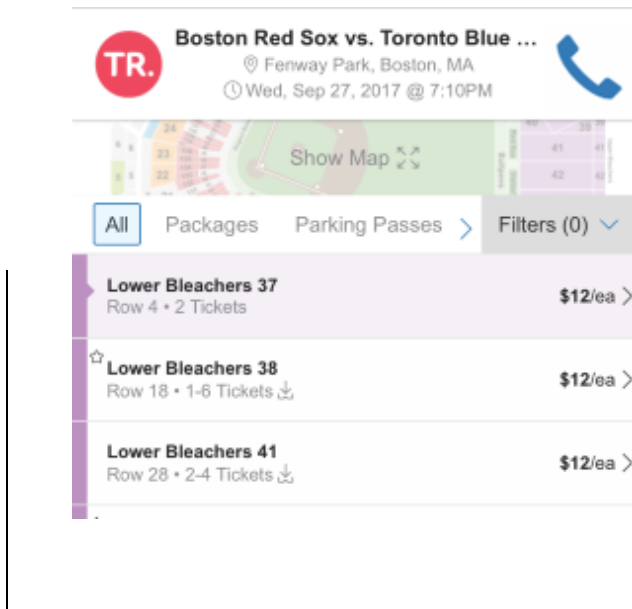


By default, the map will use an `<h1>` heading element for the event name. However, if you would like to completely control the layout of the page headings and optimize your website for SEO, you may choose to render the event name in a non-heading tag. Use `Seatics.config.useH1Tag` to configure whether the event name is rendered within an `<h1>` or a `<div>`. Using either the `<h1>` or the `<div>` element for the event name will maintain the same layout and styling. This setting is only available in Maps V3.

code snippet:

```
<script>
    Seatics.config.useH1Tag = false;
</script>
```

MOBILE HEADER CONFIGURATION



In lieu of a full header, the event info bar on mobile can be configured to display content such as the site logo and a phone icon. Use `Seatics.config.eventInfoLeftCol` to set HTML for the area on the left side of the event info. Use `Seatics.config.eventInfoRightCol` to set HTML for the area to the right side.

Code snippet:

```
<script>
    Seatics.config.eventInfoLeftCol = '<a href="/"><img
src=https://s3.amazonaws.com/ticketnetwork/demo/tr/TR_logo_s
mall2x.png height="55px" width="55px" style="padding:5px;
vertical-align:top;" /></a>',
    Seatics.config.eventInfoRightCol = '<a
href="tel:8608675309"><span style="font-size:40px; vertical-
align:top; padding:5px;" class="glyphicon glyphicon-
earphone"></span></a>';
</script>
```

These configurations can also be set from the General tab in Maps Configuration Management:

i Mobile Header Configuration

Content to display to the left and right of the event info on small screens

Left Column HTML

i Add an image URL

Populates code in "Left Column HTML" above

Right Column HTML

LARGE SCREEN DISCLOSURES CUSTOMIZATION

100% Worry-Free Guarantee

- ✓ Example Ticket Site is a resale marketplace, not the ticket seller.
- ✓ Prices are set by third-party sellers and may be above or below face value.
- ✓ Your seats are together unless otherwise noted.
- ✓ All sales are final.

Disclosures Area

Guarantee Pop Up

In the right side of the Event Info Bar is a region meant to display legal disclosures, as well guarantee language or other trust signals. `Seatics.config.desktopEventInfoRightCol` allows the site to override the default HTML.

If you remove the disclosure “We are a resale marketplace, not the ticket seller. Prices are set by third-party sellers and may be above or below face value,” from this region and you are **NOT** a ticket broker, you **MUST** include the disclosure elsewhere on the page where the notes are above the fold, conspicuous, and unavoidable.

code snippet:

```
<script>
  Seatics.config.desktopEventInfoRightCol = '<div class="event-info-disclaimer"><a id="event-info-guarantee-show"
  class="event-info-guarantee-show" href="javascript:">100% Worry-Free Guarantee</a><br><span>&#x2713; Example Ticket Site
  is a resale marketplace, not the ticket seller.</span><br><span>&#x2713; Prices are set by third-party sellers and may
```

v3.18 | TicketNetwork, Inc. © 2018-2025
Any unauthorized use or distribution is prohibited
Page 64 of 126


```

be above or below face value.</span><br><span>&#x2713; Your seats are together unless otherwise
noted.</span><br><span>&#x2713; All sales are final.</span><br><div id="event-info-guarantee" class="event-info-
guarantee" style="display: none;"><a id="event-info-guarantee-close" class="event-info-guarantee-close icon-cancel"
href="javascript:void(0)"></a><h3>We stand behind you 100%.</h3><br><p>Whether you are buying or selling tickets on our
site, we safeguard your transaction.</p><br /><p><strong>You will receive a 100% refund for your tickets
if:</strong></p><ul><li>Your order was accepted but not delivered by the seller.</li><li>Your order was accepted but not
shipped in time for the event.</li><li>Your tickets were not valid for entry.<sup>(1)</sup></li><li>Your event is
cancelled and is not rescheduled.<sup>(2)</sup></li></ul><br /><p><strong>Notes:</strong></p><p><p>(1) Verified proof must
be provided in letter form from the venue. Written or stamped "voids" do not constitute verified proof.</p><p>(2) 100%
refund for a cancelled event does not include shipping.</p></div></div>';
</script>

```

This configuration can also be set from the General tab in Maps Configuration Management:

Large Screen Disclosure Text
Use Default ☒ Use Custom

```

<h2 class="sea-offscreen">Disclaimer</h2>
<div class="event-info-disclaimer">
  <button aria-controls="event-info-guarantee" aria-expanded="false" id="event-info-

```

MOBILE DISCLOSURES CUSTOMIZATION

As of early 2018, ticket resale sites are required to obtain Google certification to advertise through AdWords. The mobile reseller disclosure helps sites to meet Google's first three requirements. There are a variety of configuration options for the placement of this disclosure.

These configurations can be set from the General tab in Maps Configuration Management:

Show Reseller Disclosure on Mobile
☒

☐ Above header i
☐ Below header i
☒ Above ticket list i

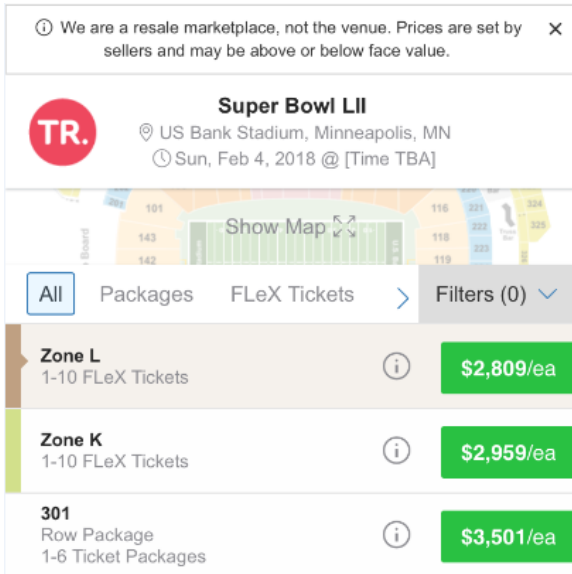
Mobile Disclosure Text
Use Default ☒ Use Custom

```

<div class="sea-list-disclosure-inner sea-list-marketing-inner">
  <span class="cm-info"></span>
  <span class="sea-list-marketing-label">Test sit are a resale marketplace, not the
venue. Prices are set by sellers and may be above or below face value.</span>
</div>

```

Or in code:



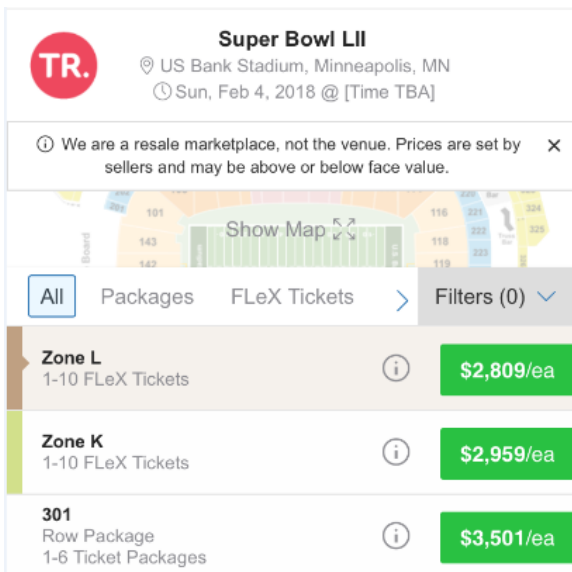
Disclosure at Top of Page

To display the disclosure at the top of the page in the small screen layout, set `enableHeaderLegalDisclosureMobile` to `true`.

Make sure that `enableLegalDisclosureMobile`—the default presentation of the disclosure—is set to `false` to prevent duplicate banner displays.

Example code:

```
<script>
  Seatics.config.enableLegalDisclosureMobile = false;
  Seatics.config.enableHeaderLegalDisclosureMobile = true;
</script>
```



Disclosure Below the Header

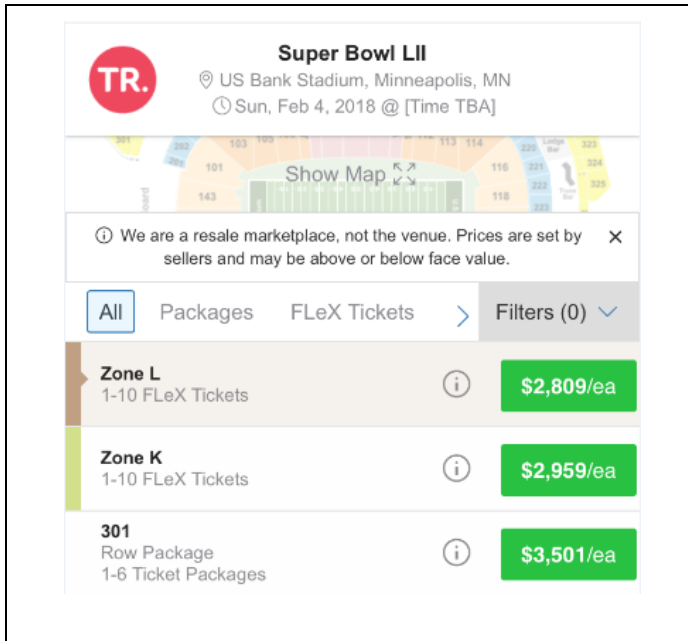
To display the disclosure below the header and above the map, set `enableHeaderLowerLegalDisclosureMobile` to `true`.

Make sure that `enableLegalDisclosureMobile`—the default presentation of the disclosure—is set to `false` to prevent duplicate banner displays.

Example code:

```
<script>
  Seatics.config.enableLegalDisclosureMobile = false;
  Seatics.config.enableHeaderLowerLegalDisclosureMobile = true;
</script>
```

Disclosure Below the Map



By default the disclosure will display below the map region and above the ticket list. To implement this option, nothing needs to be done unless you've previously disabled the small screen disclosure.

Example code:

```
<script>
    Seatics.config.enableLegalDisclosureMobile = true;
</script>
```

With all options the content of the disclosure banner is configurable as well. This allows you to edit the text to be consistent with the messaging across the rest of your site, if needed. The html of the reseller disclosure can be overridden using `Seatics.config.listLegalDisclosure`.

code snippet:

```
<script>
    Seatics.config.enableLegalDisclosureMobile = true;
    Seatics.config.listLegalDisclosure = '<div class="sea-list-marketing-inner">\
<span class="cm-info"></span>\
<span class="sea-list-marketing-label" id="sea-listdisclosurelabel">{{sea_disclosure_language}}</span>\
</div>';
</script>
```

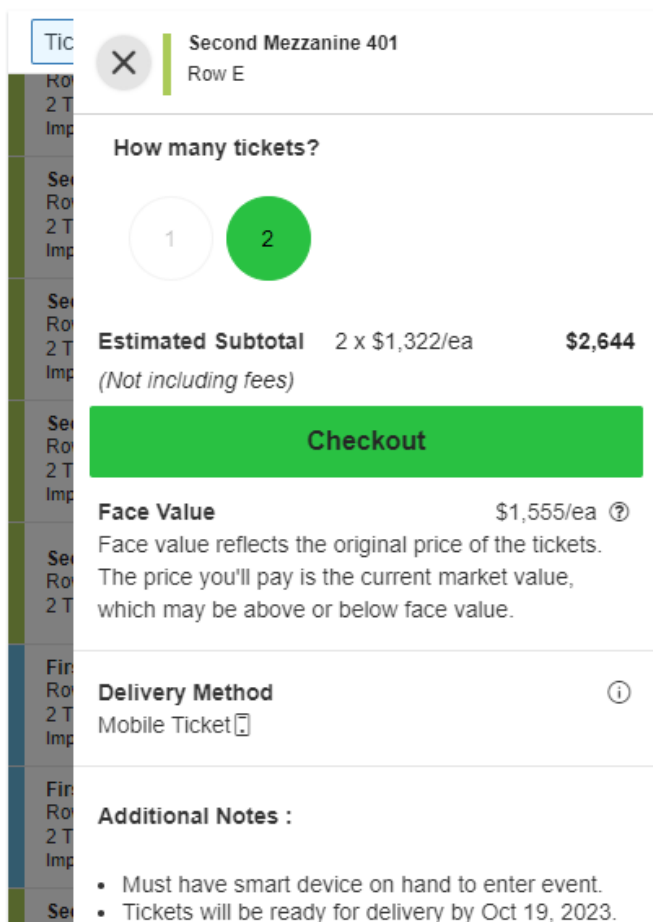
Should you decide to disable the disclosure completely, simply set any enabled disclosures to `false`. The banners will no longer display on your small screen layout.

```
Seatics.config.enableLegalDisclosureMobile = false;
Seatics.config.enableHeaderLegalDisclosureMobile = false;
Seatics.config.enableHeaderLowerLegalDisclosureMobile = false;
```

FACE VALUE DISPLAY

Google says that at a future date they intend to begin enforcing their next requirement to maintain certification to advertise through Adwords—surfacing face value on the ticket results page. Several configuration options were added in early 2018 in anticipation of this change. These are disabled by default, but can be enabled if desired.

FACE VALUE FOR REAL TICKETS



The screenshot shows a checkout interface for 'Second Mezzanine 401 Row E'. It includes a 'How many tickets?' section with a green circle selected for '2'. The 'Estimated Subtotal' is '2 x \$1,322/ea' totaling '\$2,644 (Not including fees)'. A green 'Checkout' button is prominent. Below it, the 'Face Value' section shows '\$1,555/ea' with a help icon and explanatory text: 'Face value reflects the original price of the tickets. The price you'll pay is the current market value, which may be above or below face value.' The 'Delivery Method' is 'Mobile Ticket' with an information icon. 'Additional Notes' include: 'Must have smart device on hand to enter event.' and 'Tickets will be ready for delivery by Oct 19, 2023.'

Face Value in the Ticket Details Screen

To enable the face value display on the ticket details screen, simply set `Seatics.config.displayFaceValuePreCheckout` to `true`.

Use `Seatics.config.faceValueHelpText` to edit the face value explanation. The default language is: "This is the original ticket price, as set by the primary seller. Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value."

To disable the explanation text altogether, pass in an empty string: `Seatics.config.faceValueHelpText = ""`;

example code:

```
<script>
    Seatics.config.displayFaceValuePreCheckout = true;
    Seatics.config.faceValueHelpText = "Face value
    reflects the original price of the tickets. The price you'll
    pay is the current market value, which may be above or below
    face value.";
</script>
```

If you are using the single click to checkout configuration, a user may never see the face value information. To comply with Google's policies, you may need to update your configuration settings so that the user **MUST** view the ticket details screen before proceeding to checkout.

Tickets Packages Parking Passes > Filters (0) v

Second Mezzanine 406

Row K

2 Tickets

\$1,250/ea

Continue

Face value reflects the original price of the tickets. The price you'll pay is the current market value, which may be above or below face value.

1,322/ea

Continue

Face Value: \$1,555

First Mezzanine 301

Row F

2 Tickets

\$1,475/ea

Continue

Important: Zone Seating

First Mezzanine 307

Row F

2 Tickets

\$1,475/ea

Continue

Important: Zone Seating

Second Mezzanine 403

Row K

2 Tickets

\$1,475/ea

Continue

Important: Zone Seating

Second Mezzanine 405

Row K

2 Tickets

\$1,475/ea

Continue

Important: Zone Seating

First Mezzanine 301

Row F

2 Tickets

\$1,522/ea

Continue

Face Value: \$1,790

Second Mezzanine 402

Face Value in the Main Ticket List

To enable the more visible face value display on the main ticket list, set `Seatics.config.displayFaceValueTicketList` to true.

As with the ticket details display, use `Seatics.config.faceValueHelpText` to edit the face value explanation. The default help text is: "This is the original ticket price, as set by the primary seller. Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value."

To disable the explanation text altogether, pass in an empty string: `Seatics.config.faceValueHelpText = ""`;

example code:

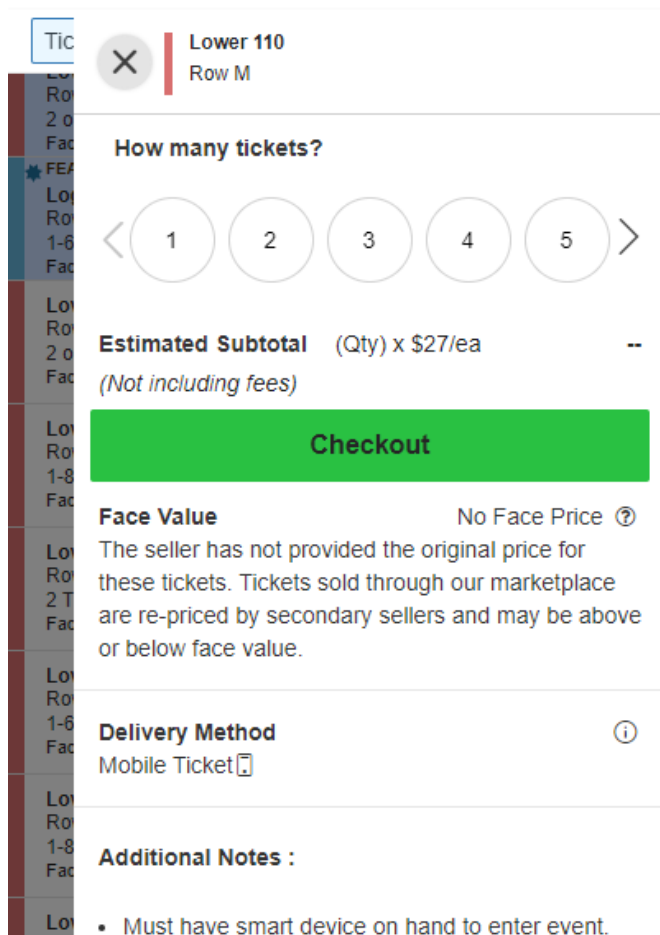
```
<script>
  Seatics.config.displayFaceValueTicketList = true;
  Seatics.config.faceValueHelpText = "Face value
  reflects the original price of the tickets. The
  price you'll pay is the current market value, which
  may be above or below face value.";
</script>
```

UNKNOWN FACE VALUE

In the case where "face" data has not been provided on a real ticket (or has been set to 0), the site can choose whether to show anything for face value. By default, Maps will not display any face value information or help text for unknown face values.

In order to toggle on the 'unknown' face value display, use the `Seatics.config.displayFaceValueForUnknownValues` configuration setting.

When unknown face values are shown, Maps will display "No Face Price" in place of where the face price would normally go. Along with this will be help text explaining that the information was not disclosed the face value to the system. Improvements will be made as needed based on Google's future plans and communications about Reseller Certification.



The screenshot shows a checkout page for a ticket. At the top, it says "Lower 110 Row M". Below that is a section "How many tickets?" with buttons for 1, 2, 3, 4, and 5. The "Estimated Subtotal" is shown as "(Qty) x \$27/ea" with a "--" next to it, and a note "(Not including fees)". A green "Checkout" button is prominent. Below the button, the "Face Value" section shows "No Face Price" with a help icon. The help text states: "The seller has not provided the original price for these tickets. Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value." The "Delivery Method" is set to "Mobile Ticket". Under "Additional Notes", it says "Must have smart device on hand to enter event."

When face value is null or \$0, Mapwidget displays the value as "No Face Price".

Where face value is displayed as "No Face Price," use `Seatics.config.faceValueHelpAlternateText` to edit the face value explanation. The default language is: "The seller has not provided the original price for these tickets. Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value."

To disable the explanation text altogether, pass in an empty string: `Seatics.config.faceValueHelpAlternateText = ""`;

example code:

```
<script>
    Seatics.config.displayFaceValuePreCheckout = true;
    Seatics.config.displayFaceValueForUnknownValues =
true;
    Seatics.config.faceValueHelpAlternateText = "The
seller has not supplied this data.";
</script>
```

FACE VALUE FOR SPECULATIVE TICKETS

The behavior for face value is different for speculative inventory because these do not represent a single, real ticket, and therefore cannot have a face value.

Ticket

Orchestra Left
Row H

How many tickets?

1

2

Estimated Subtotal

2 x \$290/ea

\$580

(Not including fees)

Checkout

Face Value

Not Applicable

Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value.

Delivery Method

eTickets

Important: Zone Seating

The tickets you receive may be in the specific section listed here or in an equivalent or better location. This listing describes tickets that the seller does not own or may not know the exact location of the seats, but is offering to obtain for you. Once your order is

When a ticket is Speculative/Category inventory, Mapwidget displays the value as "Not Applicable"

The face value explanation for speculative inventory is: "Tickets sold through our marketplace are re-priced by secondary sellers and may be above or below face value."

This setting cannot currently be configured.

CUSTOM GOOGLE RESELLER CERTIFICATION VIEW

In February 2018, Google began requiring event ticket sellers to obtain certification with them in order to advertise through Adwords. The first requirement to impact the Mapwidget UI was the inclusion and placement of ticket reseller disclosures in the small screen layout. Google says they will later require resellers to display the face value of tickets at some undetermined future date. Currently, compliance with Google's Reseller Certification requirements affects all site traffic, though the requirements are specific to Google pay-per-click traffic. Website owners have expressed an interest in creating a separate experience for Google PPC traffic so sites can adhere to Google's policies without drastically changing the customer experience for all marketing channels. Mapwidget provides a pair of configuration settings that will allow sites to auto-detect when the traffic source is from a Google PPC ad and render the desired Google-compliant Maps experience.

GOOGLE PPC AUTO-DETECTION

To enable Google PPC auto-detection, the site must add the following configuration parameter: `Seatics.config.googlePPCAutoDetect = true;` When enabled, Maps will use the following markers to identify Google PPC traffic:

1. The URL of the maps page contains the `gclid` parameter in the query string (`gclid` can have any value) OR
2. The browser has a cookie with the key `_gac_UA` (key can have any value) OR
3. The browser has a cookie with the key `_gcl_dc` (key can have any value).

If Maps determines the visitor was referred from a Google PPC ad, it will implement the site's Google Referral Settings, as explained in the next section.

GOOGLE REFERRAL SETTINGS

The `googleReferralSettings` configuration object contains a site's preferred Google Reseller Certification settings. Any of the following configuration parameters can be included:

```
enableHeaderLowerLegalDisclosureMobile
enableLegalDisclosureMobile
enableHeaderLegalDisclosureMobile
faceValueHelpText
faceValueHelpAlternateText
displayFaceValuePreCheckout
displayFaceValueTicketList
displayFaceValueForUnknownValues
```

These settings allow the site owner to specify:

- Whether or not the reseller disclosure should display on small screen devices, and where it should display when enabled
- Whether or not to enable face value display, and where that info should be surfaced when enabled
- Any customizations to face value explanation text, if applicable

For example, currently face value display is disabled for all sites, but the small screen reseller disclosure is enabled. If you want to display the disclosure only for Google PPC traffic and prefer placement between the header and the map, the following code should be added to the site's map settings:

```
Seatics.config.enableLegalDisclosureMobile = false;
Seatics.config.googlePPCAutoDetect = true;
```



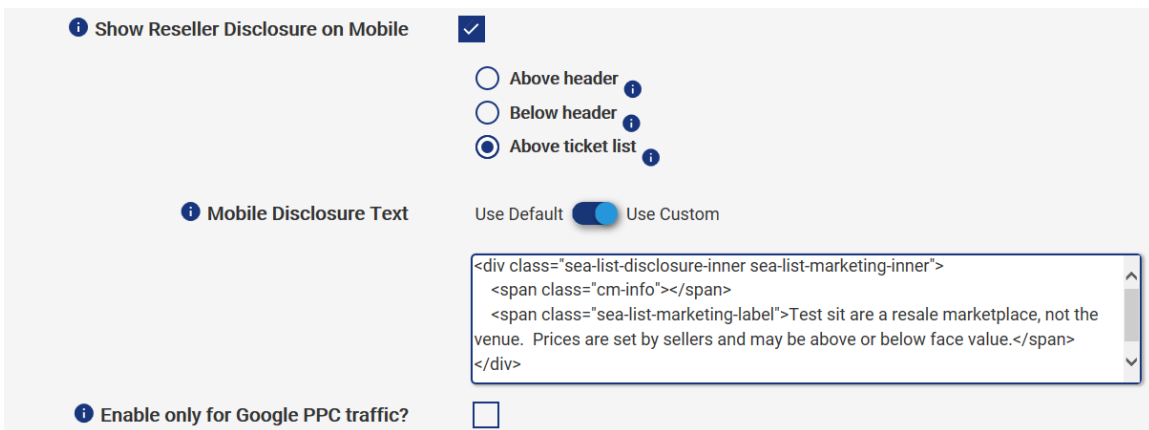
```
Seatics.config.googleReferralSettings = {
    enableHeaderLowerLegalDisclosureMobile: true
};
```

The first line disables the default legal disclosure that automatically displays on all sites using Mapwidget.

The second line enables Google PPC auto-detect.

The remaining lines specify what the user experience should be for Google PPC traffic, in this case, the reseller disclosure between the event info area and the map. The result is that **only** Google PPC traffic will now see the reseller disclosure.

In the Maps Configuration Management Tool, a checkbox below the Mobile Reseller Disclosure settings allows you to easily enable Google PPC auto-detect and apply your Mobile Reseller Disclosure settings only to Google PPC traffic. Check the box to enable these settings:



Show Reseller Disclosure on Mobile ☒

☐ Above header
 ☐ Below header
 ☒ Above ticket list

Mobile Disclosure Text Use Default ☒ Use Custom ☐

```
<div class="sea-list-disclosure-inner sea-list-marketing-inner">
  <span class="cm-info"></span>
  <span class="sea-list-marketing-label">Test sit are a resale marketplace, not the
venue. Prices are set by sellers and may be above or below face value.</span>
</div>
```

Enable only for Google PPC traffic? ☐

BYPASSING AUTO-DETECT

If you want to have a website experience specific to Google PPC traffic but don't want to use the Mapwidget autodetection, the configuration `Seatics.config.isGooglePPCReferral` can be used instead with your own logic for determining traffic source.

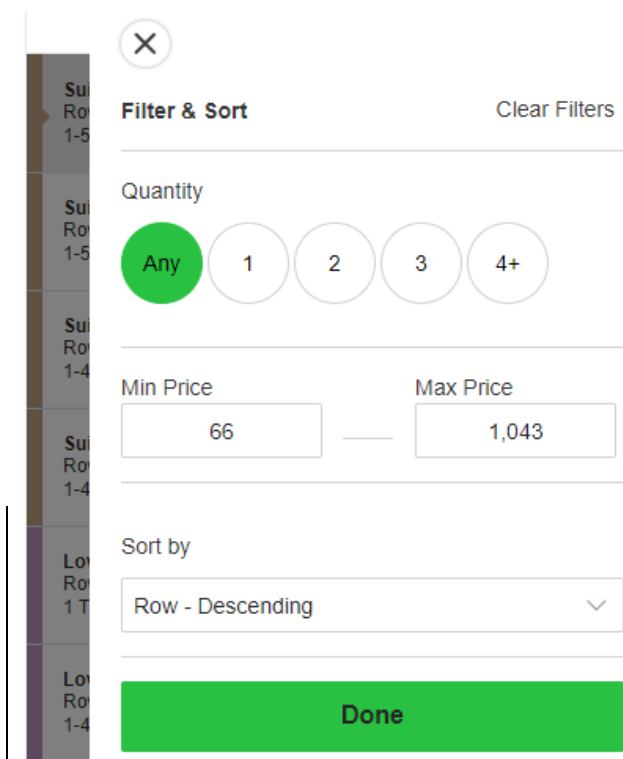
ALL-IN PRICING / FEE DISPLAY

Sites that want to show ticket fees to the user within the Map (in addition the base price of the ticket) can use the various configuration options to control service fee display.

Sites that do not want to show fees within the Map itself, but do want Checkout to show fees immediately to certain users (regardless of the normal Checkout behavior), can use the `Seatics.config.showFeesInCheckout` configuration option.

code snippet:

```
<script>
    Seatics.config.showAllInPricingToggle = false;
</script>
```



Use `Seatics.config.showAllInPricingToggle` to control whether the user is offered the option to toggle all-in pricing on and off within the filter card.

This setting is enabled by default. Set to `false` to hide this filter.

code snippet:

```
<script>
    Seatics.config.showAllInPricingToggle = false;
</script>
```

Suite A47

Row Peter Gunz VIP SRO

1-5 Tickets

Suite A47

Row Peter Gunz VIP Row 2

1-5 Tickets

Filter & Sort

Clear Filters

Quantity

Any

1

2

3

4+

Min Price

87

Max Price

1,388

Show prices with estimated fees

☒

Sort by

Row - Descending

Suite A47

Row Peter Gunz VIP SRO

1-5 Tickets

\$424.28/ea

Fees Included

Buy

Suite A47

Row Peter Gunz VIP Row 2

1-5 Tickets

\$605.15/ea

Fees Included

Buy

Use `Seatics.config.includeServiceFeesInTicketPrice` to display fees immediately when the map is first rendered. This will also toggle the “Show prices with estimated fees” filtercard setting to the “on” position.

This setting is disabled by default. Set to `true` to automatically enable all-in pricing.

code snippet:

```
<script>
  Seatics.config.includeServiceFeesInTicketPrice =
  true;
</script>
```

×

Suite A47

Row Peter Gunz VIP SRO

How many tickets?

<

1

2

3

4

5

>

Estimated Subtotal 1 x \$424.28/ea **\$424.28**

Includes \$105.28 in estimated fees

Go To Secure Checkout

PRICE BREAKDOWNS

Sites also have the option of displaying a breakdown of which portion of the price is for tickets and which is for fees. This feature is split into two configurations, one for the main list and one for the pre-checkout.

Price breakdowns will only be shown when the map is using all-in pricing. I.E. When either the user has toggled on all-in pricing within the filter card, or the site has chosen to enable all-in pricing automatically.

Suite A47
 Row Peter Gunz VIP SRO
 1-5 Tickets 

\$424.28/ea
Ticket Price \$319 + Service Fee \$105.28 + Taxes if applicable



Suite A47
 Row Peter Gunz VIP Row 2
 1-5 Tickets 

\$605.15/ea
Ticket Price \$455 + Service Fee \$150.15 + Taxes if applicable



Use `Seatics.config.showPriceBreakdownTicketList` to enable price breakdowns within the main list.

This setting is disabled by default. Set to `true` to enable the breakdowns.

code snippet:

```
<script>
    Seatics.config.showPriceBreakdownTicketList = true;
</script>
```

How many tickets?



1

2

3

4

5



Estimated Subtotal 1 x \$424.28/ea **\$424.28**
Ticket Price \$319 + Service Fee \$105.28 + Taxes if applicable



Use `Seatics.config.showPriceBreakdownPrecheckout` to enable price breakdowns within the main list.

This setting is disabled by default. Set to `true` to enable the breakdowns.

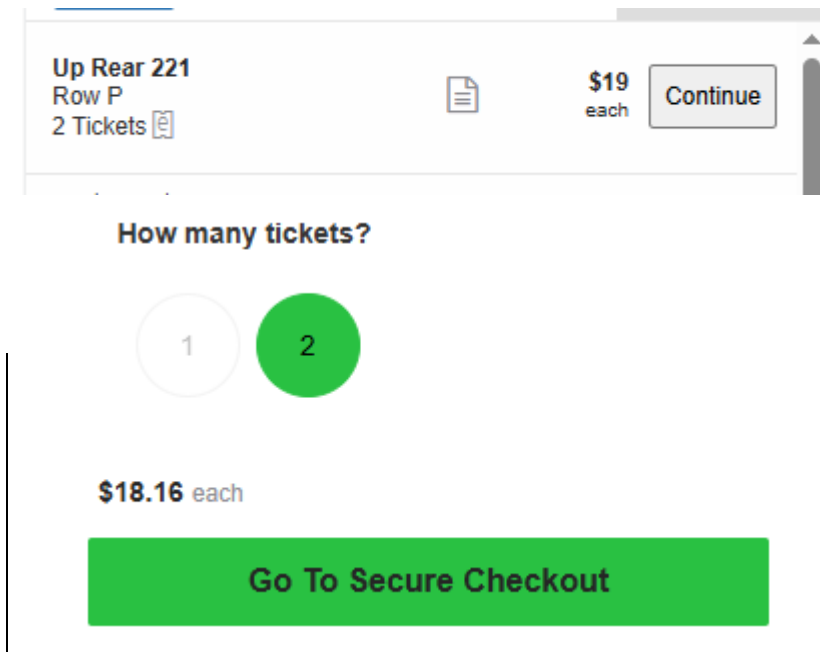
code snippet:

```
<script>
    Seatics.config.showPriceBreakdownPrecheckout = true;
</script>
```

“FEES INCLUDED” TEXT

By default, when a site is using all-in pricing but is not displaying price breakdowns, the text “Fees Included” will show in both the Ticket List and in Pre-checkout.

Sites that wish to turn this text off, for example if they already indicate that fees are included on some other part of the page, may do so by using the `Seatics.config.showFeesIncludedText` configuration option.



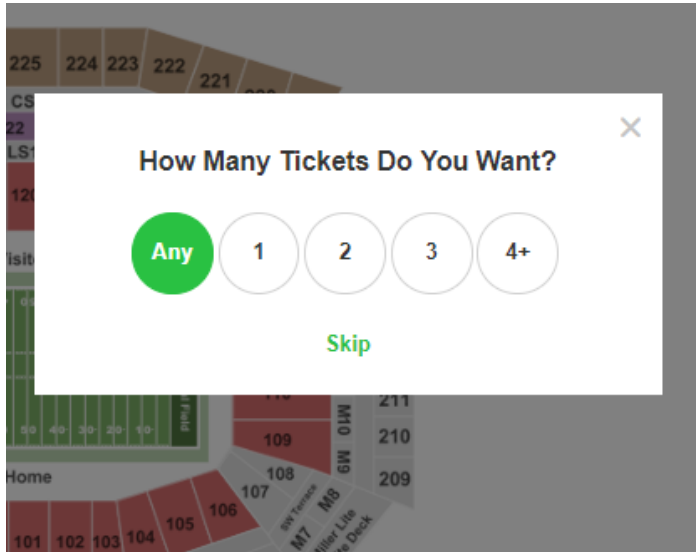
Use `Seatics.config.showFeesIncludedText` to show/hide the text “Fees Included” for all-in pricing displays on both the Ticket List and the Pre-checkout.

This setting is enabled by default. Set to `false` to hide the text.

code snippet:

```
<script>
    Seatics.config.showFeesIncludedText =
false;
</script>
```

QUANTITY MODAL



Use `Seatics.config.enableQuantityModal` to turn on the quantity modal feature, which prompts the user to select their ticket quantity when entering the maps page. The user has the option to skip this step or choose their quantity later.

This setting is disabled by default. Set to `true` to enable the modal.

Code snippet:

```
<script>
  Seatics.config.enableQuantityModal = true;
</script>
```

This configuration can also be set from the General tab in Maps Configuration Management. Check the box to enable the feature:

Quantity Modal	☐
When on, the user will be prompted to select quantity upon entering the page	

CHECKOUT DEEP LINKING

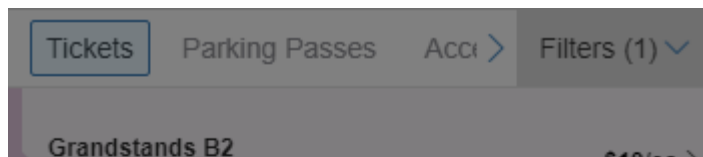
Use `Seatics.config.enableCheckoutDeepLinking` to enable or disable the deep linking feature, which allows the map to automatically direct the user straight to checkout to purchase a pre-defined group of tickets. Deep linking is typically used in marketing programs like cart abandonment when the user has already decided which exact ticket they want.

We highly recommend taking advantage of it, as it increases cart abandonment conversion rates by a significant amount.

This setting is enabled by default. Set to `false` to disable the feature.

Code snippet:

```
<script>
  Seatics.config.enableCheckoutDeepLinking = false;
</script>
```



If you are using the Pre-Built UI the user will see a message letting them know we're searching for specific tickets for them.

Just a moment

We're looking for your tickets...



If you are using a Custom UI you can optionally define an `onDeepLinking` function within your `map presentationInterface` that shows some kind of messaging to the user.

Code snippet:

```
<script>
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
});
```

```

presentationInterface: {
  updateTicketsList: myCreateListFunction
  onDeepLinking: function(deepLinkParams) {
    console.log("Deep linking you to your tickets");
  }
},
mapWidth: 525,
mapHeight: 545,
mapName: eventData.mapName,
enableSectionInfoPop-ups: true
});
</script>

```

In the case that the event doesn't exist (or has no tickets) the Pre-Built UI will automatically close the deep link messaging modal so that the user can see the default 'event expire/no tickets' messaging.

If you are using a Custom UI and you add messaging during the `onDeepLinking` step you will probably want to handle taking the messaging down in this case. You can either adjust your `noEventHandler` and `noTicketsHandler` or you can define an `onDeepLinkFailed` function within your map `presentationInterface` that handles the `Seatics.CheckoutDeepLinkOutcomes.EmptyEvent` outcome.

Code snippet:

```

<script>
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
    onDeepLinkFailed: function(outcome, deepLinkParams, ticketGroup) {
      if(outcome === Seatics.CheckoutDeepLinkOutcomes.EmptyEvent) {
        // ticketGroup will be null for this outcome
        console.log("Taking down deep linking messaging");
      }
    }
  },
  mapWidth: 525,
  mapHeight: 545,
  mapName: eventData.mapName,
  enableSectionInfoPop-ups: true
});
</script>

```


The deep link feature allows you to specify:

THE NAME OF THE CUSTOMER

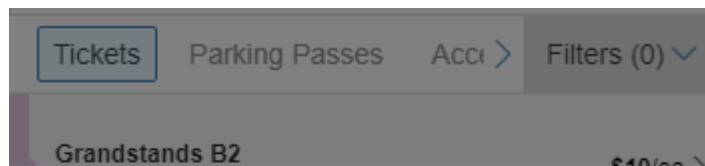
This can be used to personalize the messaging.

By default customer name is pulled from the `cstnm` query string parameter of the page's URL. Use `Seatics.config.checkoutDeepLinkCustomerNameKey` to change which query string parameter is used.

Code

snippet:

```
<script>
  Seatics.config.checkoutDeepLinkCustomerNameKey = "name";
</script>
```



If customer name is present and you are using the Pre-Built UI the 'loading' message will include the customer's name.

Just a moment Jane

We're looking for your tickets..



If you are using a Custom UI you can include the `deepLinkParams.customerName` in any of the deep link messaging. This property is available in both the `onDeepLinking` input and the `onDeepLinkFailed` input. It will be undefined or empty if not provided.

Code snippet:

```
<script>
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
    onDeepLinking: function(deepLinkParams) {
      // make sure to html encode customerName if you're including it in your UI's html. You
      // can use Seatics.Presentation.htmlEncode();
      console.log("Deep linking you to your tickets " + deepLinkParams.customerName);
    }
  }
},
```

```
mapWidth: 525,
mapHeight: 545,
mapName: eventData.mapName,
enableSectionInfoPop-ups: true
});
</script>
```

THE TICKET GROUP TO BUY

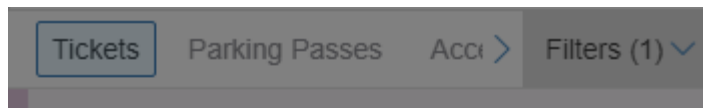
This should match the id of the tickets within the ticket data array.

Maps will check to make sure the tickets are still available before sending the user to checkout. If they are not, the user will not be sent to checkout.

By default ticket group id is pulled from the `tgid` query string parameter of the page's URL. Use `Seatics.config.checkoutDeepLinkTgIDKey` to change which query string parameter is used.

Code snippet:

```
<script>
    Seatics.config.checkoutDeepLinkTgIDKey = "ticket";
</script>
```



We're sorry!

Those tickets are no longer available.

Please select new tickets



If tickets are not available and you are using the Pre-Built UI the user will see a message, and be prompted to pick new tickets.

If you are using a Custom UI you can optionally define an `onDeepLinkFailed` function within your map `presentationInterface` that handles the `Seatics.CheckoutDeepLinkOutcomes.TicketGone` outcome and shows messaging to the user for this failure case.

*Code snippet:*

```
<script>
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
    onDeepLinkFailed: function(outcome, deepLinkParams, ticketGroup) {
      if(outcome === Seatics.CheckoutDeepLinkOutcomes.TicketGone) {
        // ticketGroup will be null for this outcome
        console.log("Those tickets aren't available anymore");
      }
    }
  },
  mapWidth: 525,
  mapHeight: 545,
  mapName: eventData.mapName,
  enableSectionInfoPop-ups: true
});
</script>
```

THE QUANTITY OF TICKETS TO BUY

Maps will check to make sure this quantity is still available before sending the user to checkout. If it is not, the user will not be sent to checkout.

If the quantity parameter is omitted, the quantity availability check will be skipped, and the user will be sent to checkout with the default number of tickets for that group (usually 2).

By default quantity is pulled from the qty query string parameter of the page's URL. Use `Seatics.config.checkoutDeepLinkQuantityKey` to change which query string parameter is used.

Code snippet:

```
<script>
  Seatics.config.checkoutDeepLinkQuantityKey = "quantity";
</script>
```

[Tickets](#)
[Parking Passes](#)
[Account](#)
[Filters \(1\) ▼](#)

We're sorry!

You can no longer purchase 4 of these tickets.

Please select new tickets

Row Row • 1-12 or 14 Tickets [E]

If the quantity is unavailable and you are using the Pre-Built UI the user will see a message, and be prompted to pick new tickets.

If you are using a Custom UI you can optionally define an `onDeepLinkFailed` function within your map `presentationInterface` that handles the `Seatics.CheckoutDeepLinkOutcomes.QuantityUnavailable` outcome and shows messaging to the user for this failure case.

Code snippet:

```
<script>
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
    onDeepLinkFailed: function(outcome, deepLinkParams, ticketGroup) {
      if(outcome === Seatics.CheckoutDeepLinkOutcomes.QuantityUnavailable) {
        // NOTE: deeplinkParams.quantity will be a number
        console.log("You can't buy " + deepLinkParams.quantity + " of those tickets
        anymore");
      }
    }
  },
  mapWidth: 525,
  mapHeight: 545,
  mapName: eventData.mapName,
  enableSectionInfoPop-ups: true
});
</script>
```

THE PRICE (EACH) THE USER EXPECT TO PAY FOR THE TICKETS

Maps will make sure the price of the tickets hasn't increased before sending the user to checkout. If it has, the user will not be sent to checkout.

If the price parameter is omitted, the price increase check will be skipped.

By default expected price (each) is pulled from the `prc` query string parameter of the page's URL. Use `Seatics.config.checkoutDeepLinkExpectedPriceKey` to change which query string parameter is used.

Code snippet:

```
<script>
  Seatics.config.checkoutDeepLinkExpectedPriceKey = "price";
</script>
```



Aw man!

The price of these tickets has gone up to \$64 from \$60.

That's cool, start my order

Never mind, I'll pick new tickets

If the price has gone up and you are using the Pre-Built UI the user will see a message, and they can choose to continue or pick new tickets.

If you are using a Custom UI you can optionally define an `onDeepLinkFailed` function within your `map presentationInterface` that handles the `Seatics.CheckoutDeepLinkOutcomes.PriceIncrease` outcome and shows messaging to the user for this failure case.

Code snippet:

```
<script>
Seatics.MapComponent.create({
  imgSrc: eventData.mapImage,
  tickets: ticketData,
  mapData: mapData,
  vfsUrl: 'https://vfs.seatics.com',
  container: container,
  presentationInterface: {
    updateTicketsList: myCreateListFunction
    onDeepLinkFailed: function(outcome, deepLinkParams, ticketGroup) {
      if(outcome === Seatics.CheckoutDeepLinkOutcomes.PriceIncrease) {
        // NOTE: deepLinkParams.expectedPrice will be a number
      }
    }
  }
});
```

```

        console.log("These tickets used to be " +
Seatics.formatCurrency(deepLinkParams.expectedPrice) + " but now they're " +
Seatics.formatCurrency(ticketGroup.tgPrice));
    }
},
mapWidth: 525,
mapHeight: 545,
mapName: eventData.mapName,
enableSectionInfoPop-ups: true
});
</script>

```

This feature can also be configured from the General tab in Maps Configuration Management. Uncheck the box to disable the feature. Or set each parameter to 'Custom' and adjust the text in the field to change which query string parameter they are read from.

Example: <https://mysite.com/event/927771?tgid=112455&qty=3&prc=52&cstnm=Jane>

i Deep Link to Checkout Allows deep linking to checkout using query string parameters	☒
i Query String Key for tgID Use Default ☒ Use Custom	tgid
i Query String Key for quantity Use Default ☒ Use Custom	qty
i Query String Key for expected price Use Default ☒ Use Custom	prc
i Query String Key for customer name Use Default ☒ Use Custom	cstnm

Some sites might want to use an alternate way to specify deep links other than the query string. For these cases a site can provide their own functions to determine each deep link parameter value. If configured, these functions will take precedence over the query string setting. You can mix and match query string and custom functions – maps will only use the custom functions you’ve defined and will fall back to using the query string if you haven’t specified a custom function.

Example code:

```
<script>
  Seatics.config.checkoutDeepLinkTgID = function(){ return 1234775; };
  Seatics.config.checkoutDeepLinkQuantity = function(){ return 3; };
  Seatics.config.checkoutDeepLinkExpectedPrice = function(){ return 60; };
  Seatics.config.checkoutDeepLinkCustomerName = function(){ return "Jane"; };
</script>
```

FEEDBACK FEATURE

Feedback?

We appreciate your feedback. i

Questions, comments, concerns?

Include your e-mail address? (optional)

So we can send you a response.

By hitting Submit, you agree to our [Idea Submission Policy](#).

Use `Seatics.config.enableFeedback` to enable or disable the feedback feature, which allows the user to submit comments, suggestions, and questions about the maps experience. Feedback is emailed to an internal TicketNetwork team. That group responds to user questions and forwards comments to the appropriate TicketNetwork departments.

This setting is enabled by default. Set to `false` to disable the feature.

Code snippet:

```
<script>
  Seatics.config.enableFeedback = true;
</script>
```

This configuration can also be set from the General tab in Maps Configuration Management. Uncheck the box to disable the feature:

i **Feedback Button**
☒

Collects user input about the maps

MAP COLOR SCHEMES



Full Palette

Maps are colored by venue level. Large venues can use up to eighteen colors

code snippet:

```
<script>
  Seatics.config.levelColors =
    ["#d86f6f", "#e2a6a6", "#f4ad58", "#ffd797",
     "#bc91c1", "#e5cae2", "#66b6dd", "#a3ddf8",
     "#abcb59", "#d2e08b", "#bfa082", "#d9d7ba",
     "#589caf", "#bcd3e7", "#fff871", "#fff7a9",
     "#9bc9a2", "#c7ddc8"];
  Seatics.config.useSuperLevels = false;
</script>
```



Simplified Palette

Similar to Full Palette, but for complex venues like baseball stadiums, levels are grouped together into 'superlevels' to simplify the appearance of the map. This is the default color scheme.

Code snippet:

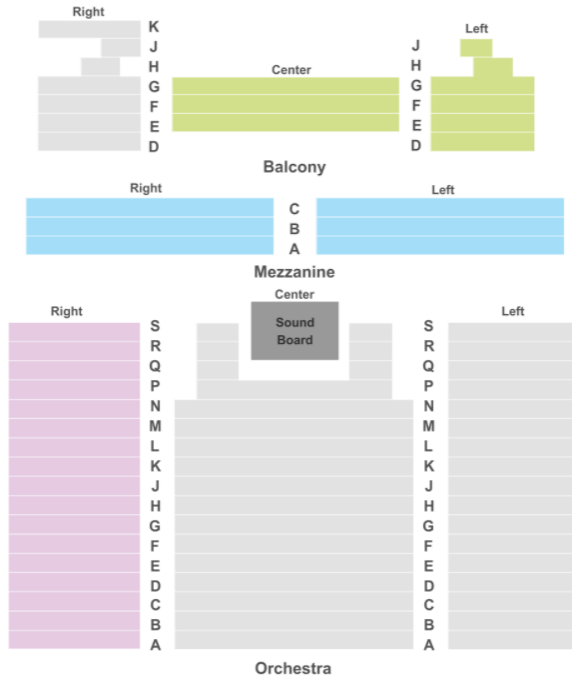
```
<script>
  Seatics.config.levelColors =
    ["#d86f6f", "#e2a6a6", "#f4ad58", "#ffd797",
     "#bc91c1", "#e5cae2", "#66b6dd", "#a3ddf8",
     "#abcb59", "#d2e08b", "#bfa082", "#d9d7ba",
     "#589caf", "#bcd3e7", "#fff871", "#fff7a9",
     "#9bc9a2", "#c7ddc8"];
  Seatics.config.useSuperLevels = true;
</script>
```

Small Palette

Small venues with three levels or less are configured separately and use a subset of the large color pallet.

Code snippet:

```
<script>
    Seatics.config.levelColorsSmallVenue =
    ["#e5cae2", "#a3ddf8", "#d2e08b"];
</script>
```



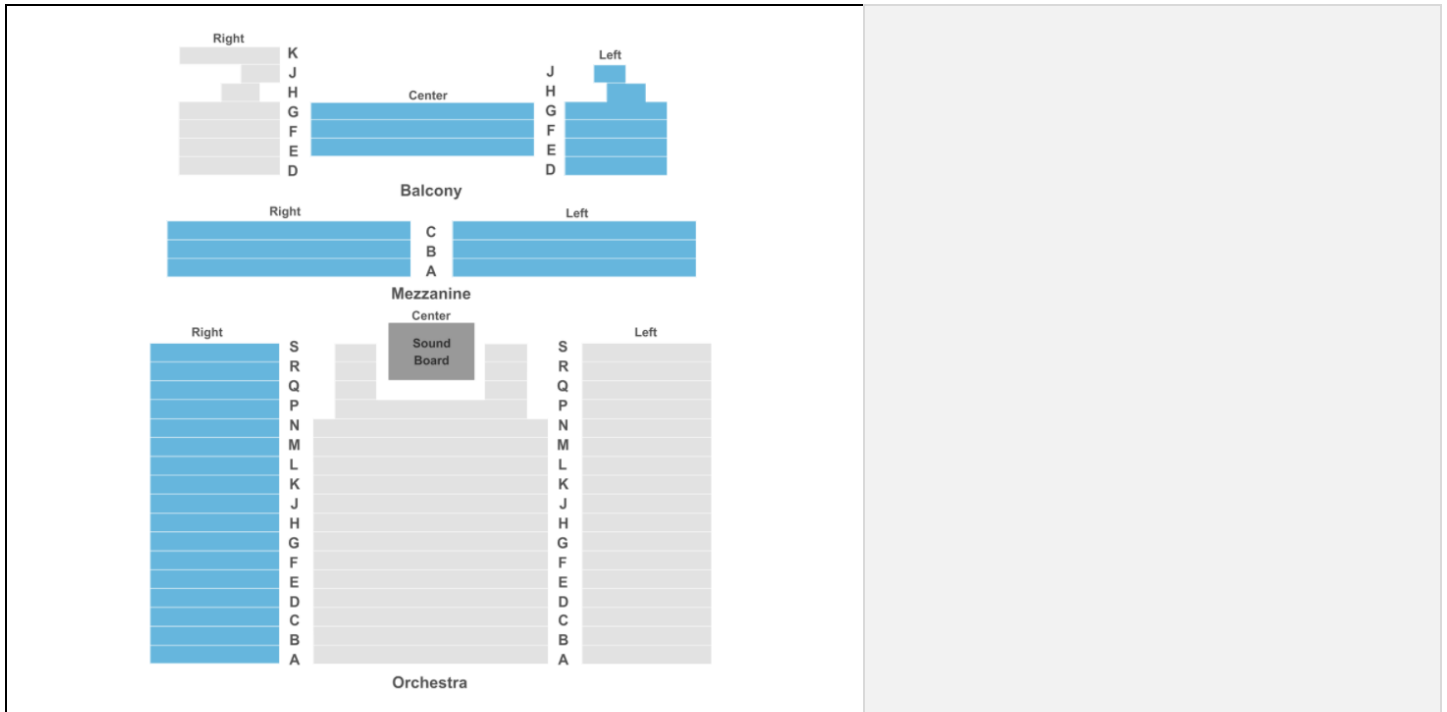
Single Color

Sections with tickets available are colored and sections with no tickets available are grey. The color for sections with inventory available is configurable through use of the `Seatics.config.levelColors` and `Seatics.config.levelColorsSmallVenue` properties. Specify a single color instead of an array of colors.

Code snippet:

```
<script>
    Seatics.config.levelColors =
    Seatics.config.levelColorsSmallVenue =
    ["#66b6dd"];
</script>
```





These configurations can also be set from the Venue tab in Maps Configuration Management:

i **Seating Chart Color Palette**
Colors for sections with inventory

Use Default ☒ Use Custom

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

i **Section with No Available Tickets**
Use Default ☒ Use Custom

▼

i **Simplified Palette**
Simplifies section groupings on complex maps

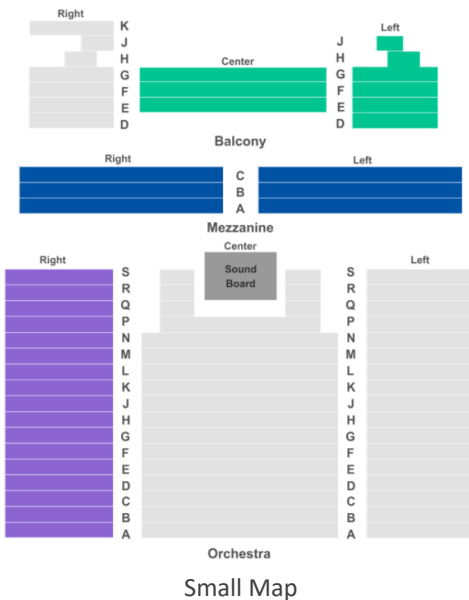
☒

Single Color Palette
Uses only the first color of the seating chart color palette

☐



Regular Map



Small Map

Notes About the Color Palette

The maps color palette was carefully selected by our designer with warmer colors closer to the action and cooler colors as the seats get farther away. Choices were informed by color theory and accommodate colorblind users. If a site owner wants to use a different palette, however, that is possible by changing a couple configuration settings.

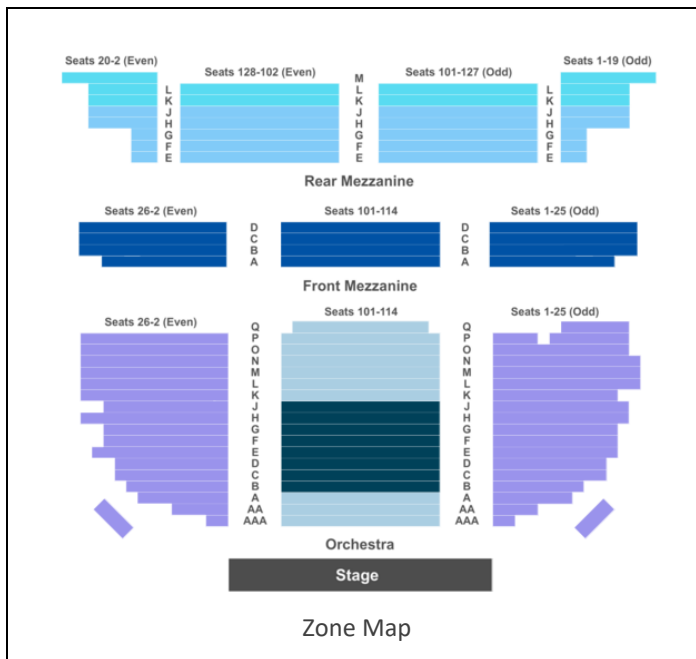
A new array of level colors can be specified with `Seatics.config.levelColors` and `Seatics.config.levelColorsSmallVenue`.

To update zone maps, the same palette must be specified with `Seatics.config.zoneColors`. Zone maps should always have a full color array specified, even if the site is using the single color scheme for non-zone maps.

To avoid repeating colors and creating customer confusion, it is imperative that enough colors are included in the array when changes are made to the full palette, simplified palette, or zone color scheme.

Code snippet:

```
<script>
  Seatics.config.levelColors = ["#004159", "#65a8c4",
    "#aacee2", "#8c65d3", "#9a93ec", "#cab9f1", "#0052a5",
    "#413bf7", "#81cbf8", "#00adce", "#59dbf1", "#9ee7fa",
    "#00c590", "#73ebae", "#b5f9d3", "#846d74", "#b7a6ad",
    "#d3c9ce"];
  Seatics.config.levelColorsSmallVenue = ["#8c65d3",
    "#0052a5", "#00c590"];
  Seatics.config.zoneColors = ["#004159", "#65a8c4",
    "#aacee2", "#8c65d3", "#9a93ec", "#cab9f1", "#0052a5",
    "#413bf7", "#81cbf8", "#00adce", "#59dbf1", "#9ee7fa",
    "#00c590", "#73ebae", "#b5f9d3", "#846d74", "#b7a6ad",
    "#d3c9ce"];
</script>
```



SECTION SELECTION SCHEMES



Scheme 0

When set to `Seatics.config.selectionScheme = 0`, selecting a section desaturates the colors of unselected sections. This is the default selection scheme.

Code snippet:

```
<script>
    Seatics.config.selectionScheme = 0;
</script>
```




Scheme 1

When set to `Seatics.config.selectionScheme = 1`, selecting a section changes the fill color of the selected section to #ffff00 or the color set with `Seatics.config.selectionColor`.

code snippet:

```
<script>
    Seatics.config.selectionScheme = 1,
    Seatics.config.selectionColor = "#7FFF00";
</script>
```

This can also be set through the Venue tab in Maps Configuration Management:

☒ **Section Selection Scheme**

☒ **Desaturate unselected sections**

☐ **Change fill color of selected sections to**

SMALL SCREEN LAYOUT CONFIGURATION

Fully Shown Map

`Seatics.config.smallScreenMapLayout` is used to choose how much of the map to display on page load. When set to `Seatics.SmallScreenMapOptions.FullyShown`, the map is fully displayed on page load with the option to hide it to view more of the ticket list.

Code snippet:

```
<script>
    Seatics.config.smallScreenMapLayout =
    Seatics.SmallScreenMapOptions.FullyShown;
</script>
```


Boston Red Sox vs. Toronto Blue ...
 Fenway Park, Boston, MA
 Wed, Sep 27, 2017 @ 7:10PM



[All](#)
[Packages](#)
[Parking Passes](#)
[Filters \(0\)](#)

Lower Bleachers 37 Row 4 • 2 Tickets	\$12/ea >
☆ Lower Bleachers 38 Row 18 • 1-6 Tickets ↓	\$12/ea >
Lower Bleachers 41 Row 28 • 2-4 Tickets ↓	\$12/ea >


Boston Red Sox vs. Toronto Blue ...
 Fenway Park, Boston, MA
 Wed, Sep 27, 2017 @ 7:10PM



[All](#)
[Packages](#)
[Parking Passes](#)
[Filters \(0\)](#)

Lower Bleachers 37 Row 4 • 2 Tickets	\$12/ea >
☆ Lower Bleachers 38 Row 18 • 1-6 Tickets ↓	\$12/ea >
Lower Bleachers 41 Row 28 • 2-4 Tickets ↓	\$12/ea >
☆ Lower Bleachers 36 Row 20 • 1 Ticket ↓	\$13/ea >
☆ Lower Bleachers 37 Row 19 • 1-3 Tickets ↓	\$13/ea >

50% Map

When `Seatics.config.smallScreenMapLayout` is set to `Seatics.SmallScreenMapOptions.HalfShown`, half of the map is shown with the ability to expand the map area to reveal the full map.

Code snippet:

```
<script>
  Seatics.config.smallScreenMapLayout =
  Seatics.SmallScreenMapOptions.HalfShown;
</script>
```

Mostly Hidden Map

TR. Boston Red Sox vs. Toronto Blue ...
 Fenway Park, Boston, MA
 Wed, Sep 27, 2017 @ 7:10PM

Show Map

All Packages Parking Passes > Filters (0) v

Lower Bleachers 37 Row 4 • 2 Tickets	\$12/ea >
☆ Lower Bleachers 38 Row 18 • 1-6 Tickets	\$12/ea >
Lower Bleachers 41 Row 28 • 2-4 Tickets	\$12/ea >
☆ Lower Bleachers 36 Row 20 • 1 Ticket	\$13/ea >
☆ Lower Bleachers 37 Row 19 • 1-3 Tickets	\$13/ea >
☆ Lower Bleachers 37 Row 20 • 1-3 Tickets	\$13/ea >
☆ Lower Bleachers 39 Row 30 • 2 Tickets	\$13/ea >

When `Seatics.config.smallScreenMapLayout` is set to `Seatics.SmallScreenMapOptions.HiddenWithPreview`, a sliver of the map displays, allowing the user to know a map is available while giving more room for ticket groups. Tapping 'Show Venue Map' expands the region to reveal the full map.

This is the default setting.

Code snippet:

```
<script>
  Seatics.config.smallScreenMapLayout =
  Seatics.SmallScreenMapOptions.HiddenWithPreview;
</script>
```

TR. Boston Red Sox vs. Toronto Blue ...
 Fenway Park, Boston, MA
 Wed, Sep 27, 2017 @ 7:10PM

Show Map

All Packages Parking Passes > Filters (0) v

Lower Bleachers 37 Row 4 • 2 Tickets	\$12/ea >
☆ Lower Bleachers 38 Row 18 • 1-6 Tickets	\$12/ea >
Lower Bleachers 41 Row 28 • 2-4 Tickets	\$12/ea >
☆ Lower Bleachers 36 Row 20 • 1 Ticket	\$13/ea >
☆ Lower Bleachers 37 Row 19 • 1-3 Tickets	\$13/ea >
☆ Lower Bleachers 37 Row 20 • 1-3 Tickets	\$13/ea >
☆ Lower Bleachers 39 Row 30 • 2 Tickets	\$13/ea >

Fully Hidden Map

When `Seatics.config.smallScreenMapLayout` is set to `Seatics.SmallScreenMapOptions.FullyHidden`, no part of the map shows at all, giving maximum focus to the ticket groups. Tapping 'Show Venue Map' expands the region to reveal the full map.

Code snippet:

```
<script>
  Seatics.config.smallScreenMapLayout =
  Seatics.SmallScreenMapOptions.FullyHidden;
</script>
```


This configuration can also be set through the Venue tab in Maps Configuration Management:

 **Maps Display on Mobile**
How much of the map is visible on page load

☐ Fully Shown
☐ Half Shown
☒ Hidden with Preview
☐ Fully Hidden

SCROLL TO ZOOM

Some site owners prefer to allow users to scroll their mouse wheels to zoom the map. Others find this scroll to zoom functionality ‘hijacks’ the page scroll. By default, scroll to zoom is disabled. However, if a site owner would like to enable this feature, it can be toggled on by setting `Seatics.config.mouseWheelZoomEnabled` to `true`.

Code snippet:

```
<script>  
  Seatics.config.mouseWheelZoomEnabled = true;  
</script>
```

This configuration can also be set through the Venue tab in Maps Configuration Management. Check the box to enable scroll to zoom:

 **Zoom with Mouse Wheel**
When on, users can zoom the map by scrolling the mouse wheel

☐

ZOOM CONTROLS



Use `Seatics.config.showZoomControls` to enable plus and minus buttons users can tap to zoom on mobile devices. By default, the controls are not displayed on the stacked layout. Pinch to zoom is supported whether or not zoom controls are displayed.

Please note that if you set this to `false`, it will hide the zoom controls on desktop. To hide the controls on small screen but have them visible on large screen, do not explicitly set this parameter.

Code snippet:

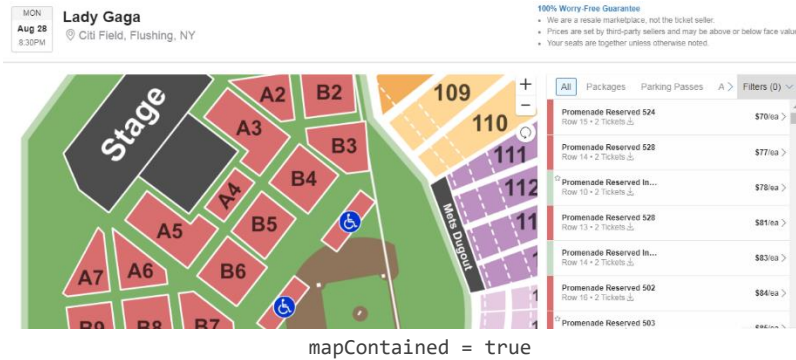
```
<script>
  Seatics.config.showZoomControls = true;
</script>
```

This configuration can also be set through the Venue tab in Maps Configuration Management. Check the box to display plus and minus zoom controls on mobile:

Show Zoom Controls on Mobile
Adds plus and minus controls to mobile

☐

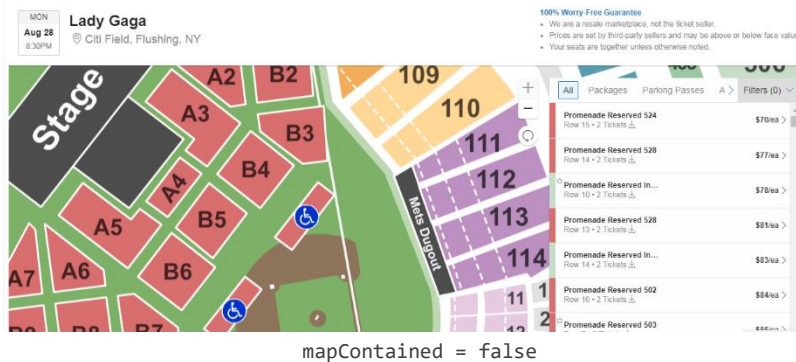
MAP CONTAINER



Seatics.config.mapContained supports two configurations.

The first is the 'traditional' configuration where the map is wrapped in its own container to free up the surrounding area for customization and branding.

The second option is the 'modern' configuration where the page is streamlined and the focus is on the map, which functions as if it is the background on the page. By default, the map is not contained.



Code snippet:

```
<script>
    Seatics.config.mapContained = true;
</script>
```

This configuration can also be set through the Venue tab in Maps Configuration Management. Check the box to restrict the map from zooming or panning outside its container. It's recommended to enable this if your map page has a footer:

Map Contained ☐

When on, the map will not overflow the container

LEGEND DISPLAY

There are several options for the display of the map legend on the large screen layout. These can be configured through the Venue tab in Maps Configuration Management:

i Large Screen Legend Display

☒ Dropdown style

Legend expanded:

☐

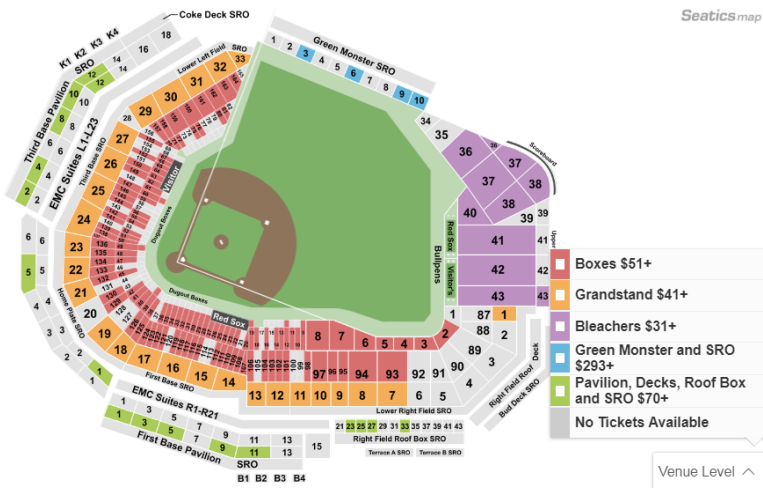
☐ Checkbox style

Show legend first on mobile:

☐

When on, the user is asked to select levels/zones before viewing the ticket list

Or through code:



`Seatics.config.legendExpanded` specifies how the default map legend should display on page load. The legend is collapsed by default. Set to true to display the legend expanded.

Code snippet:

```
<script>
    Seatics.config.legendExpanded = true;
</script>
```

ALTERNATE LEGEND PRESENTATION

Verizon LTE

1:24 PM

46%

output.jsbin.com

TR.

Boston Red Sox vs. Tampa Bay R...
Fenway Park, Boston, MA
Sun, Aug 19, 2018 @ 1:05PM



Hide Map

What section would you like to sit in?

Boxes \$52+

Grandstand \$45+

Bleachers \$44+

Green Monster and SRO \$348+

Show Available Tickets ^

<

>

Share

Bookmark

Print

The default presentation of the legend keeps it out of the way so the map takes focus. On the large screen layout, the legend can be configured to be expanded on page load; however, it may cover part of the map. Another option for those that want the legend always visible is an alternate presentation below the map. Set `legendDriven` to true to enable the ever-present legend.

To encourage customers to use the map on small screen devices to narrow down purchase options before viewing the ticket list, you can pair `Seatics.SmallScreenMapOptions.FullyShown`, (displays the full map on mobile) with `showLegendFirstOnMob` to display the legend on page load. Once the user taps into the ticket list, the legend options are available from the filter menu.

Please note that you must set both `legendDriven` and `showLegendFirstOnMob` to enable the alternate legend display on small screen devices. Setting only `legendDriven` will enable the alternate presentation just for the large screen layout.

Code snippet:

```
<script>
  Seatics.config.ticketsState = {
    legendDriven: true,
    showLegendFirstOnMob:true};
  Seatics.config.smallScreenMapLayout =
    Seatics.SmallScreenMapOptions.FullyShown;
</script>
```

TICKETRIGHT.
CONCERTS ▾
SPORTS ▾
THEATER ▾

SAT
Sep 9
7:10PM

Boston Red Sox vs. Tampa Bay ...
📍 Fenway Park, Boston, MA

100% Worry-Free Guarantee

- We are a resale marketplace, not the ticket seller.
- Prices are set by third-party sellers and may be above or below face value.
- Your seats are together unless otherwise noted.

All Packages Parking Passes > Filters (0) ▾

Upper Bleachers 38	Row 34	2 Tickets	\$30/ea \$32	Buy
Upper Bleachers 38	Row 38	1-9 Tickets	\$31/ea \$33	Buy
Upper Bleachers 43	Row 47	2 Tickets	\$31/ea \$34	Buy
Upper Bleachers 38	Row 34	1-5 Tickets	\$32/ea \$35	Buy
Lower Bleachers 37	Row 20	2 Tickets	\$33/ea \$36	Buy
Upper Bleachers 37	Row 35	1-9 Tickets	\$35/ea \$39	Buy
Lower Bleachers 37	Row 39	2 Tickets	\$36/ea \$39	Buy



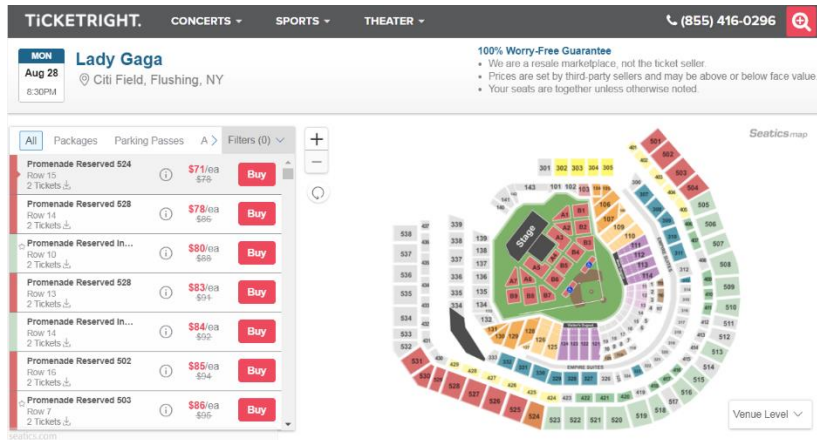
Seatics map

- Boxes \$51+
- Grandstand \$41+
- Bleachers \$30+
- Green Monster and SRO \$290+
- Pavilion, Decks, Roof Box and SRO \$70+

code snippet:

```
<script>
  Seatics.config.ticketsState = {
    legendDriven: true
  };
</script>
```


LARGE SCREEN MAPS PLACEMENT



`Seatics.config.ticketListOnRight` is used to specify whether the ticket list should appear on the left or right side of the page. By default, the ticket list is on the right and the map is on the left. Setting the configuration parameter to `false` puts the ticket list on the left and the map on the right.

code snippet:

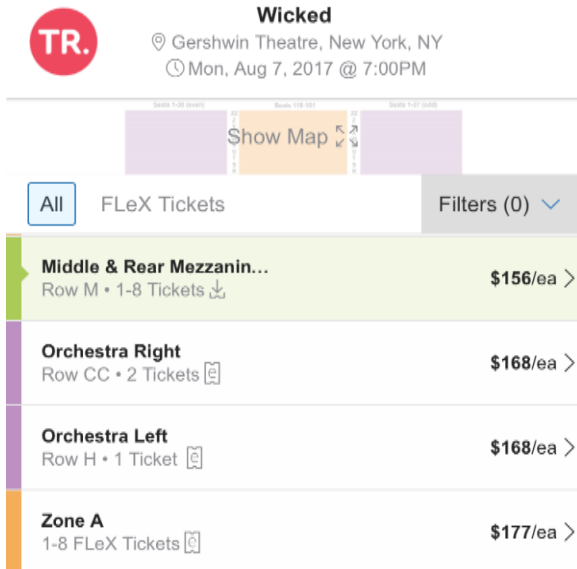
```
<script>
    Seatics.config.ticketListOnRight =
    false;
</script>
```

This configuration can also be set through the Venue tab in Maps Configuration Management:

i **Large Screen Maps Placement**

☒ **Maps on left**
☐ **Maps on right**

CTA BEHAVIOR ON MOBILE



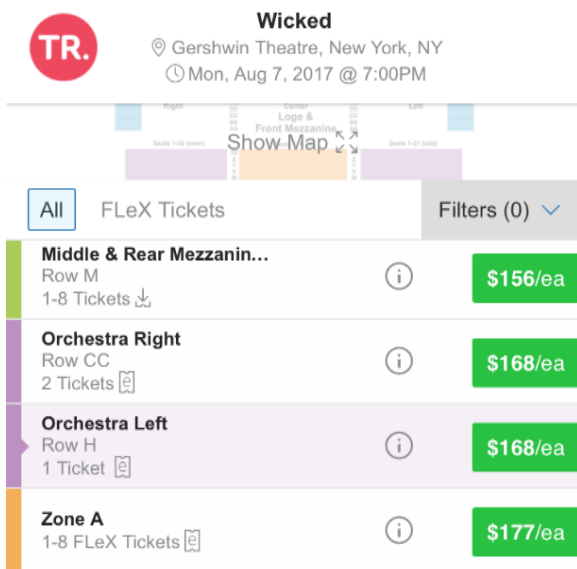
Two Clicks to Checkout

The two click configuration requires the user to view additional ticket details in a pre-checkout screen before proceeding to make a purchase.

This is the default behavior.

Code snippet:

```
<script>
  Seatics.config.skipPrecheckoutMobile = false;
</script>
```



Single Click to Checkout

Set `Seatics.config.skipPrecheckoutMobile` to `true` to make it optional for the user to view the additional ticket information. The CTA is given stronger visual emphasis and takes the user directly to checkout. Tapping elsewhere in the row opens the additional ticket group details.

Code snippet:

```
<script>
  Seatics.config.skipPrecheckoutMobile = true;
</script>
```

This configuration can also be set through the Ticket List tab in Maps Configuration Management:

i **Single Click Checkout on Mobile**



CTA BEHAVIOR ON LARGE SCREEN

All	Packages	Parking Passes	A >	Filters (0) v
Promenade Reserved 524	Row 15 • 2 Tickets	\$70/ea	>	
Promenade Reserved 528	Row 14 • 2 Tickets	\$77/ea	>	
☆ Promenade Reserved In...	Row 10 • 2 Tickets	\$78/ea	>	
Promenade Reserved 528	Row 13 • 2 Tickets	\$81/ea	>	
Promenade Reserved In...	Row 14 • 2 Tickets	\$83/ea	>	
Promenade Reserved 502	Row 16 • 2 Tickets	\$84/ea	>	
☆ Promenade Reserved 503	Row 12 • 1-8 Tickets	\$85/ea	>	

Two Clicks to Checkout

The two click configuration requires the user to view additional ticket details in a pre-checkout screen before proceeding to make a purchase.

This is the default behavior.

Code snippet:

```
<script>
    Seatics.config.skipPrecheckoutDesktop = false;
</script>
```

All	Packages	Parking Passes	A >	Filters (0) v
Promenade Reserved 524	Row 15 2 Tickets		\$70/ea	
Promenade Reserved 528	Row 14 2 Tickets		\$77/ea	
☆ Promenade Reserved In...	Row 10 2 Tickets		\$78/ea	
Promenade Reserved 528	Row 13 2 Tickets		\$81/ea	
Promenade Reserved In...	Row 14 2 Tickets		\$83/ea	
Promenade Reserved 502	Row 16 2 Tickets		\$84/ea	

Show more ticket details

Single Click to Checkout

Set `Seatics.config.skipPrecheckoutDesktop` to true to make it optional for the user to view the additional ticket information. The CTA is given stronger visual emphasis and takes the user directly to checkout. Tapping or clicking elsewhere in the row opens the additional ticket group details.

Code snippet:

```
<script>
    Seatics.config.skipPrecheckoutDesktop = true;
</script>
```

This configuration can also be set through the Ticket List tab in Maps Configuration Management:

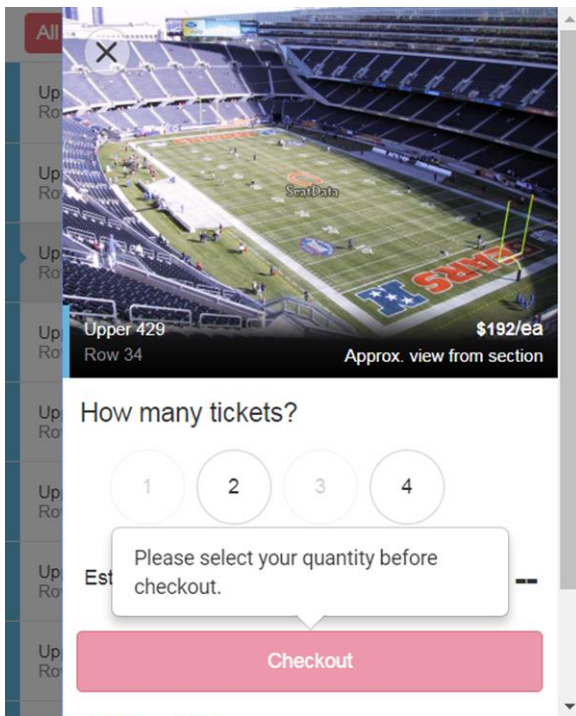
i Single Click Checkout on Large Screen



Specifying a “trigger” element within buyButtonContentHtml

If you are using the single click to checkout configuration (`Seatics.config.skipPrecheckoutDesktop = true`) and overriding the buy button html (`Seatics.config.buyButtonContentHtml`), you can add a class ‘venue-ticket-list-checkout-trigger-js’ to your buy button html. With this class is included, only clicks on the element with that class will result in the user going to checkout. If the ‘venue-ticket-list-checkout-trigger-js’ class is not provided, any click within the CTA table cell (`<td class=“venue-ticket-list-cta-js venue-ticket-list-cta-col”>`) will cause the user to go to checkout.

FORCED QUANTITY SELECTION



Use `Seatics.config.disableBuyButtonNoQuantitySelection` to force the user to choose a ticket quantity before starting checkout. When set to true, the CTA button in the precheckout screen will be inactive until the user specifies the ticket quantity desired. Clicking the CTA button without selecting ticket quantity pops up a tooltip.

This setting should be paired with the two clicks to checkout configuration.

Code snippet:

```
<script>
    Seatics.config.disableBuyButtonNoQuantitySelection =
    true;
</script>
```

CTA IN THE TICKET GROUP ROW

All	Packages	Parking Passes	>	Filters (0) ▾
☆	Lower Bleachers 38 Row 18 • 1-6 Tickets ↕	\$12/ea		Buy
	Lower Bleachers 41 Row 28 • 2-4 Tickets ↕	\$12/ea		Buy
☆	Lower Bleachers 36 Row 20 • 1 Ticket ↕	\$13/ea		Buy
☆	Lower Bleachers 37 Row 19 • 1-3 Tickets ↕	\$13/ea		Buy
☆	Lower Bleachers 37 Row 20 • 1-3 Tickets ↕	\$13/ea		Buy
☆	Lower Bleachers 39	\$13/ea		Buy

Use `Seatics.config.buyButtonContentHtml` to customize the content of the call to action. This parameter injects a text or HTML string into the button tag. Use the dynamic key `{{Price}}` to include the price per ticket.

If the CTA is altered not to include the price, the price will appear to the left of the CTA.

Code snippet:

```
<script>
    Seatics.config.buyButtonContentHtml = '<div
    style="background:#33CC33; color:#ffffff; padding: 8px 5px;
    font-weight:bold; margin-right:5px;">Buy</div>';
</script>
```

This configuration can also be set through the Ticket List tab in Maps Configuration Management:

i TG List CTA Button Buy button configuration

Use Default ☒ Use Custom

FEATURED INVENTORY DISPLAY

Quantity Any Quantity	Price Range \$61 - \$17,492	More Options	
Sort by Price - Low to High			
✱400s Level 404 Row 27 • 2 eTickets	\$61/ea \$67	Buy	
✱400s Level 420 Row 26 • 2 eTickets	\$61/ea \$67	Buy	
✱400s Level 401 Row 27 • 2 eTickets	\$63/ea \$69	Buy	
✱400s Level 401 Row 27 • 2 eTickets	\$63/ea \$70	Buy	
✱400s Level 402 Row 27 • 2 eTickets	\$63/ea \$69	Buy	
✱400s Level 404 Row 26 • 2 eTickets	\$63/ea \$70	Buy	

If the site marks tickets, `Seatics.config.tgMarkHtml` and `Seatics.config.tgMarkTooltipText` can be used to customize the display. `Seatics.config.tgMarkHtml` specifies the symbol, character, or image the site will use to mark featured inventory. The HTML entered for this configuration parameter will be inserted into the mark column of a marked ticket group. The default mark is ✱.

`Seatics.config.tgMarkTooltipText` is used to customize the text that explains what the mark means. There is a limit of 100 characters (including spaces) for this text. The default text is "We recommend this seller's tickets."

Example code:

```
<script>
  Seatics.config.tgMarkHtml = '<span style="vertical-align:middle; padding-left:2px;">&alt;</span>',
  Seatics.config.tgMarkTooltipText = 'These tickets are featured as a great value.';
</script>
```

These configurations can also be set through the Ticket List tab in Maps Configuration Management:

Featured Inventory Display

Use Default ☒ Use Custom

Symbol used to mark featured inventory in the ticket list

Use Default ☒ Use Custom

Tooltip text that displays when a user hovers over the symbol

UNMAPPED INVENTORY DISPLAY

Unmapped inventory may include any of the below ticket types. `Seatics.config.ticketSeparationOptions` gives the site the ability to choose how to handle each type. The “Integrate” option merges the unmapped inventory into the main ticket list. The “Separate” option groups unmapped inventory into segments at the bottom of the main ticket list.

TG Type	TG Type Description	Event Access	Default Setting
1, 2	Unmapped event tickets or suites	Yes	unmappedStandard: <code>Seatics.TicketGroupSeparationOptions.Separate</code> Grouped under heading: Other Offers
1	Unmapped Flex tickets or other multi-section/row listings	Yes	unmappedFlex: <code>Seatics.TicketGroupSeparationOptions.Integrate</code>
3	Parking	No	parking: <code>Seatics.TicketGroupSeparationOptions.Separate</code> Grouped under heading: Parking Passes (Not valid for entry to event)
4	Hotels	No	hotels: <code>Seatics.TicketGroupSeparationOptions.Separate</code> Grouped under heading: Hotel Deals (Not valid for entry to event)
5	Packages	Yes	packages: <code>Seatics.TicketGroupSeparationOptions.Integrate</code>
6-10	Passes (Backstage Passes, Pit Passes, VIP Passes, Meet/Greet Passes, Other Passes)	No	passes: <code>Seatics.TicketGroupSeparationOptions.Separate</code> Grouped under heading: Special Access Passes (Not valid for entry to event)

Some Unmapped Inventory configuration options:

PARKING Row PASS • 1-5 Parking Passes	\$10/ea >
Zone E 1-6 Flex Tickets	\$11/ea >
Lower 27 Row R • 1-8 Instant Download	\$14/ea >
Upper End Zone 1-6 Flex Tickets	\$18/ea >
Zone D 1-6 Flex Tickets	\$28/ea >
Zone C 1-6 Flex Tickets	\$55/ea >
Lower 16 Row K • 1-8 Tickets	\$66/ea >
1 Row B • 2-4 VIP Passes	\$220/ea >
BACKSTAGE Row PASS • 2-4 Backstage Passes	\$303/ea >
VIP Row PACKAGE • 1-7 Ticket Packages	\$385/ea >
VIP Row PASS • 1-15 VIP Passes	\$468/ea >

All Unmapped Inventory Integrated

Zone E 1-6 Flex Tickets	\$11/ea >
Lower 27 Row R • 1-8 Instant Download	\$14/ea >
Zone D 1-6 Flex Tickets	\$28/ea >
Zone C 1-6 Flex Tickets	\$55/ea >
Lower 16 Row K • 1-8 Tickets	\$66/ea >
Ticket Packages	
VIP Row PACKAGE • 1-7 Ticket Packages	\$385/ea >
Parking Passes (Not valid for entry to event)	
PARKING Row PASS • 1-5 Parking Passes	\$10/ea >
Special Access Passes (Not valid for entry to event)	
1 Row B • 2-4 VIP Passes	\$220/ea >
BACKSTAGE Row PASS • 2-4 Backstage Passes	\$303/ea >
VIP Row PASS • 1-15 VIP Passes	\$468/ea >

All Unmapped Inventory Separated

code snippet:

```
<script>
  Seatics.config.ticketSeparationOptions = {
    packages: Seatics.TicketGroupSeparationOptions.Separate,
    parking: Seatics.TicketGroupSeparationOptions.Separate,
    passes: Seatics.TicketGroupSeparationOptions.Separate,
    hotels: Seatics.TicketGroupSeparationOptions.Separate,
    unmappedFlex: Seatics.TicketGroupSeparationOptions.Separate,
    unmappedStandard: Seatics.TicketGroupSeparationOptions.Separate
  };
</script>
```

These configurations can also be set through the Ticket List tab in Maps Configuration Management:

 Unmapped Inventory Display Mix unmapped inventory into the main list or group it at the bottom of the list	Unmapped event/suite ☐ Integrate ☒ Separate Parking ☐ Integrate ☒ Separate Packages ☒ Integrate ☐ Separate	Unmapped FLeX/cats ☒ Integrate ☐ Separate Hotels ☐ Integrate ☒ Separate Special Access Passes ☐ Integrate ☒ Separate
--	---	---

QUANTITY DISPLAY

Infield Grandstand 29 Row 78 1-24 or 26 Tickets	\$37/ea
Field Box B69 Row 62 1-27 or 29 Tickets	\$38/ea
Field Box B82 Row 87 1-25 or 27 Tickets	\$39/ea
Infield Grandstand 31 Row 72 1-12 or 14 Tickets	\$39/ea

In user testing we found that when retail customers filtered the ticket list by quantity, they preferred the quantity line to update to show that desired quantity. However, the `alwaysShowAvailQty` setting allows a site to always show the total available quantity of tickets, if desired.

Code snippet:

```
<script>
    Seatics.config.alwaysShowAvailQty = true;
</script>
```

This configuration can also be set through the Ticket List tab in Maps Configuration Management:

i **Quantity Display**
How to display quantity available when filter is applied

☒ **Filtered quantity**
☐ **Total available**

PRICE DISPLAY

All	FLeX Tickets	Filters (0) ▼
☐	Orchestra Left Row H • 1 Ticket	\$168.00/ea >
☐	Zone A 1-8 FLeX Tickets	\$177.00/ea >
☐	Zone C 1-8 FLeX Tickets	\$180.00/ea >

If you wish to display prices as decimals instead of whole numbers, set `Seatics.config.showCents` to `true` to show dollars and cents.

Code snippet:

```
<script>
    Seatics.config.showCents = true;
</script>
```

This configuration can also be set through the Ticket List tab in Maps Configuration Management:

☒ Price Display
 ☒ Whole dollars
 ☐ Show cents

ENABLING MY LIST

Add up to 10 tickets to My List to compare them
Got it

Filters (0)

☐	Orchestra Left Row K • 1-3 Tickets	\$132/ea
☐	Orchestra Center Row Y • 1-11 Tickets	\$135/ea
☐	Orchestra Center Row X • 1-14 Tickets	\$144/ea
☐	Zone D 1-8 FLeX Tickets	\$145/ea
☐	Zone B 1-8 FLeX Tickets	\$152/ea
☐	Orchestra Center Row W • 1-16 Tickets	\$153/ea

The 'My List' feature allows users to check off and compare ticket groups on mobile and desktop. By default, this feature is disabled on all sites using the MapWidget.

If you wish to have My List enabled on your site, set `Seatics.config.enableMyList` to true.

Please note that the feature does not appear in the mobile landscape layout whether or not the feature is enabled.

Code snippet:

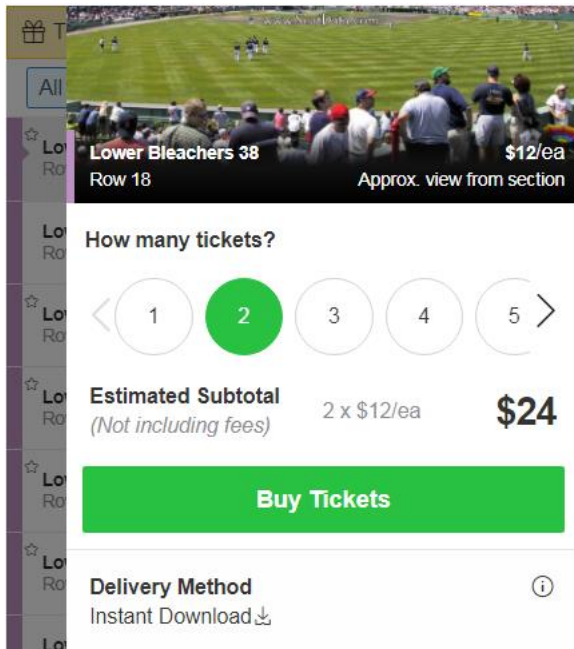
```
<script>
  Seatics.config.enableMyList = true;
</script>
```

This configuration can also be set through the Ticket List tab in Maps Configuration Management:

☒ MyList Feature
 ☐

When enabled, users are able to save and compare ticket groups

PRE-CHECKOUT SCREEN CUSTOMIZATION



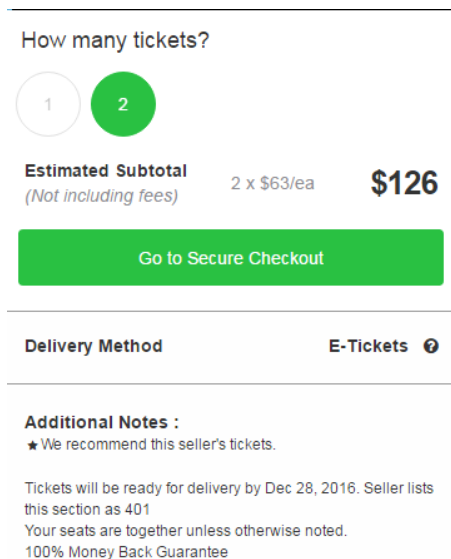
Use `Seatics.config.preCheckoutButtonHtml` to customize the content of the call to action button in the pre-checkout interstitial. This parameter injects a text or HTML string into the button tag.

This button appears only when there are two clicks to checkout, meaning `Seatics.config.skipPrecheckout` is false.

Default text is "Go to Secure Checkout."

Code snippet:

```
<script>
    Seatics.config.preCheckoutButtonHtml = 'Buy
    Tickets';
</script>
```



In addition to having customization options for the pre-checkout CTA button, there is a `tgGuaranteeNoteHtml` setting which allows you to edit the guarantee note in the pre-checkout interstitial. The parameter injects a text or HTML string into the notes field.

Default text is "100% Worry-Free Guarantee."

Code snippet:

```
<script>
    Seatics.config.tgGuaranteeNoteHtml = '100% Money
    Back Guarantee';
</script>
```

These configurations can also be set through the Ticket List tab in Maps Configuration Management. Simply replace the default text with the desired text:

i Precheckout Button Text Buy button configuration	Go to Secure Checkout
i Precheckout Guarantee Note Text	100% Worry-Free Guarantee

DEFAULT SORT

All	FLeX Tickets	Filters (0) v
☆ Orchestra Center	Row D 2-4 Tickets	<i>i</i> \$329/ea \$365 Buy
☆ Orchestra Center	Row D 1 Ticket	<i>i</i> \$329/ea \$365 Buy
☆ Orchestra Center	Row D 2 Tickets	<i>i</i> \$329/ea \$365 Buy
☆ Orchestra Center	Row D 1-3 Tickets	<i>i</i> \$329/ea \$365 Buy
Orchestra Left	Row K 1-3 Tickets	<i>i</i> \$135/ea \$149 Buy
Zone D	1-8 FLeX Tickets	<i>i</i> \$147/ea \$163 Buy
Zone B	1-8 FLeX Tickets	<i>i</i> \$154/ea \$171 Buy

Use `Seatics.config.defaultSort` to specify how ticket groups are to be sorted on page load. The default setting is `Seatics.SortOptions.PriceAsc`, which sorts inventory by price from low to high. Other options:

`Seatics.SortOptions.PriceDesc` (Price – High to Low)
`Seatics.SortOptions.SectionAsc` (Section – Ascending)
`Seatics.SortOptions.SectionDesc` (Section – Descending)
`Seatics.SortOptions.Marked` (Featured Tickets – Show first)

code snippet:

```
<script>
  Seatics.config.defaultSort =
  Seatics.SortOptions.Marked;
</script>
```

This configuration can also be set through the Filters tab in Maps Configuration Management:

Default Sort How inventory is sorted on page load	☒ Price Ascending ☐ Price Descending ☐ Section Ascending ☐ Section Descending ☐ Show Featured Tickets First
---	---

SAVING FILTER SETTINGS

Often users may bounce back and forth between the ticket results page and checkout a few times before making a purchase. To prevent the frustration of having to reset their filter options every time they return to the map, we save the filter settings so the page will appear as the user left it. The filter settings will also persist when the user refreshes the page. If a site owner would like to disable this feature, it can be toggled off by setting `Seatics.config.saveFilterOptions` to `false`.

Code snippet:

```
<script>
    Seatics.config.saveFilterOptions = false;
</script>
```

This configuration can also be set through the Filters tab in Maps Configuration Management:

Save User Filters

☒

Prevents users from having to reset their filters if they return to the maps page from checkout

LOCAL PICKUP FILTER

All

Filters (0) ▾

Orchestra Center

Row H • 1-4 or 6 Tickets ⓘ

\$231/ea >

Local Pickup

Orchestra Center

Row G • 1-3 or 5 Tickets ⓘ

\$256/ea >

Use `Seatics.config.showLocalPickupFilter` to enable delivery icons and filtering for Local Pickup tickets. Local Pickup includes tickets with the delivery methods Will Call, Local Pickup Near Venue, and Last Minute Pickup.

This option is disabled by default. Set to `true` to enable it.

Min Price

231

Max Price

537

Electronic Delivery

☐

Local Pickup

☐

Done

Code snippet:

```
<script>
    Seatics.config.showLocalPickupFilter = true;
</script>
```

PACKAGES MARKETING

To give ticket package offers more visibility, marketing banners were created that act as filters to isolate this premium inventory. There are several options for the display of the banners. These can be configured through the Filters tab in Maps Configuration Management:

☒ **Map Area Packages Banner (large screen)**

HTML when filter is off:
 Use Default ☒ Use Custom

```

<span class="sea-marketing-header-gift cm-package"></span>
<div id="sea-map-marketing-close" class="sea-marketing-header-close cm-close"></div>
<span class="sea-marketing-header-label">Ticket Packages</span>
<span class="sea-marketing-header-label-show">Show</span>
      
```

HTML when filter is on:
 Use Default ☐ Use Custom

```

<span class="sea-marketing-header-gift cm-package"></span>
<div id="sea-map-marketing-close" class="sea-marketing-header-close cm-close"></div>
<span class="sea-marketing-header-label">Ticket Packages</span>
<span class="sea-marketing-header-label-show">Hide</span>
      
```

☒ **Event Info Area Packages Banner (large screen)**

HTML when filter is off:
 Use Default ☒ Use Custom

```

<span class="sea-marketing-header-gift cm-package"></span>
<div id="sea-marketing-header-close" class="sea-marketing-header-close cm-close"></div>
<span class="sea-marketing-header-label">Ticket Packages</span>
<span class="sea-marketing-header-label-show">Show</span>
      
```

HTML when filter is on:
 Use Default ☐ Use Custom

```

<span class="sea-marketing-header-gift cm-package"></span>
<div id="sea-marketing-header-close" class="sea-marketing-header-close cm-close"></div>
<span class="sea-marketing-header-label">Ticket Packages</span>
<span class="sea-marketing-header-label-show">Hide</span>
      
```

1 Ticket List Packages Banner (large screen) ☐

1 Ticket List Packages Banner (mobile) ☐

1 HTML when filter is off:

Use Default ☒ Use Custom

```
<div class="sea-list-marketing-inner">
  <span class="cm-package"></span>
  <span class="sea-list-marketing-label" id="sea-list-marketing-label">Ticket Packages Available</span>
  <span class="sea-list-marketing-action-label" id="sea-list-marketing-action-label">Show Packages</span>
</div>
<div id="sea-list-marketing-close" class="sea-list-marketing-close cm-close"></div>
```

1 HTML when filter is on:

Use Default ☒ Use Custom

```
<div class="sea-list-marketing-inner">
  <span class="cm-package"></span>
  <span id="sea-list-marketing-label">Ticket Packages Available</span>
  <span class="sea-list-marketing-action-label" id="sea-list-marketing-action-label">Hide Packages</span>
</div>
<div id="sea-list-marketing-close" class="sea-list-marketing-close cm-close"></div>
```

Or in code:

Map Banner (large screen)

Seatics.config.enableMapMarketing is used to display packages marketing in the map area of the widget. By default, this is set to false. Use the config setting Seatics.config.mapMarketingHtmlFilter to customize the content of the packages marketing banner and the setting Seatics.config.mapMarketingHtmlAll to customize the content of the banner that returns the user to the main list.

Code snippet:

```
<script>
  Seatics.config.enableMapMarketing = true;
  Seatics.config.mapMarketingHtmlFilter = '<span
class="sea-marketing-header-gift icon-gift"></span><div
id="sea-map-marketing-close" class="sea-marketing-header-
close icon-cancel"></div><span class="sea-marketing-header-
label">Ticket Packages</span><span class="sea-marketing-
header-label-show">Show</span>';
  Seatics.config.mapMarketingHtmlAll = '<span
class="sea-marketing-header-gift icon-gift"></span><div
id="sea-map-marketing-close" class="sea-marketing-header-
close icon-cancel"></div><span class="sea-marketing-header-
label">Ticket Packages</span><span class="sea-marketing-
header-label-show">Hide</span>';
</script>
```




Ticket Packages
[Show](#)

100% Worry-Free Guarantee

- We are a resale marketplace, not the ticket seller.
- Prices are set by third-party sellers and may be above or below face value.
- Your seats are together unless otherwise noted.

+	All	Packages	Parking Passes	A >	Filters (0) v
-					
↺					
	Promenade Reserved 524	Row 15	2 Tickets ↴	i	\$70/ea
	Promenade Reserved 528	Row 14	2 Tickets ↴	i	\$77/ea
	Promenade Reserved In...	Row 10	2 Tickets ↴	i	\$78/ea
	Promenade Reserved 528	Row 13	2 Tickets ↴	i	\$81/ea
	Promenade Reserved In...	Row 14	2 Tickets ↴	i	\$83/ea
	Promenade Reserved 502	Row 16	2 Tickets ↴	i	\$84/ea
	Promenade Reserved 503	Row 12	1-8 Tickets ↴	i	\$85/ea
	Promenade Reserved 503	Row 7	2 Tickets ↴	i	\$85/ea

Event Info Banner (large screen)

`Seatics.config.enableHeaderMarketing` is used to display packages marketing in the event info area of the widget. By default, this is set to `true`. Use the config setting `Seatics.config.headerMarketingHtmlFilter` to customize the content of the packages marketing banner and the setting `Seatics.config.headerMarketingHtmlAll` to customize the content of the banner that returns the user to the main list.

Code snippet:

```
<script>
    Seatics.config.enableHeaderMarketing = true;
    Seatics.config.headerMarketingHtmlFilter = '<span
class="sea-marketing-header-gift icon-gift"></span><div
id="sea-marketing-header-close" class="sea-marketing-header-
close icon-cancel"></div><span class="sea-marketing-header-
label">Ticket Packages</span><span class="sea-marketing-
header-label-show">Show</span>';
    Seatics.config.headerMarketingHtmlAll = '<span
class="sea-marketing-header-gift icon-gift"></span><div
id="sea-marketing-header-close" class="sea-marketing-header-
close icon-cancel"></div><span class="sea-marketing-header-
label">Ticket Packages</span><span class="sea-marketing-
header-label-show">Hide</span>';
</script>
```

🎁 Ticket Packages Available! Hide Packages ×		
All	Packages	Parking Passes A > Filters (0) ▾
Gold-Fan Row Package 1-6 Ticket Packages Note: Gold Fan Package inc		\$235/ea
Silver-Fan Row Package 1-6 Ticket Packages Note: Silver Fan Package in		\$235/ea
Diamond-Fan Row Package 1-6 Ticket Packages Note: Diamond Fan Packag		\$288/ea
Gold-Deluxe Row Package 1-6 Ticket Packages Note: Gold Deluxe Package		\$363/ea
Silver-Deluxe Row Package 1-6 Ticket Packages Note: Silver Deluxe Package		\$363/ea
Diamond-Deluxe Row Package 1-6 Ticket Packages Note: Diamond Deluxe Pack		\$416/ea
STANDARD ☆ Row HOTEL PACKAGE 2-4 Ticket Packages Note: Standard Hotel Pack		\$686/ea

Ticket List Banner (large screen)

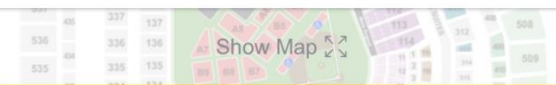
Seatics.config.enableListMarketingDesktop is used to display packages marketing above the ticket list on the large screen layout. By default, this is set to false. Use the config setting Seatics.config.listMarketingHtmlFilter to customize the content of the packages marketing banner and the setting Seatics.config.listMarketingHtmlAll to customize the content of the banner that returns the user to the main list. Content changes will also affect the banner above the ticket list on small screens.

Code snippet:

```
<script>
    Seatics.config.enableListMarketingDesktop = true;
    Seatics.config.listMarketingHtmlFilter = '<div
class="sea-list-marketing-inner vertical-align-middle"><span
class="icon-gift"></span><span class="sea-list-marketing-
label" id="sea-list-marketing-label">Ticket Packages
Available!</span><span class="sea-list-marketing-action-
label" id="sea-list-marketing-action-label">Show
Packages</span><span id="sea-list-marketing-close"
class="sea-list-marketing-close icon-cancel"></span></div>';
    Seatics.config.listMarketingHtmlAll = '<div
class="sea-list-marketing-inner vertical-align-middle"><span
class="icon-gift"></span><span class="sea-list-marketing-
label" id="sea-list-marketing-label">Ticket Packages
Available!</span><span class="sea-list-marketing-action-
label" id="sea-list-marketing-action-label">Hide
Packages</span><span id="sea-list-marketing-close"
class="sea-list-marketing-close icon-cancel"></span></div>';
</script>
```



Lady Gaga
 Citi Field, Flushing, NY
 Mon, Aug 28, 2017 @ 8:30PM


[Show Map](#)


Ticket Packages Available! [Show Packages](#) 

[All](#)
[Packages](#)
[Parking Passes](#)
[Filters \(0\)](#)

Promenade Reserved 524 Row 15 2 Tickets		\$70/ea
Promenade Reserved 528 Row 14 2 Tickets		\$77/ea
Promenade Reserved In... Row 10 2 Tickets		\$78/ea

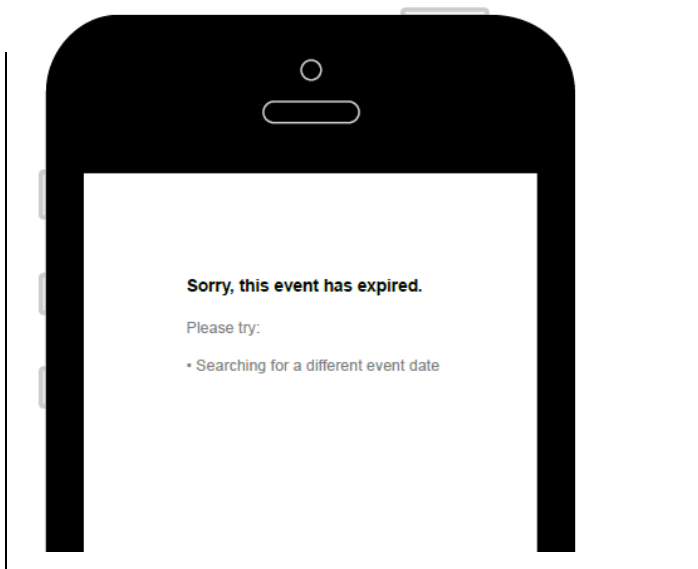
Ticket List Banner (mobile)

Seatics.config.enableListMarketingMobile is used to display packages marketing above the ticket list on the small screen layout. By default, this is set to true. Use the config setting Seatics.config.listMarketingHtmlFilter to customize the content of the packages marketing banner and the setting Seatics.config.listMarketingHtmlAll to customize the content of the banner that returns the user to the main list. Content changes will also affect the banner above the ticket list on large screens.

Code snippet:

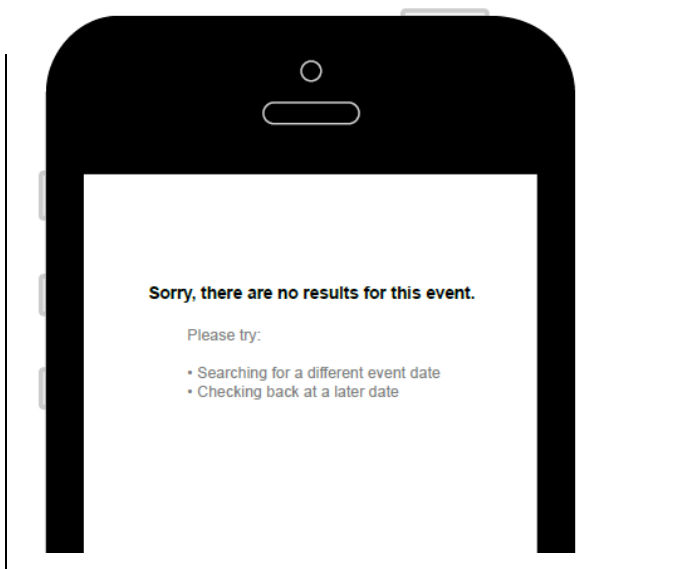
```
<script>
    Seatics.config.enableListMarketingMobile = true;
    Seatics.config.listMarketingHtmlFilter = '<div
class="sea-list-marketing-inner vertical-align-middle"><span
class="icon-gift"></span><span class="sea-list-marketing-
label" id="sea-list-marketing-label">Ticket Packages
Available!</span><span class="sea-list-marketing-action-
label" id="sea-list-marketing-action-label">Show
Packages</span><span id="sea-list-marketing-close"
class="sea-list-marketing-close icon-cancel"></span></div>';
    Seatics.config.listMarketingHtmlAll = '<div
class="sea-list-marketing-inner vertical-align-middle"><span
class="icon-gift"></span><span class="sea-list-marketing-
label" id="sea-list-marketing-label">Ticket Packages
Available!</span><span class="sea-list-marketing-action-
label" id="sea-list-marketing-action-label">Hide
Packages</span><span id="sea-list-marketing-close"
class="sea-list-marketing-close icon-cancel"></span></div>';
</script>
```


NO TICKETS HANDLER



Expired Events

If the user reaches a ticket results page for an event that has already occurred, an error message displays. The site can override this handling with custom handling using `Seatics.config.noEventHandler`—sending the user to a search results page for related events, for example.



No Inventory

If the user reaches a ticket results page for event that has no inventory available, an error message displays. The site can override this handling with custom handling using `Seatics.config.noTicketsHandler`—sending the user to a search results page for related events, for example.

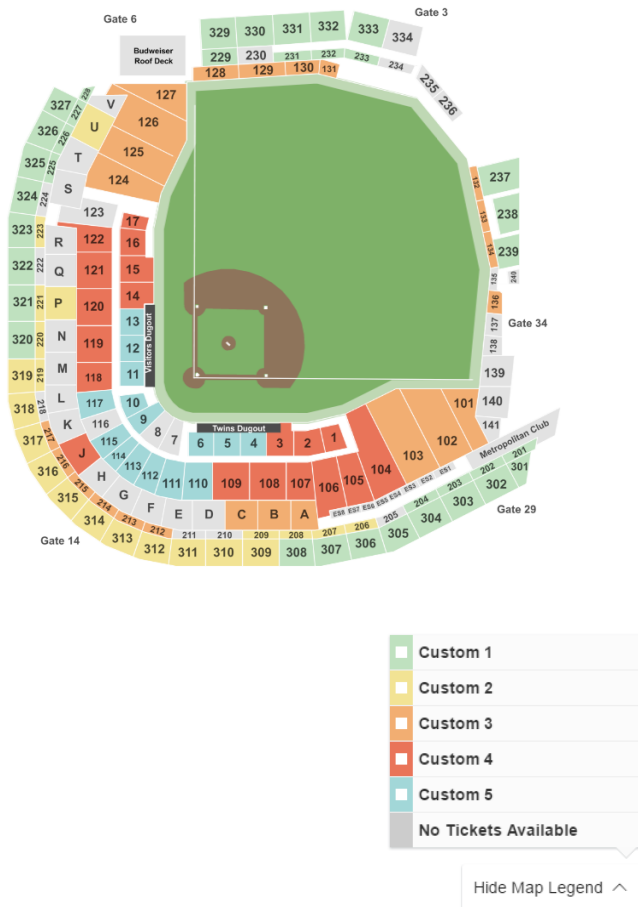


SPECIAL FUNCTIONS

`Seatics.config.onBuyButtonClicked` is a function that will execute after the user clicks the call to action button but before the user is redirected to checkout. It can be used for tracking that the user clicked the 'Buy' button.

`Seatics.config.mapFinishedRenderingHandler` is a function that will execute after the map is displayed and all UI elements have been loaded. It can be used to track interactions with the UI. Because of the way the ticket group list is loaded, some additional jQuery is needed to listen for interactions on the list.

CUSTOM LEVELS



Generally, when venue maps are created, sections are grouped into the same levels as are defined on maps released by the primary market. This one size fits all approach works for most Seatics Maps users because we provide venue layouts consistent with others in the marketplace. However, some maps clients want a more simplified organization of sections. On the other end of the spectrum, other clients want more granularity.

Seatics.config.customLevels allows the site to define custom levels. Create your own uniquely defined level by giving it a name and assigning an array of sections to it. You can also specify a color for the level (in HTML format), or allow the new level to use the default palette. If the level codes specified by our cartographers are available to you, you can use those to include a full venue level in your new custom level instead of listing each section in that level individually.

When you create custom levels, all sections in the venue must be assigned to newly defined levels. If you want some levels to remain the same as in the default map, you will have to redefine them when creating your custom levels.

Example code:

```
<script>
  Seatics.config.customLevels=[{ name:"Custom 1", sections:["201", "202", "203", "204", "224", "225", "226", "227",
    "228", "308", "320", "321", "322" ], levels: ["Hm$Rn$Porch$Vw", "GS",
    "Budweiser$Roof$Deck", "Hm$Rn$Porch$Terr", "Fld$Vw", "Skyline$Vw", "Met$Clb"],
    color:"#bfe1bf"},
    { name:"Custom 2", sections:["P", "Q", "R", "S", "T", "U", "V", "219", "220", "221",
    "222", "223", "205", "206", "207", "208", "209", "309", "310", "311", "312", "313",
    "314", "315", "316", "317", "318", "319"], levels: ["Event$Ste"], color:"#f2e394"},
    { name:"Custom 3", sections: ["123", "124", "A", "B", "C", "M", "N", "201", "211",
    "212", "213", "214", "215", "216", "217", "218"], levels: ["Fld$Bx", "L$Fld$B1",
    "Overlook", "The$Pav", "R$Fld$B1"], color:"#f2ae72"},
    { name:"Custom 4", sections: ["1", "2", "3", "14", "15", "16", "17", "104", "105",
    "106", "107", "108", "109", "118", "119", "120", "121", "122", "D", "E", "F", "G", "H",
    "J", "K", "L"], color:"#e97459"},
    { name:"Custom 5", sections: ["4", "5", "6", "11", "12", "13", "110", "111", "112",
    "113", "114", "115", "116", "117"], levels: ["Champ$Clb"], color:"#a2d7d8"}];
</script>
```

CUSTOMIZING SEATICS.TEXT

The Seatics libraries contain additional text used to build the map, ticket list, and other page components. We only recommend changing this text when absolutely necessary and keeping the changes as minor as possible. As we develop new features we may add new text, or adjust the language in the existing text. If you are using a Custom UI please ignore this section as the way we load the Text object is different and the text itself will not generally apply to you.

If you are using the Pre-Built UI and you want to make changes to the text, please code them as described below to ensure compatibility with future releases. In order to keep maps performant, we initialize pieces of the text at different times (depending on when that text is shown in the UI). We may rearrange the Text object from time to time as the UI changes or remove text that is no longer applicable, but if you follow the guidance here you will never run into an issue.

The code to adjust the Text contents should be executed immediately after the MapAndLayout javascript.

Example code:

```
<script src="https://www.tn-apis.com/maps/v3/MapAndLayout?websiteConfigId=xxx&consumerKey=xxxxxx&eventName=xxx&venue=xxx&dateTime=xxx"></script>
<script>
    // Seatics.Text is a javascript object. You can use dot-notation or string-based property access to
    adjust individual pieces of text
    Seatics.Text.DeepLink_CloseModalButton = "Select new tickets"; // specify completely new text
    Seatics.Text["Desktop_DisclosuresTextCurrency"] = function(text){ return text.replace("USD",
    "Points"); }; // or specify a function that will adjust the contents of the text when it's initialized
</script>
```

Do not try and do a replace on a value in the text directly after calling MapAndLayout as the values in Text will not be loaded yet and you will get an undefined error. (i.e. don't do `Seatics.Text["Desktop_DisclosuresTextCurrency"] = Seatics.Text["Desktop_DisclosuresTextCurrency"].replace("USD", "Points");`).

CHANGE TABLE

Contributor Name	Description of Change	Change Date
Stacey Bednarz	Added change table, fixed typos, shifted order of configs, added face value configs	3/19/18
Stacey Bednarz	Added event mapping explanation	4/19/18
Stacey Bednarz	Added note for mobile delivery identification	6/8/18
Stacey Bednarz	Edited checkout integration section, added note about admin app	7/12/18
Stacey Bednarz	Major overhaul to configuration details in Appendix	7/27/18
Stacey Bednarz	Numerous edits to custom UI instructions, added information about disclaimer pop-ups	12/19/18
Stacey Bednarz	Added instructions for additional configuration options	1/21/2020
Sarah Knowles	Added checkout deep linking, updated inventory disclaimer screenshots	5/17/2021
Sarah Knowles	Added instructions for updating Seatics.Text and use of the ticket group bitmap for custom splits	6/28/2021
Sarah Knowles	Added documentation for missing configuration option <code>Seatics.config.levelColorsSmallVenue</code> .	12/29/2021
Sarah Knowles	Added documentation about how to add tickets using Mercury REST API	6/24/2022
Sarah Knowles	Updated earliest supported jQuery version	9/12/2022
Sarah Knowles	Added currency symbol and currency globalization configurations	10/4/2022
Sarah Knowles	Added H1 tag configuration	2/20/2023
Sarah Knowles	Updated/extended configuration documentation for Face Value. Added documentation for All-In Pricing and Service Fee display options. Added documentation for how to provide service fee information within the Ticket Data.	4/6/2023
Sarah Knowles	Expanded the section pop-up documentation to include instructions for <code>Seatics.config.largeScreenFormat</code> and <code>Seatics.config.hoverEnabled</code> , as well as how to manually control pop-up display.	7/22/2024
Sarah Knowles	Fixed the information for how Pre-Built UI integrations should add ticket data to the Map. The old documentation was using the wrong method, which resulted in some Pre-Built UI elements not initializing properly. Affected elements include: ticket types tabs, the Filter card button, the Feedback button, and the Select-by-level dropdown list.	1/15/2025
Sarah Knowles	Added documentation for the new <code>Seatics.config.showFeesIncludedText</code> property. Added an example of how to build the most common <code>tgServiceFeeCalculation</code> string.	6/2/2025

